Distillation, Low-Rank and Tensor Factorization, Parameter Sharing, and Lossless Weight Compression for Large Language Models

A Monograph-Style Survey, Series Paper 4

Literature coverage through October 2026

29 October 2026

Scope note. This document is a survey written from public literature and web sources. Numbers attributed to a source are quoted from that source's abstract or the page the author could read; numbers marked "typical", "approximate" or "illustrative" are the author's own round figures or synthetic examples and are not measurements. Priority scores in Chapter 19 are subjective judgments.

Abstract

Compressing a large language model can mean very different things. A student can be trained to imitate a teacher, a weight matrix can be replaced by a product of thin factors or by a structured or tensorized operator, layers can be made to share parameters and be applied repeatedly, a family of fine-tuned checkpoints can be stored as a base plus tiny deltas, and the bits of an unchanged model can be entropy-coded without altering a single output. This survey treats these families under one mathematical roof: rate, distortion, and the computational cost of decoding. We present the theory of knowledge distillation (soft targets, temperature, forward and reverse KL, f- and Jensen–Shannon divergences, sequence-level and on-policy objectives), the rise of black-box and reasoning-trace distillation, and the legal and security questions that follow. We derive truncated-SVD optimality, activation-aware and whitened SVD for LLM layers, heterogeneous rank allocation, low-rank-plus-sparse and low-rank-plus-quantized decompositions, and the Kronecker, Monarch, butterfly, tensor-train and hashing families. We analyse cross-layer sharing, looped and recursive transformers, mixture-of-recursions, and the dense-versus-MoE parameter-efficiency framing. We examine merging and delta compression for model families, small-model scaling as the baseline compression must beat, and lossless coding of floating-point weights, whose gains come from the low entropy of exponent bits. We connect all of this to the view of language models as compressors, minimum description length, and compression-based generalization bounds. We close with a timeline from 2022 to October 2026, a table of what is achievable, a catalogue of risks, a decision guide, and a transparent, explicitly subjective priority assessment.

Contents

Abstract ii

1 Executive Summary of Findings 1

1.1 The one-paragraph version . . . 1
1.2 Summary table of method families . . . 2
1.3 What to do, in brief . . . 2
1.4 How confident are we . . . 4

2 Introduction and Motivation 5

2.1 Why compress language models at all . . . 5
2.2 Scope and relation to other methods . . . 6
2.3 A unifying view: rate, distortion, and decode cost . . . 6
2.4 A short history . . . 7
2.5 Four recurring themes . . . 7
2.6 Methodology of this survey . . . 8

3 Mathematical Foundations 9

3.1 Entropy, cross-entropy and divergences . . . 9
3.2 Sequence models and code length . . . 9
3.3 Matrix approximation . . . 10
3.3.1 SVD and the Eckart–Young–Mirsky theorem . . . 10
3.3.2 Stable rank and spectral decay . . . 11
3.3.3 Whitened approximation . . . 11
3.3.4 Randomized SVD . . . 11
3.4 Rate–distortion theory . . . 12
3.5 Source coding . . . 12
3.6 Floating-point formats . . . 13
3.7 Generalization and compression . . . 13
3.8 Notation summary . . . 13

4 Taxonomy of Compression Approaches 14

4.1 Axes of classification . . . 14
4.2 Comparison across axes . . . 15

4.3 What each family leaves unchanged . . . 15
4.4 Combinations . . . 16
4.5 Decision-relevant dimensions . . . 16

5 Knowledge Distillation I: Theory 18
5.1 Soft targets and temperature . . . 18
5.2 Why does distillation help? . . . 19
5.3 Choice of divergence . . . 20
5.3.1 Forward KL . . . 20
5.3.2 Reverse KL . . . 20
5.3.3 An exactly solvable case . . . 20
5.3.4 Jensen–Shannon and skewed divergences . . . 21
5.3.5 General f-divergences . . . 21
5.4 Intermediate representations . . . 21
5.5 The capacity gap . . . 22
5.6 Sequence-level distillation . . . 23
5.7 Exposure bias and on-policy training . . . 23
5.8 Summary . . . 23

6 Knowledge Distillation II: Methods for Language Models 24
6.1 Cost accounting for distillation . . . 24
6.2 Distillation inside frontier-lab training pipelines . . . 25
6.3 On-policy distillation . . . 25
6.3.1 GKD: generalized knowledge distillation . . . 25
6.3.2 MiniLLM: reverse KL with policy gradients . . . 26
6.3.3 DistiLLM and skewed objectives . . . 26
6.3.4 On-policy distillation as dense-reward RL . . . 26
6.4 Matching hidden states and attention in LLMs . . . 27
6.5 Pruning plus distillation . . . 27
6.6 Distillation for speculative decoding . . . 28
6.7 Tokenizer-mismatch distillation . . . 28
6.8 Multi-teacher and self-distillation . . . 29
6.9 Data selection for distillation . . . 29

7 Knowledge Distillation III: Black-Box, Synthetic-Data and Reasoning Distillation 31
7.1 White-box, grey-box and black-box access . . . 31

7.1.1 How much does a sample reveal? . . . 31
7.2 The instruction-imitation era (2023) . . . 32
7.3 Chain-of-thought distillation . . . 32
7.4 The distilled-reasoning-model wave (2025–2026) . . . 33
7.4.1 What is being transferred? . . . 33
7.4.2 Cost of producing a distilled reasoning model . . . 33
7.4.3 On-policy and RL stages after SFT . . . 34
7.5 Capacity gap and style compatibility . . . 35
7.6 Safety, alignment and bias in students . . . 35
7.7 Legal, terms-of-service and IP considerations . . . 36
7.8 Model collapse and synthetic-data loops . . . 37
7.9 Summary . . . 37

8 Low-Rank Factorization of Weights 38
8.1 Parameter and FLOP accounting . . . 38
8.2 Why plain SVD fails on LLM weights . . . 38
8.3 Data-aware low-rank methods . . . 39
8.3.1 FWSVD: Fisher-weighted SVD . . . 39
8.3.2 ASVD: activation-aware scaling . . . 39
8.3.3 SVD-LLM: truncation-aware whitening . . . 40
8.3.4 SVD-LLM V2 and Dobi-SVD . . . 40
8.3.5 Cross-layer basis sharing and dictionary learning . . . 41
8.3.6 SliceGPT: low-rank in disguise . . . 41
8.4 Error propagation and bounds . . . 41
8.5 Rank selection and heterogeneous allocation . . . 42
8.6 Healing and recovery . . . 43
8.7 Low-rank plus sparse . . . 43
8.8 Low-rank plus quantization . . . 44
8.9 LoRA, adapters and low-rank structure of updates . . . 44
8.10 Low-rank structure in activations and the KV cache . . . 45
8.11 Summary of the low-rank landscape . . . 45

9 Structured Matrices, Tensor Decompositions, Hashing and Embedding Compression 46
9.1 Framework: structure, parameters, and matvec cost . . . 46
9.2 Kronecker products . . . 47
9.3 Circulant, Toeplitz and displacement structure . . . 47

9.4 Butterfly and Monarch . . . 47
9.4.1 Butterfly factorizations . . . 47
9.4.2 Monarch matrices . . . 48
9.5 Tensor decompositions . . . 48
9.6 Hashing . . . 49
9.7 Embedding tables and vocabularies . . . 50
9.7.1 The share of parameters in embeddings . . . 50
9.7.2 Factorized embeddings . . . 50
9.7.3 Tensorized and hashed embeddings . . . 51
9.7.4 Vocabulary and tokenizer size as compression choices . . . 51
9.8 Evidence and assessment . . . 51

10 Parameter Sharing and Recurrence 52
10.1 Forms of sharing . . . 52
10.2 Accounting: what sharing saves and what it does not . . . 52
10.3 Relaxed Recursive Transformers . . . 53
10.4 Looped transformers and recurrent depth . . . 54
10.4.1 Theory: depth without parameters . . . 54
10.4.2 Recurrent depth and test-time scaling . . . 54
10.4.3 Ouro . . . 54
10.5 Mixture-of-Recursions . . . 55
10.6 Mixture-of-experts versus dense versus recursive: a common map . . . 56
10.7 Assessment . . . 56

11 Model Families: Merging, Delta Compression, Small-Model Scaling and NAS 58
11.1 Task vectors and merging . . . 58
11.2 Delta compression . . . 59
11.2.1 BitDelta: one bit per delta weight . . . 59
11.2.2 Mixed-precision and low-rank deltas . . . 59
11.2.3 Multi-tenant adapter serving . . . 60
11.3 The baseline: train a small model well . . . 60
11.3.1 Compute-optimal and inference-aware scaling . . . 60
11.3.2 Distillation versus training from scratch . . . 61
11.3.3 Pruning, NAS and hardware-aware search . . . 62
11.4 Putting the baseline to work . . . 62

12 Lossless Compression of Weights 63

12.1 Where the redundancy lives . . . 63
12.1.1 A derivation for bell-shaped weights . . . 63
12.1.2 The ceiling . . . 64
12.2 Entropy coders . . . 65
12.3 Decoding on a GPU: the systems problem . . . 66
12.3.1 Why variable-length codes are awkward . . . 66
12.3.2 Traffic analysis . . . 66
12.3.3 When does lossless coding pay off? . . . 67
12.4 Training-time and checkpoint compression . . . 67
12.5 Lower-precision formats: less to gain . . . 68
12.6 Entropy-coded quantized weights . . . 68
12.7 Lossless versus lossy: a decision frame . . . 69

13 Language Models as Compressors, MDL and Generalization Bounds 70
13.1 Prediction is compression . . . 70
13.2 Empirical results on LMs as compressors . . . 71
13.3 Kolmogorov complexity, MDL and the prequential view . . . 71
13.4 Generalization bounds via compression . . . 72
13.4.1 Occam and PAC-Bayes . . . 72
13.4.2 From small networks to LLMs . . . 73
13.4.3 Intrinsic dimension . . . 73
13.5 Information bottleneck . . . 73
13.6 Scaling, precision and Pareto frontiers . . . 74
13.7 Summary . . . 75

14 Systems Considerations and Evaluation Methodology 76
14.1 Roofline reasoning for LLM inference . . . 76
14.2 When does low-rank win wall-clock? . . . 77
14.2.1 The model . . . 77
14.2.2 Prefill and compute-bound regimes . . . 78
14.2.3 Other considerations . . . 78
14.3 Memory-format overhead . . . 78
14.4 KV cache and long contexts . . . 78
14.5 System costs of distillation and recurrent models . . . 79
14.6 Evaluation methodology . . . 79
14.6.1 What to measure . . . 79
14.6.2 Statistical power . . . 80

14.6.3 Contamination, judges and imitation traps . . . 80
14.6.4 A reporting checklist . . . 81

15 Recent Advances: A Timeline from 2022 to October 2026 82
15.1 Timeline table . . . 82
15.2 Trends and inflection points . . . 84
15.2.1 From imitation to principled on-policy distillation . . . 84
15.2.2 Distilled reasoning as the dominant "compression" story . . . 84
15.2.3 Post-hoc low-rank: from SVD to activation-aware to hybrid . . . 84
15.2.4 Recursion returns through reasoning . . . 84
15.2.5 Lossless coding becomes a GPU-kernel topic . . . 84
15.2.6 Security and legal turn . . . 85
15.2.7 A sketch . . . 85

16 Capabilities: What Is Achievable Today 86
16.1 Distillation and derived small models . . . 86
16.2 Low-rank, structured and tensor methods . . . 86
16.3 Sharing, recursion and looping . . . 87
16.4 Delta compression, merging and serving . . . 87
16.5 Lossless compression . . . 87
16.6 Cost to produce . . . 87
16.7 What practitioners can count on, with ranges . . . 87

17 Pros, Cons and Risks 95
17.1 Strengths and weaknesses by family . . . 95
17.2 Risk register . . . 95
17.2.1 Capability and skill loss hidden by benchmark parity . . . 96
17.2.2 Distribution shift . . . 96
17.2.3 Hallucination increase . . . 96
17.2.4 Safety and alignment transfer or loss . . . 97
17.2.5 License, ToS and IP exposure . . . 97
17.2.6 Teacher-bias inheritance . . . 97
17.2.7 Model collapse from synthetic data . . . 97
17.2.8 Benchmark contamination . . . 97
17.2.9 Low-rank brittleness . . . 98
17.2.10 The no-wall-clock-speedup trap . . . 98
17.2.11 Entropy-decoding overhead . . . 98

17.2.12 Additional risks . . . 98
17.3 Risk matrix . . . 99

18 Practical Decision Guide **100**
18.1 Flowchart . . . 100
18.2 Decision tables . . . 100
18.3 Worked scenarios . . . 101
18.3.1 Scenario A: a 70B model on one 80 GB GPU . . . 101
18.3.2 Scenario B: a 1–3B model for on-device use . . . 102
18.3.3 Scenario C: 500 customer fine-tunes of a 7B base . . . 102
18.3.4 Scenario D: distributing 140 GB of weights to 1,000 nodes . . . 102
18.3.5 Scenario E: a small reasoning model from an API-only teacher . . . 102
18.4 Rules of thumb . . . 102

19 Estimated Priority in LLM Engineering **104**
19.1 Rubric . . . 104
19.2 Scores . . . 105
19.3 Justification of the scores . . . 105
19.4 Final ranking, with confidence . . . 107

20 Open Problems and Conclusions **109**
20.1 Open problems . . . 109
20.1.1 Distillation . . . 109
20.1.2 Low-rank, structured and hybrid . . . 109
20.1.3 Sharing and recursion . . . 110
20.1.4 Lossless and entropy-coded compression . . . 110
20.1.5 Theory, evaluation and system co-design . . . 111
20.2 Conclusions . . . 111

A Notation and Glossary **113**
A.1 Notation . . . 113
A.2 Glossary . . . 113

B Selected Derivations **116**
B.1 Policy gradient of the reverse KL (MiniLLM) . . . 116
B.2 Correctness of speculative sampling . . . 116
B.3 Hoeffding–Occam bound . . . 117
B.4 Entropy of a two-sided geometric exponent . . . 117

B.5 Entropy-coded scalar quantization gap 117
B.6 Variance of the signed hashing estimator 117
B.7 Parameter count of Monarch and tensor-train layers 118
B.8 Break-even in the roofline model of factorized layers 118

Chapter 1

Executive Summary of Findings

This chapter condenses the survey into the claims the author believes are best supported, together with the confidence attached to each. Numbers are attributed to the cited source; where a figure is the author's rounding or an order-of-magnitude estimate it is marked as such. Later chapters give derivations and caveats.

1.1 The one-paragraph version

Of the four families covered here, only two are now routine in production LLM pipelines. *Distillation*, especially on-policy and reasoning-trace distillation, is the default way small and mid-sized frontier-lab models are produced and post-trained: the Gemma 2/3 reports, Meta's Llama 3.2 announcement, and the Qwen3 report all describe distillation from larger models as an ingredient in building smaller ones [47–49, 51]. *Lossless entropy coding of BF16/FP8 weights* is a young but low-risk technique whose gains are bounded (roughly 30% for BF16, because the exponent field carries far fewer than its 8 nominal bits of information) and which is attractive precisely because it changes nothing about model outputs [200, 201, 203, 204]. *Low-rank factorization* of dense weights is a useful tool but, for post-training compression of already-trained LLMs, has remained a niche behind quantization: it works best in combination (low-rank plus quantization, low-rank KV latents as in Multi-head Latent Attention [104], LoRA-style deltas) rather than as a standalone way to shrink a pretrained dense matrix. *Parameter sharing and recurrence* are an active research frontier, with a clear compute-versus-memory trade-off and growing evidence of value for reasoning, but only limited production adoption so far [147–149].

Key findings at a glance

1. **Distillation has shifted from imitation to on-policy, dense-reward training.** The 2023–2024 results (GKD, MiniLLM, DistiLLM) established that sampling from the student and scoring with the teacher removes the train/inference mismatch of offline distillation [22–24]. In October 2025 a public write-up reported that on-policy distillation reached roughly 70% on AIME'24 from a 60% SFT starting point in about 150 steps, and cited the Qwen3 report for a pipeline whose on-policy stage used 1,800 GPU hours against 17,920 for the RL stage [47, 52].
2. **Reasoning distillation is data-efficient but capacity-gated.** Roughly 800k teacher samples sufficed for the DeepSeek-R1 distilled family [41]; only about a thousand curated traces were enough to induce long-reasoning style in s1 and LIMO [42, 43]. But small students can fail to learn from very strong reasoners [46], and on-policy distillation works best when student and teacher share compatible reasoning styles [59].
3. **Distillation is also a legal, security and safety surface.** Terms-of-service vio-

lations, extraction campaigns (Anthropic reported over 16 million exchanges through about 24,000 fraudulent accounts in a February 2026 allegation [63]), hidden-signal trait transfer [64], and model collapse from recursive synthetic data [65] are not edge cases but first-order risks.

4. **Weight-space low-rank compression needs activation awareness.** Plain truncated SVD is Frobenius-optimal (Eckart–Young–Mirsky [86, 87]) but is the wrong objective for LLM layers, whose inputs are highly anisotropic. Whitening by the activation Gram matrix (SVD-LLM and relatives [93–96]) turns the truncation loss into the true output error, and is the single most important practical idea in this family.
5. **Low-rank rarely wins wall-clock at batch size one unless the rank is small.** A matrix $m \times n$ replaced by rank-r factors saves memory traffic only if $r < mn/(m+n)$; at square sizes the break-even rank is $n/2$, and kernel-launch and memory-format overheads move it lower (Chapter 14).
6. **Parameter sharing converts memory savings into a compute bill.** Looped and recursive models cut unique parameters but not FLOPs; relaxed variants add layer-wise LoRA to recover capacity [147]. Mixture-of-Recursions adds token-level routing of recursion depth at scales of 135M to 1.7B parameters [148]. Whether sharing beats simply training a smaller dense model under a total-FLOP budget remains an open question at scale.
7. **Lossless compression is real but bounded.** Reported results: DFloat11 about 70% of original BF16 size with bit-identical outputs and kernel support for GPU decoding [200]; ZipNN around 33% savings on popular models [201]; exponent-concentration work (ECF8) giving 9.8–14.8% memory savings on FP8 LLMs, up to 26.9% on a video-generation model, and a theoretical limit near FP4.67 [203]; ZipServ reporting up to 30% size reduction and about 1.22× mean end-to-end speedup over vLLM [204].
8. **The baseline keeps moving.** Over-trained small models (Llama 3 8B on about 15T tokens [50]), Densing-law trends [176], and pruning-plus-distillation (Minitron, Puzzle [60, 61]) set a high bar: a compression method must beat *training or distilling a smaller model of equal footprint*, not just a naive baseline.

1.2 Summary table of method families

Table 1.1 gives the author's synthesis. The column "typical size reduction" lists the *order of magnitude commonly reported or reasonable to expect*, not a benchmark; the cost column is the author's qualitative estimate.

1.3 What to do, in brief

1. If you need a smaller model and have training budget: distill (on-policy where possible) into a student at or near the target size, possibly after structured pruning (Chapters 6 and 11).
2. If you need to ship the *same* model in less memory with zero behaviour change: entropy-code the weights, then decode on the fly inside the GEMM kernel only if the kernel exists for your

Table 1.1: Method families at a glance. Ratios are typical orders of magnitude (author's synthesis from the sources cited in later chapters), not measurements of a single benchmark.

Family	Typical size reduction	Output change	Production cost	Maturity (author's view)
Distillation (logit / on-policy)	2–100× params via smaller student	Changes model	High (training), but amortized	Mature; default for small models
Reasoning-trace distillation	Same as above	Changes model, can transfer skills	Medium (teacher sampling dominates)	Mature, ToS-sensitive
Post-hoc low-rank (SVD-type)	1.1–2× for 20–50% parameter cut	Lossy, degrades with ratio	Low (minutes–hours)	Research / niche
Low-rank + quantized	Beyond 4-bit effective	Lossy	Low–medium	Research, niche
Structured / tensor	2–10× if trained from scratch	Changes model	High (needs training)	Research; Monarch-style niche
Parameter sharing / recurrence	2–4× unique params	Changes model	Medium–high (uptraining)	Frontier research
Delta / merge compression	>10× per fine-tune delta (BitDelta reports >10× memory reduction)	Lossy but small	Low	Production for multi-tenant serving
Lossless entropy coding	≈1.3–1.5× (BF16)	**None**	Low–medium (kernel work)	Early production

hardware (Chapter 12).

3. If you serve many fine-tunes of one base: store deltas (low-rank or 1-bit-sign plus scale) and use multi-tenant adapter serving (Chapter 11).

4. If you want to try low-rank on a frozen model: whiten with calibration activations, allocate ranks heterogeneously, add a short LoRA healing step, and measure *wall-clock*, not parameter count (Chapters 8 and 14).

5. If you work on reasoning: treat looped computation as a way to buy extra depth per parameter, not as a compression method in the narrow sense (Chapter 10).

Principal risks in one place

Benchmark parity can hide skill loss; synthetic-data loops can narrow the model's distribution; students may inherit teacher bias or alter safety behaviour; terms of service on teacher APIs may forbid use of outputs to train competing models; low-rank models can degrade sharply past a threshold ratio; a smaller parameter count may bring no wall-clock gain; and entropy decoding adds latency unless fused into the compute kernel. These are developed in Chapter 17.

1.4 How confident are we

We assign confidence labels in the priority chapter (Chapter 19). The short version: high confidence that on-policy distillation and delta-serving are net-positive and durable; medium confidence that lossless weight coding becomes standard in model-distribution and some serving stacks; medium-low confidence that low-rank post-hoc compression becomes mainstream on its own; and low confidence about whether recursion and mixture-of-recursions displace dense or MoE designs at frontier scale. None of these are measured quantities; they are the author's judgment given the cited evidence.

Chapter 2

Introduction and Motivation

2.1 Why compress language models at all

A dense decoder-only transformer with P parameters stored in 16-bit floating point needs $2P$ bytes of weights. A 70-billion-parameter model therefore occupies about 140 GB before a single key–value cache entry is allocated, which already exceeds the 80 GB of HBM on a single current-generation accelerator. During autoregressive decoding at small batch size each generated token must stream essentially all active weights from memory to the compute units once, so token latency is bounded below by roughly bytes moved/memory bandwidth. With a bandwidth of about 3 TB/s (an approximate figure for a recent high-end datacenter GPU) and 140 GB of weights, the weight-streaming floor is around 47 ms per token for a single sequence on a single device if the weights could be spread on one device; sharding across devices divides this floor but adds communication. The arithmetic is deliberately rough, yet it explains why every bit removed from the weight stream shows up in latency and cost.

Example 2.1 (Weight-streaming floor, illustrative arithmetic)**.** Let a model have $P = 8 \times 10^9$ parameters in BF16 (2 bytes), so 16 GB. At an assumed 2 TB/s effective bandwidth the floor is $16/2000\,\mathrm{s} = 8$ ms/token, about 125 tokens/s per sequence at batch size one. Shrinking the weights by a factor ρ (whatever the mechanism) lowers the floor to $8/\rho$ ms only if the decoder of the compression scheme is free. A 30% lossless reduction gives $\rho = 1/0.7 \approx 1.43$ and a floor of about 5.6 ms; a 4-bit quantizer gives $\rho \approx 4$ and about 2 ms; a student with a quarter of the parameters in BF16 also gives $\rho = 4$. These numbers are **illustrative**; real systems are limited by KV-cache traffic, kernel efficiency, and communication long before they reach the floor.

Compression matters for five distinct reasons, each favouring a different method family.

1. **Serving cost.** Smaller or lower-bandwidth models cost less per token. This favours distillation (fewer parameters *and* fewer FLOPs), quantization (fewer bytes), and sometimes lossless coding (fewer bytes, same FLOPs).

2. **Fit.** On-device and single-GPU deployment require a hard memory cap. Here the question is feasibility rather than cost, and lossy methods are tolerated if quality is acceptable.

3. **Storage and distribution.** Model hubs, container registries and checkpoint stores move petabytes of weights. Lossless compression pays off even if decoding happens only at load time [201].

4. **Multi-tenancy.** A provider serving thousands of customised variants of a base model cannot afford a full copy per variant; delta compression and adapter serving make this tractable [167, 171].

5. **Understanding.** Compression is also a lens: the minimum description length of a model and its fit to data bound generalization [219, 227], and the predictive loss of an LLM is literally a code length [211].

2.2 Scope and relation to other methods

This paper concerns methods that are not primarily *pruning* or *quantization*. It covers:

- **Distillation** in all its LLM forms (Chapters 5 to 7).
- **Low-rank factorization** and its hybrids (Chapter 8), and **structured and tensorized operators and hashing** (Chapter 9).
- **Parameter sharing and recurrence** (Chapter 10), and **model-family compression** via merging, deltas and small-model scaling (Chapter 11).
- **Lossless weight coding** (Chapter 12), and the theory connecting compression with learning (Chapter 13).
- **Systems and evaluation** questions that decide whether any of the above pays off (Chapter 14).

Pruning and quantization are treated in companion papers of the series; here they appear only where they interact with the topics at hand (for example low-rank plus quantization, entropy-coded quantized weights, or pruning followed by distillation [61]). We cite quantization and pruning algorithms such as GPTQ, AWQ and SparseGPT [186–188] as baselines.

2.3 A unifying view: rate, distortion, and decode cost

All methods can be located in a three-dimensional space.

Definition 2.1 (Rate, distortion, decode cost)**.** For a model f_θ compressed into a representation c with decoder D producing $\hat{\theta} = D(c)$:

- the *rate* $R = |c|$ is the number of bits stored (or transmitted, or resident in HBM—these differ);
- the *distortion* Δ is a measure of functional change, such as the expected KL divergence between output distributions, task loss increase, or bit-exact equality ($\Delta = 0$);
- the *decode cost* C_D is the computation (and memory traffic) required to use $\hat{\theta}$ at inference, which may be paid once at load time, per forward pass, or fused into the matrix multiply.

Lossless compression sits on the $\Delta = 0$ plane and trades rate against C_D. Low-rank factorization changes the *function* (nonzero Δ) but decreases both R and FLOPs when r is small. Distillation does not compress a particular θ at all: it trains a new smaller model with a different parameter vector and measures Δ on output distributions. Parameter sharing re-uses the same θ many times and thus reduces R without reducing FLOPs. This distinction—*what is reduced*: stored bits, resident bits, FLOPs, or KV-cache—is the organizing principle of the taxonomy in Chapter 4.

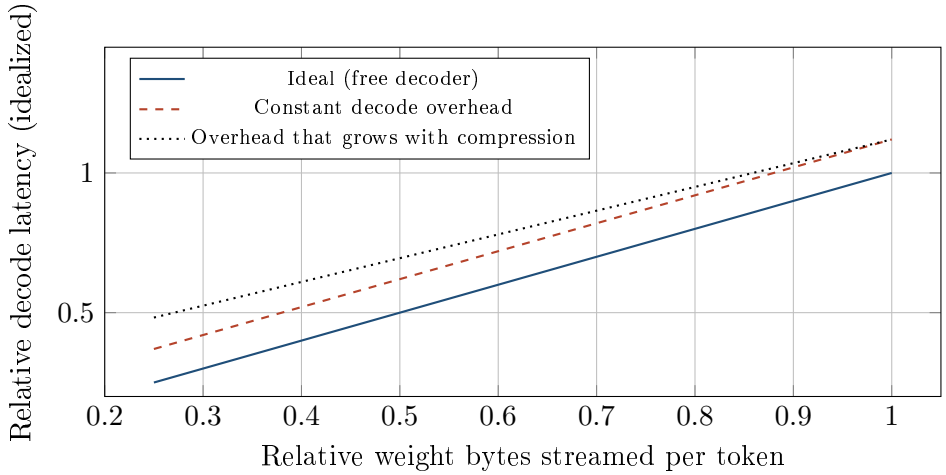


Figure 2.1: **Illustrative, synthetic schematic.** The memory-bound latency floor scales with bytes streamed (solid). Decode overhead (dashed, dotted) shifts the curve upward and can erase the benefit when the compression ratio is modest. The offsets are invented for illustration and do not represent any measured kernel.

2.4 A short history

The idea that a smaller model can mimic a larger one predates deep learning: Buciluǎ et al. compressed ensembles into a single network in 2006 [1], and Ba and Caruana showed in 2014 that shallow networks could approximate deeper ones when trained on logits [2]. Hinton, Vinyals and Dean formalized soft targets with temperature in 2015 [3]. In parallel, low-rank factorization of speech-recognition output layers [90] and of convolutional filters [91, 92] established the post-hoc matrix-decomposition route, while hashing [136], circulant projections [124], structured transforms [125] and tensor trains [128] supplied structured alternatives. Deep Compression combined pruning, quantization and Huffman coding in 2016 [185], which is the earliest widely cited instance of entropy-coded network weights.

With transformers came DistilBERT [11], TinyBERT [12], MobileBERT [13] and MiniLM [14] for encoders, ALBERT's cross-layer sharing and factorized embeddings [140], and the Universal Transformer's recurrence [144]. LLM-scale distillation took off with instruction-following imitation (Alpaca [26], Orca [28]), then with principled on-policy methods [22, 23], and after January 2025 with distilled reasoning models [41]. A detailed timeline appears in Chapter 15.

2.5 Four recurring themes

Compression versus training a smaller model. Because pretraining a small model on enough tokens is a strong baseline [173–175], every compression method implicitly argues that the large model's information is cheaper to extract than re-learn. Distillation scaling laws make this explicit: distillation beats supervised pretraining only when the teacher already exists or many students are produced [53].

Calibration data and distribution. Post-hoc methods (SVD-LLM, GPTQ) rely on small calibration sets; the choice of calibration distribution shapes which directions are deemed unimportant. Distillation depends even more strongly on the training distribution of prompts and, for on-policy methods, the student's own samples.

Evaluation is part of the method. Perplexity can remain nearly flat while downstream capability collapses [189, 190]; a method is only as good as the evaluation that supports it (Chapter 14).

Hardware decides. Whether a method wins depends on the arithmetic intensity of the workload and on kernels (Chapter 14). A method with elegant theory but no fused kernel can lose to a simple dense baseline.

2.6 Methodology of this survey

The author searched the literature through October 2026 using web search and read abstracts or full texts of the works used for quantitative claims. Papers cited from prior knowledge are standard references and are cited with venue and year. Where a number is quoted it is attributed to the paper; where the author has computed something (for instance a break-even rank) the derivation is shown. Where figures are drawn with pgfplots they are labelled as synthetic or illustrative unless the data come from a cited table. Readers should consult primary sources before relying on any number for engineering decisions. Section titles in the later chapters flag what is established, what is reported but unreplicated, and what is the author's inference.

How to read this paper

Chapters 3–4 supply notation and the taxonomy. Chapters 5–13 are the technical core, one per family. Chapter 14 covers systems and evaluation. Chapters 15–18 summarize the state of the art, risks and a decision procedure. Chapter 19 gives the priority assessment, and chapter 20 lists open problems. Appendices collect notation, longer derivations and a glossary.

Chapter 3

Mathematical Foundations

This chapter collects the tools used throughout: information-theoretic quantities, divergences, matrix approximation, rate–distortion theory and source coding. Proofs are given where short; standard references are [88, 130, 199, 210].

3.1 Entropy, cross-entropy and divergences

Let p, q be distributions on a finite set $\mathcal{X}$. The Shannon entropy is $H(p) = -\sum_x p(x) \log p(x)$, the cross-entropy is $H(p, q) = -\sum_x p(x) \log q(x)$, and the Kullback–Leibler divergence is

$$\mathrm{KL}(p \,\|\, q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = H(p, q) - H(p). \tag{3.1}$$

KL is nonnegative (Gibbs' inequality), zero iff $p = q$, and asymmetric. Pinsker's inequality relates it to total variation, $\mathrm{TV}(p, q) \le \sqrt{\frac{1}{2} \mathrm{KL}(p \,\|\, q)}$.

Definition 3.1 (f-divergence)**.** For convex $f : (0, \infty) \to \mathbb{R}$ with $f(1) = 0$, $D_f(p\|q) = \sum_x q(x) f\!\left(\frac{p(x)}{q(x)}\right)$.

Choosing $f(t) = t \log t$ yields forward KL $\mathrm{KL}(p \,\|\, q)$; $f(t) = -\log t$ yields reverse KL $\mathrm{KL}(q \,\|\, p)$; $f(t) = \frac{1}{2}|t - 1|$ yields total variation; and the Jensen–Shannon divergence

$$\mathrm{JS}(p, q) = \tfrac{1}{2} \mathrm{KL}\big(p \,\big\|\, \tfrac{p+q}{2}\big) + \tfrac{1}{2} \mathrm{KL}\big(q \,\big\|\, \tfrac{p+q}{2}\big) \tag{3.2}$$

is an f-divergence that is symmetric and bounded by $\log 2$. Skewed variants $\mathrm{JS}_\beta(p, q) = \beta \,\mathrm{KL}(p \,\|\, \beta p + (1-\beta)q) + (1-\beta)\,\mathrm{KL}(q \,\|\, \beta p + (1-\beta)q)$ interpolate between the two KL directions as β approaches 0 or 1 (up to a scaling of order β), and generalized-JS losses of this kind are used in on-policy distillation [23]. f-divergence minimization for sequence-level distillation was studied in [25].

Mode-covering versus mode-seeking. Consider fitting a unimodal student q_θ to a bimodal teacher p. Minimizing forward KL $\mathrm{KL}(p \,\|\, q_\theta)$ penalizes any region where $p > 0$ but $q_\theta \approx 0$ (the integrand blows up), so q_θ spreads mass to cover both modes (*mode-covering*). Minimizing reverse KL $\mathrm{KL}(q_\theta \,\|\, p)$ penalizes mass where $q_\theta > 0$ but $p \approx 0$, so q_θ concentrates on one mode (*mode-seeking*). This is the principal reason MiniLLM argues for reverse KL when the student is small [22]; Chapter 5 analyses it in detail.

3.2 Sequence models and code length

A language model assigns $p_\theta(y_{1:T} \mid x) = \prod_t p_\theta(y_t \mid x, y_{<t})$. The negative log-likelihood of a sequence, in bits, is its *code length* under an ideal arithmetic coder driven by the model

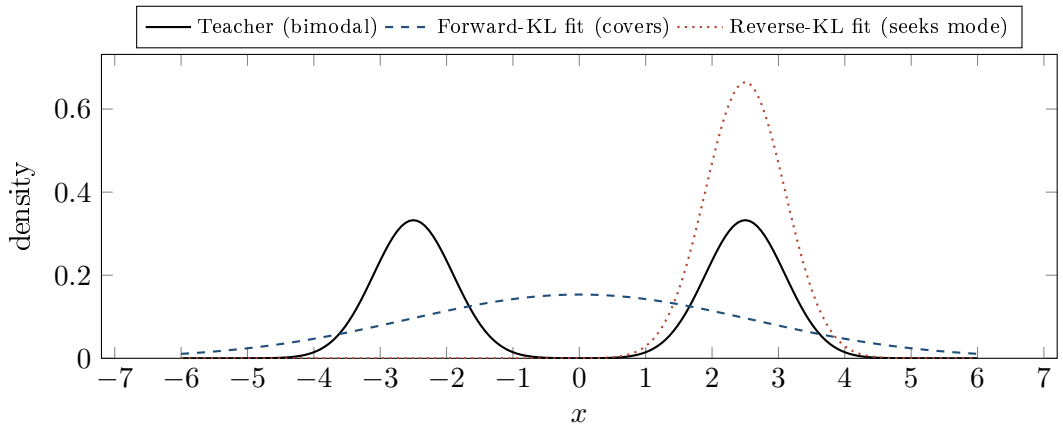


Figure 3.1: **Analytic illustration** (Gaussian fits drawn by hand for a symmetric two-component mixture; the widths are approximate moment-matched and mode-locked solutions, not numerical optima). Forward KL pulls the student between modes; reverse KL locks onto one.

(Chapters 12 and 13). Sequence-level KL decomposes through the chain rule:

$$\mathrm{KL}(p(\cdot \mid x) \,\|\, q(\cdot \mid x)) = \sum_{t=1}^{T} \mathbb{E}_{y_{<t} \sim p} \, \mathrm{KL}(p(\cdot \mid x, y_{<t}) \,\|\, q(\cdot \mid x, y_{<t})) \,. \tag{3.3}$$

The expectation is over prefixes drawn from p for the forward direction; for the reverse direction $\mathrm{KL}(q \,\|\, p)$ the prefixes are drawn from q, which is why reverse-KL distillation is naturally *on-policy* with respect to the student.

3.3 Matrix approximation

3.3.1 SVD and the Eckart–Young–Mirsky theorem

Every $\boldsymbol{W} \in \mathbb{R}^{m \times n}$ has a singular value decomposition $\boldsymbol{W} = U \Sigma V^\top = \sum_{i=1}^{\min(m,n)} \sigma_i u_i v_i^\top$ with $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. Write $\boldsymbol{W}_r = \sum_{i \leq r} \sigma_i u_i v_i^\top$.

Theorem 3.1 (Eckart–Young–Mirsky [86, 87])**.** *For any unitarily invariant norm $\|\cdot\|$, in particular the spectral and Frobenius norms, $\min_{\mathrm{rank}(\boldsymbol{B}) \leq r} \|\boldsymbol{W} - \boldsymbol{B}\| = \|\boldsymbol{W} - \boldsymbol{W}_r\|$. Moreover $\|\boldsymbol{W} - \boldsymbol{W}_r\|_2 = \sigma_{r+1}$ and $\|\boldsymbol{W} - \boldsymbol{W}_r\|_{\mathrm{F}}^2 = \sum_{i>r} \sigma_i^2$.*

Proof sketch for the spectral and Frobenius cases. Spectral norm. Take any $\boldsymbol{B}$ of rank $\leq r$. Its null space has dimension $\geq n - r$. The span of $v_1, \ldots, v_{r+1}$ has dimension $r + 1$, so the two subspaces intersect in a unit vector z with $\boldsymbol{B}z = 0$. Then $\|(\boldsymbol{W} - \boldsymbol{B})z\|_2^2 = \|\boldsymbol{W}z\|_2^2 = \sum_{i \leq r+1} \sigma_i^2 (v_i^\top z)^2 \geq \sigma_{r+1}^2$, hence $\|\boldsymbol{W} - \boldsymbol{B}\|_2 \geq \sigma_{r+1}$, attained by $\boldsymbol{W}_r$.

Frobenius norm. Weyl's inequality for singular values states $\sigma_{i+j-1}(A + B) \leq \sigma_i(A) + \sigma_j(B)$. Taking $A = \boldsymbol{W} - \boldsymbol{B}$, $B = \boldsymbol{B}$ with $\mathrm{rank}\, \boldsymbol{B} \leq r$ gives $\sigma_{r+i}(\boldsymbol{W}) \leq \sigma_i(\boldsymbol{W} - \boldsymbol{B}) + \sigma_{r+1}(\boldsymbol{B}) = \sigma_i(\boldsymbol{W} - \boldsymbol{B})$ for $i \geq 1$. Squaring and summing, $\|\boldsymbol{W} - \boldsymbol{B}\|_{\mathrm{F}}^2 \geq \sum_{i \geq 1} \sigma_{r+i}^2$. Equality holds at $\boldsymbol{B} = \boldsymbol{W}_r$. □

3.3.2 Stable rank and spectral decay

Rank selection depends on how fast singular values decay. The *stable rank* $\operatorname{srank}(\boldsymbol{W}) = \|\boldsymbol{W}\|_{\mathrm{F}}^2 / \|\boldsymbol{W}\|_2^2 = \sum_i \sigma_i^2/\sigma_1^2$ is a robust proxy. The *energy fraction* retained at rank r is $\eta(r) = \sum_{i\le r}\sigma_i^2 / \sum_i \sigma_i^2$. A power-law spectrum $\sigma_i \propto i^{-\alpha}$ with $\alpha > 1/2$ has $1-\eta(r) \approx r^{1-2\alpha}$ (for large n), so the tail shrinks polynomially, but not fast enough for aggressive truncation unless α is large. Empirically, trained LLM weight matrices are far from low-rank in the Frobenius sense even though their *activations* are low-rank-like; the point made in [102]. Chapter 8 builds on this.

Example 3.1 (Energy retained, illustrative)**.** Let $\boldsymbol{W} \in \mathbb{R}^{4096\times 4096}$ have $\sigma_i = i^{-0.7}$ (a synthetic example). The total energy is $\sum_{i=1}^{4096} i^{-1.4} \approx \zeta(1.4) - \int_{4096}^{\infty} x^{-1.4}dx \approx 3.11 - 0.09 \approx 3.0$. Retaining $r = 1024$ (a 50% parameter cut for a square matrix) leaves tail energy $\approx \int_{1024}^{4096} x^{-1.4}dx = (1024^{-0.4} - 4096^{-0.4})/0.4 \approx (0.0625 - 0.0359)/0.4 \approx 0.067$, i.e. about 2.2% of the energy is lost, a relative Frobenius error near $\sqrt{0.022} \approx 15\%$. This is large for a layer-wise error, which illustrates why plain SVD of LLM weights compresses poorly and why the choice of the metric matters (Chapter 8).

3.3.3 Whitened approximation

Let $\boldsymbol{X} \in \mathbb{R}^{n\times N}$ be calibration activations feeding the layer $\boldsymbol{W} \in \mathbb{R}^{m\times n}$. We care about $\|(\boldsymbol{W}-\boldsymbol{B})\boldsymbol{X}\|_{\mathrm{F}}$, not $\|\boldsymbol{W}-\boldsymbol{B}\|_{\mathrm{F}}$.

Proposition 3.1 (Whitened truncation is output-optimal)**.** *Let $\boldsymbol{X}\boldsymbol{X}^\top = SS^\top$ with S invertible (Cholesky or symmetric square root). Let $\boldsymbol{W}S = U\Sigma V^\top$ be an SVD and $\boldsymbol{B}^\star = (\boldsymbol{W}S)_r S^{-1}$ where $(\cdot)_r$ is rank-r truncation. Then $\boldsymbol{B}^\star$ minimizes $\|(\boldsymbol{W}-\boldsymbol{B})\boldsymbol{X}\|_{\mathrm{F}}$ over $\operatorname{rank}\boldsymbol{B} \le r$, and the minimum equals $\left(\sum_{i>r}\sigma_i^2\right)^{1/2}$ where σ_i are the singular values of $\boldsymbol{W}S$.*

Proof. $\|(\boldsymbol{W}-\boldsymbol{B})\boldsymbol{X}\|_{\mathrm{F}}^2 = \operatorname{tr}((\boldsymbol{W}-\boldsymbol{B})\boldsymbol{X}\boldsymbol{X}^\top(\boldsymbol{W}-\boldsymbol{B})^\top) = \|(\boldsymbol{W}-\boldsymbol{B})S\|_{\mathrm{F}}^2 = \|\boldsymbol{W}S - \boldsymbol{B}S\|_{\mathrm{F}}^2$. Since S is invertible, $\boldsymbol{B} \mapsto \boldsymbol{B}S$ is a rank-preserving bijection, so the problem is equivalent to best rank-r approximation of $\boldsymbol{W}S$, solved by Theorem 3.1. □

This simple result underlies SVD-LLM and its relatives [93–96]; Chapter 8 discusses the practical differences (diagonal scaling in ASVD, Fisher weighting in FWSVD, Cholesky whitening in SVD-LLM).

3.3.4 Randomized SVD

Computing a full SVD of a 4096×11008 matrix is cheap enough on a GPU, but at larger scales or inside search loops randomized methods are useful [88]: draw a Gaussian sketch $\Omega \in \mathbb{R}^{n\times(r+p)}$, form $Y = (\boldsymbol{W}\boldsymbol{W}^\top)^q \boldsymbol{W}\Omega$, orthonormalize $Y = QR$, and take the SVD of $Q^\top\boldsymbol{W}$. With oversampling $p \ge 5$ and $q \in \{1, 2\}$ power iterations the expected error is within a small factor of σ_{r+1} for matrices with moderate spectral decay.

3.4 Rate–distortion theory

For a source X with distortion measure d, the rate–distortion function is $R(D) = \min_{p(\hat{x}|x):\mathbb{E}d\le D} I(X;\hat{X})$. For a memoryless Gaussian source $\mathcal{N}(0,\sigma^2)$ under squared error, $R(D) = \max\{0, \frac{1}{2}\log_2(\sigma^2/D)\}$. For a vector of independent Gaussians with variances $\lambda_1,\dots,\lambda_k$ the optimum is *reverse water-filling*: with water level θ, component i is allotted distortion $\min(\theta,\lambda_i)$ and rate $\max\{0,\frac{1}{2}\log_2(\lambda_i/\theta)\}$ bits, where θ is chosen so that the total distortion is D.

Proposition 3.2 (Rank truncation is water-filling at infinite resolution)**.** *For a Gaussian vector with covariance eigenvalues λ_i, the optimal D-distortion code zeroes all components with $\lambda_i \le \theta$ and codes the rest with rate $\frac{1}{2}\log_2(\lambda_i/\theta)$. Rank-$r$ projection onto the top eigenvectors corresponds to the limit in which components above the water level are kept at full precision and those below are discarded.*

This connects low-rank truncation to transform coding: SVD is the Karhunen–Loève transform of the weight matrix viewed as a random matrix with correlated rows. In practice, one *quantizes* the kept components rather than keeping them at full precision, which is what low-rank plus quantization methods do [108, 109].

Operational meaning for LLMs. Rate–distortion theory gives a lower bound on bits for a given distortion if the distribution of the source is known. Neural weights are not samples from a known distribution, and the distortion that matters is functional, so these results guide intuition rather than predict compression ratios. The theory does show two robust facts: (i) allocating bits (or rank) unevenly according to component variance is optimal, justifying heterogeneous rank allocation; and (ii) the gain from entropy coding scales with how non-uniform the symbol distribution is.

3.5 Source coding

Theorem 3.2 (Kraft–McMillan)**.** *A binary prefix code with codeword lengths ℓ_x exists iff $\sum_x 2^{-\ell_x} \le 1$.*

Theorem 3.3 (Shannon source coding)**.** *The expected length of any uniquely decodable code satisfies $\mathbb{E}\ell \ge H(p)$ bits, and there is a prefix code (e.g. with $\ell_x = \lceil -\log_2 p(x)\rceil$) achieving $\mathbb{E}\ell < H(p)+1$. Huffman coding [196] is optimal among symbol-by-symbol prefix codes.*

Sketch. Let $q(x) = 2^{-\ell_x}/\sum_y 2^{-\ell_y}$. Then $\mathbb{E}\ell - H(p) = \mathrm{KL}(p\,\|\,q) - \log_2\sum_y 2^{-\ell_y} \ge 0$ by Gibbs' inequality and Kraft. The Shannon code satisfies Kraft with equality bound and $\ell_x < -\log_2 p(x) + 1$. □

Arithmetic coding [197] and asymmetric numeral systems [198] approach $H(p)$ per symbol more closely (overhead $O(1/N)$ bits per symbol for long messages, with $O(10^{-3})$–$O(10^{-2})$ bit inefficiency in finite-precision practice, which is the author's typical figure rather than a measured one) and avoid the integer-length penalty of Huffman, which matters when probabilities are far from dyadic. Huffman's penalty is up to 1 bit per symbol in the worst case but is small for the 8-bit-exponent alphabet whose probabilities are spread over a few dozen symbols (Chapter 12).

Universal coding with a learned model. If a coder uses model q while the data come from p, the expected code length is $H(p, q) = H(p) + \mathrm{KL}(p \,\|\, q)$, so the *excess* is exactly the KL divergence. This identity is the bridge between language modeling and compression (Chapter 13) and also explains why KL divergence is the natural distillation loss: minimizing $\mathrm{KL}(p_T \,\|\, p_S)$ minimizes the extra bits a coder pays when it uses the student in place of the teacher.

3.6 Floating-point formats

IEEE-754 binary32 has 1 sign, 8 exponent and 23 mantissa bits; BF16 has 1/8/7; FP16 has 1/5/10; FP8 variants are E4M3 (1/4/3) and E5M2 (1/5/2). A normal number is $(-1)^s \, 2^{e-\mathrm{bias}} \, (1 + m/2^M)$. For trained weights, magnitudes cluster within a few orders of magnitude, so the exponent field takes only a handful of values with significant probability while sign and mantissa bits are close to uniformly random. Chapter 12 quantifies this.

3.7 Generalization and compression

Compression yields generalization bounds. For a finite hypothesis class $\mathcal{H}$ encoded by prefix code of length $\ell(h)$ bits, with probability $1 - \delta$ over n i.i.d. samples, for losses in $[0, 1]$,

$$L(h) \le \hat{L}(h) + \sqrt{\frac{\ell(h) \ln 2 + \ln(1/\delta)}{2n}} \qquad \forall h \in \mathcal{H}, \tag{3.4}$$

by Hoeffding and a union bound weighted by $2^{-\ell(h)}$ (Occam bound). PAC-Bayes versions replace $\ell(h) \ln 2$ by $\mathrm{KL}(Q \,\|\, P)$ [228]; Chapter 13 applies them to LLMs [227].

3.8 Notation summary

Further notation is collected in Appendix A. We write p_{T} for the teacher, p_θ for the student, $\boldsymbol{W}$ for a weight matrix, $\boldsymbol{X}$ for activations, r for rank, τ for temperature, and $\mathcal{V}$ for the vocabulary.

Chapter 4

Taxonomy of Compression Approaches

4.1 Axes of classification

No single axis cleanly separates the methods in this paper, so we use four.

1. **What is produced.** (a) *A new model* trained to imitate (distillation, training from scratch); (b) *a re-parameterization of an existing model* (low-rank, structured, tensorized, shared layers); (c) *a re-encoding of an existing model* (lossless coding, delta coding); (d) *a combination of existing models* (merging, soups).

2. **What is reduced.** Stored bytes, resident bytes, parameters, FLOPs per token, KV-cache, or only transfer size.

3. **Fidelity.** Bit-exact (lossless), function-preserving up to tolerance, or distribution-matching only.

4. **When the cost is paid.** Offline once (post-training), as a training phase (re-training or distillation), at load time, or at every forward pass (decode-on-the-fly).

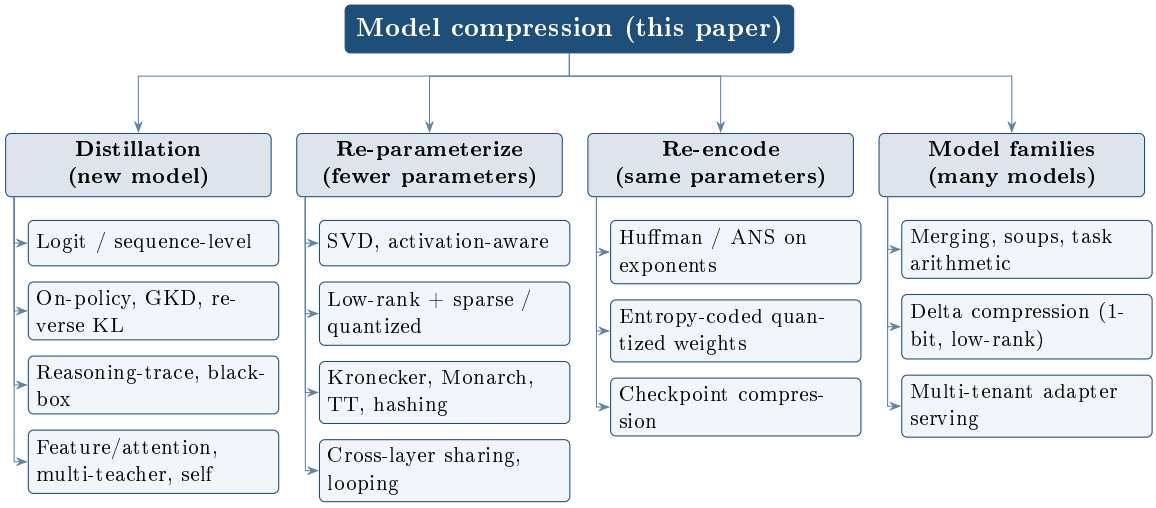


Figure 4.1: Taxonomy used in this paper. Lines indicate containment. The neighbouring families, quantization and pruning, are covered in companion papers and are not drawn.

Table 4.1: Taxonomy matrix. "Training needed" refers to the typical form of the method as published. FLOPs entry: ↓ reduced, = unchanged, ↑ increased relative to the dense original.

Family	Produces	FLOPs	Fidelity	Training needed	Cost paid
Logit/sequence distillation	new model	↓	distributional	yes (large)	offline
On-policy distillation	new model	↓	distributional	yes	offline
Black-box distillation	new model	↓	imitative	yes	offline + API
Truncated SVD (post-hoc)	re-param.	↓ (if r small)	approximate	none or short	offline
Low-rank + quantization	re-param.	↓ / =	approximate	short	offline
Monarch / butterfly / TT	re-param.	↓	approximate or retrained	often from scratch	offline
Hashing	re-param.	=	approximate	yes	offline
Cross-layer sharing / looping	re-param.	= or ↑	approximate	uptraining	offline
Merging / soups	combination	=	functional blend	none	offline
Delta compression	re-encode / approx.	=	approx. delta	none or short	per request
Lossless coding	re-encode	=	**exact**	none	load / per layer

4.2 Comparison across axes

4.3 What each family leaves unchanged

Distillation keeps neither the architecture nor the parameter values. It transfers a *conditional distribution* on a chosen input distribution. Anything not exercised by that distribution (rare knowledge, long-tail languages, safety edge cases) can silently disappear.

Factorization and structure keep the architecture's graph but change the operator inside each linear layer. The approximation quality depends on how well the weights (or activation-weighted weights) lie near the chosen manifold: the rank-r matrices, the Monarch matrices, or the tensor-train format of a given rank [119, 128].

Sharing keeps the function class but forces equality constraints between parameter blocks. A model with L layers sharing K unique blocks has the same FLOPs as the unshared one but a parameter count lowered by a factor $\approx L/K$ [140, 147].

Lossless coding keeps everything. Its limit is Shannon's entropy bound of the weight symbols conditioned on whatever context the coder models (Theorem 3.3).

Merging and deltas treat a family of models as one object: store a base, express members as small offsets. They exploit the observation that fine-tuning moves weights little relative to the base magnitude [161, 167].

4.4 Combinations

Practical systems compose methods; the typical order is dictated by what each step preserves.

1. *Prune then distill*: structured pruning (depth, width, heads) followed by distillation to recover accuracy, as in Minitron and Puzzle [60, 61].

2. *Distill then quantize*: distill to a mid-size student, then apply post-training quantization. Order matters; quantizing a teacher and distilling into a quantized student is also used (quantization-aware distillation).

3. *Low-rank plus quantized*: factor $\boldsymbol{W} \approx Q + LR$ with Q low-bit and L, R higher precision [108, 109].

4. *Quantize then entropy-code*: after quantization the index distribution is non-uniform, so Huffman or ANS coding gains a further 10–20% (author's rule of thumb; see Chapter 12) at the cost of decode complexity [185].

5. *Lossless on top of anything*: exponent coding of a BF16 model, or entropy coding of whatever remains [200].

Reading the taxonomy

Methods that *do not* change function (lossless coding) are the easiest to adopt but have a ceiling set by entropy. Methods that *change the model* (distillation, factorization, sharing) can reach larger reductions but impose a validation burden: one must demonstrate that the capability you care about survived. The more the method relies on a calibration or training distribution, the larger the burden.

4.5 Decision-relevant dimensions

Three questions sort methods into practical bins.

Do you control training? If you can train, distillation and recurrent/shared architectures are open to you. If you can only post-process frozen weights, you are limited to low-rank/structured approximation, lossless coding, and delta coding.

Is bit-exactness required? Regulated, audited, or reproducibility-critical settings often require the exact same outputs; only lossless coding qualifies.

Is the bottleneck memory capacity, bandwidth or FLOPs? Capacity is relieved by anything that reduces resident bytes; bandwidth by anything that reduces bytes streamed per token (including lossless coding if decode is fused); FLOPs only by methods that change the compute graph (distillation, low-rank with small r, structured matrices).

Chapter 18 turns these questions into flowcharts and tables.

Chapter 5

Knowledge Distillation I: Theory

Knowledge distillation (KD) trains a *student* p_θ to reproduce the behaviour of a *teacher* p_{T}. The idea is old [1, 2]; the form that launched the modern literature is Hinton, Vinyals and Dean's soft-target objective [3]. This chapter covers the classification-era theory that LLM distillation inherits: soft targets and temperature, why distillation helps, the choice of divergence, intermediate-representation matching, and the capacity gap. LLM-specific methods follow in Chapters 6 and 7.

5.1 Soft targets and temperature

Given logits $z \in \mathbb{R}^K$, the temperature-τ softmax is $p_k^{(\tau)} = \exp(z_k/\tau)/\sum_j \exp(z_j/\tau)$. The standard objective mixes hard-label cross-entropy with a distillation term:

$$\mathcal{L}_{\mathrm{KD}}(\theta) = (1-\lambda)\, H\big(y, p_\theta^{(1)}\big) + \lambda\, \tau^2\, \mathrm{KL}\Big(p_{\mathrm{T}}^{(\tau)} \,\Big\|\, p_\theta^{(\tau)}\Big), \tag{5.1}$$

where y is the one-hot label and the factor τ^2 restores gradient magnitude, as shown next.

Proposition 5.1 (Gradient of the soft-target loss [3])**.** *Let s be the student logits and t the teacher logits, $q = \mathrm{softmax}(s/\tau)$, $p = \mathrm{softmax}(t/\tau)$. Then $\partial\, \mathrm{KL}(p\,\|\,q)\,/\partial s_k = \frac{1}{\tau}(q_k - p_k)$. If τ is large relative to the logit range and logits are zero-mean per example, then $q_k - p_k \approx \frac{1}{K\tau}(s_k - t_k)$ and so $\tau^2\, \partial \mathrm{KL}(p\,\|\,q)\,/\partial s_k \approx \frac{1}{K}(s_k - t_k)$: distillation becomes logit regression.*

Proof. $\mathrm{KL}(p\,\|\,q) = \sum_j p_j \log p_j - \sum_j p_j \log q_j$ and $\log q_j = s_j/\tau - \log \sum_i exp(s_i/\tau)$. Differentiating the second sum yields $-\frac{1}{\tau}p_k + \frac{1}{\tau}q_k \sum_j p_j = \frac{1}{\tau}(q_k - p_k)$. For large τ, $\exp(s_k/\tau) \approx 1 + s_k/\tau$, so $q_k \approx (1 + s_k/\tau)/(K + \sum_j s_j/\tau) \approx \frac{1}{K}(1 + s_k/\tau - \bar{s}/\tau)$ and likewise for p_k; with zero-mean logits the difference is $\frac{1}{K\tau}(s_k - t_k)$. Hence $\partial\, \mathrm{KL}(p\,\|\,q)\,/\partial s_k = \frac{1}{\tau}(q_k - p_k) \approx \frac{1}{K\tau^2}(s_k - t_k)$, and multiplying by τ^2 yields the stated limit. □

Hinton et al. interpret the small probabilities that a teacher assigns to wrong classes as *dark knowledge*: the relative magnitudes among incorrect classes encode similarity structure absent from one-hot labels.

Example 5.1 (Temperature in numbers)**.** Take four logits $z = (8, 5, 2, -1)$. At $\tau = 1$ the softmax is $(0.950, 0.047, 0.0024, 0.0001)$ with entropy 0.30 bits: the runner-up classes carry almost no gradient signal. At $\tau = 2$: $(0.779, 0.174, 0.039, 0.009)$, entropy 0.96 bits. At $\tau = 4$: $(0.555, 0.262, 0.124, 0.059)$, entropy 1.59 bits. At $\tau = 8$: $(0.403, 0.277, 0.190, 0.131)$, entropy 1.88 bits, close to the maximum of 2. Higher τ exposes the ordering and spacing of the lower logits; too high a τ approaches uniform and washes out the signal.

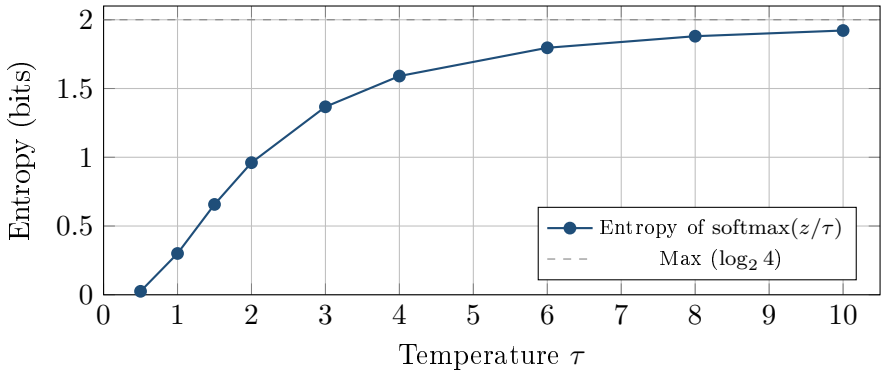


Figure 5.1: Entropy of the temperature softmax for the example logits $z = (8, 5, 2, -1)$ of Example 5.1. Values were computed exactly from the formula; the logits themselves are invented for illustration.

Algorithm 1 Classical logit distillation (offline teacher)

Require: Teacher p_{T}, student p_θ with parameters θ, dataset $\mathcal{D}$, temperature τ, mix λ

1: **for** each minibatch $(x, y) \in \mathcal{D}$ **do**
2: $\quad t \leftarrow \mathrm{logits}_{p_{\mathrm{T}}}(x)$ ▷ no gradient; can be precomputed and cached (top-k)
3: $\quad s \leftarrow \mathrm{logits}_{p_\theta}(x; \theta)$
4: $\quad \mathcal{L} \leftarrow (1-\lambda)\,\mathrm{CE}(y, \mathrm{softmax}(s)) + \lambda\tau^2\,\mathrm{KL}(\mathrm{softmax}(t/\tau) \,\|\, \mathrm{softmax}(s/\tau))$
5: $\quad \theta \leftarrow \theta - \eta\,\nabla_\theta \mathcal{L}$
6: **end for**

5.2 Why does distillation help?

Several partly complementary explanations coexist. None is a complete theory of deep networks, so the claims below are stated with their scope.

Soft labels as lower-variance risk estimates. Menon et al. show that the Bayes class-probability function $\eta(x) = P(y \mid x)$ yields a lower-variance estimator of the risk than one-hot labels, so a student trained on teacher probabilities that approximate η enjoys a better bias–variance trade-off; the benefit depends on how close p_{T} is to η [15]. Intuitively, replacing the noisy label y_i by its conditional expectation $\eta(x_i)$ removes the label-sampling noise from the empirical risk; the formal statements and conditions are in the cited paper.

Privileged information. Lopez-Paz et al. connect distillation to learning using privileged information: the teacher has access to information (a bigger model, more data, extra features) unavailable to the student, and transmits its consequences through outputs [16]. This view applies directly to the recent self-distillation methods in which the teacher sees a ground-truth reasoning trace while the student does not [54].

Linear and kernel analyses. Phuong and Lampert analyse a linear student and deep linear teacher and identify three factors: the student sees data geometry through the teacher, there is a form of early stopping, and the teacher's logit norm acts as a data-dependent prior [17]. In

kernel regression, repeated self-distillation increases regularization strength and can first help then hurt as the solution collapses [7].

Fidelity versus generalization. A cautionary finding is that students often do not match the teacher's predictive distribution closely even when they have capacity to; better fidelity does not always imply better accuracy, and optimization difficulty can limit transfer [18]. For LLMs, where the supports are enormous and the student sees only a tiny fraction of the possible contexts, the fidelity gap is large and cannot be measured exhaustively.

Regularization and label smoothing. Soft targets reduce overconfidence. Label smoothing is distillation from a uniform teacher mixed with the label; "born-again" networks in which the student has the same size as the teacher can outperform it, showing that regularization effects exist beyond compression [6, 19].

5.3 Choice of divergence

Let $p = p_{\mathrm{T}}(\cdot \mid x)$, $q = p_\theta(\cdot \mid x)$.

5.3.1 Forward KL

$\mathrm{KL}(p \,\|\, q)$ is the cross-entropy to the teacher distribution (up to the constant $H(p)$). It is easy to optimize, requires teacher probabilities (full or top-k) or teacher samples, and is equivalent to maximum-likelihood on teacher samples in expectation: SFT on teacher outputs is a Monte-Carlo estimate of forward KL at the sequence level [10].

5.3.2 Reverse KL

$\mathrm{KL}(q \,\|\, p) = \mathbb{E}_{y\sim q}[\log q(y) - \log p(y)]$ is the expected log-ratio under the student's own samples. The gradient needs a score-function estimator (policy gradient) because the sampling distribution depends on θ:

$$\nabla_\theta \,\mathrm{KL}(q_\theta \,\|\, p) = \mathbb{E}_{y\sim q_\theta}\big[(\log q_\theta(y) - \log p(y))\,\nabla_\theta \log q_\theta(y)\big]. \tag{5.2}$$

This is the REINFORCE gradient with reward $r(y) = \log p(y) - \log q_\theta(y)$, a viewpoint that underlies on-policy distillation [22, 52, 55].

5.3.3 An exactly solvable case

Proposition 5.2 (Gaussian fit to a mixture)**.** *Let p be a distribution with mean μ_p and variance σ_p^2, and let $q = \mathcal{N}(\mu, \sigma^2)$. Then $\arg\min_q \mathrm{KL}(p \,\|\, q)$ is the moment-matched Gaussian $\mu = \mu_p, \sigma = \sigma_p$. For $p = \frac{1}{2}\mathcal{N}(-a, s^2) + \frac{1}{2}\mathcal{N}(a, s^2)$ with $a \gg s$, the minimizers of $\mathrm{KL}(q \,\|\, p)$ include (approximately) the two single-mode fits $\mathcal{N}(\pm a, s^2)$, each with $\mathrm{KL}(q \,\|\, p) \approx \log 2$.*

Sketch. $\mathrm{KL}(p \,\|\, q) = -H(p) + \frac{1}{2}\log(2\pi\sigma^2) + \mathbb{E}_p[(x-\mu)^2]/(2\sigma^2)$. Setting derivatives to zero yields $\mu = \mathbb{E}_p x$, $\sigma^2 = \mathbb{E}_p(x-\mu)^2$. For the reverse direction, if q is concentrated on one component,

$p(y) \approx \frac{1}{2}\mathcal{N}(y; a, s^2)$ on the support of q, so $\log q(y) - \log p(y) \approx \log 2$ at $\sigma = s$, $\mu = a$, giving $\mathrm{KL}(q \,\|\, p) \approx \log 2$. The moment-matched fit with variance $a^2 + s^2$ places most mass where p is nearly zero, and has a much larger reverse KL. □

In Figure 3.1, the forward fit has standard deviation $\sqrt{a^2 + s^2} = \sqrt{6.25 + 0.36} \approx 2.57$ for $a = 2.5$, $s = 0.6$. Mode-seeking is a feature when the student cannot represent all teacher modes and the downstream use is to generate one good answer; it is a flaw when diversity matters, since reverse KL can drop modes entirely.

5.3.4 Jensen–Shannon and skewed divergences

JS is bounded and symmetric, which can stabilize training when supports differ, but its gradient vanishes when p and q have nearly disjoint support (the classical GAN-era observation). Skewed KL ($\mathrm{KL}(p \,\|\, \alpha p + (1-\alpha)q)$ and its reverse) are used in DistiLLM to mix the properties of both directions and to stabilize optimization [24]. GKD introduces a generalized JSD with a tunable interpolation coefficient β [23].

5.3.5 General f-divergences

Wen et al. study f-divergence minimization for sequence-level KD and report that symmetric divergences (JS, total variation) can beat both KL directions on some tasks [25]; the preferred divergence is task- and model-dependent, and no single choice dominates. A caution from the on-policy literature is that the direction matters less than *where the prefixes come from*: one recent analysis separates the two choices, trajectory source and KL direction, producing four distinct objectives and finding that they behave differently on math reasoning [57].

Table 5.1: Properties of common distillation divergences (qualitative).

Divergence	Behaviour	Needs	Typical risk
Forward KL $\mathrm{KL}(p \,\|\, q)$	mode-covering	teacher probabilities or samples	over-smoothed, hallucination from spread mass
Reverse KL $\mathrm{KL}(q \,\|\, p)$	mode-seeking	student samples + teacher log-probs	mode collapse, low diversity
JS / skewed JS	intermediate, bounded	both	weaker gradients when disjoint
Total variation	robust, non-smooth	both	slow optimization
Squared logit error (MSE on logits)	high-τ limit of KL	teacher logits	ignores calibration

5.4 Intermediate representations

Matching only outputs discards information in intermediate layers. Three families of intermediate losses exist.

Hints and feature maps. FitNets add a regressor from a student hidden layer to a teacher hidden layer, $\mathcal{L}_{\text{hint}} = \left\| g(h_\ell^S) - h_{\ell'}^T \right\|_2^2$ [4]. Dimension mismatches require projection layers, which are discarded afterwards.

Attention maps. Attention transfer matches normalized spatial attention maps [5]; for transformers, attention-distribution matching $\mathrm{KL}(A_{\ell'}^T \,\|\, A_\ell^S)$ per head is used by TinyBERT [12]. MiniLM matches only the last layer's self-attention distributions and the *value relations* $\mathrm{softmax}(VV^\top/\sqrt{d_k})$, which removes the need to match layer counts and hidden sizes [14]. MobileBERT matches feature maps and attention with a bottleneck architecture [13].

Layer mapping. When the student has fewer layers, a map $\phi : \{1..L_S\} \to \{1..L_T\}$ must be chosen: uniform spacing, last-layer only, or learned. TinyBERT's two-stage scheme (general distillation, then task distillation) and DistilBERT's initialization from alternating teacher layers are common recipes [11, 12].

Reported scale of classical encoder compression

DistilBERT reports a 40% smaller model that retains about 97% of BERT's language-understanding performance and runs about 60% faster [11]. TinyBERT reports a model about 7.5× smaller and 9.4× faster at inference that retains a large fraction of BERT-base performance on GLUE [12]. These are paper-level claims on GLUE-type tasks and do not transfer unchanged to generative LLMs.

5.5 The capacity gap

A student can be *too small* for a teacher to help. Cho and Hariharan observed that a larger, more accurate teacher can yield a worse student than a smaller one, and that early-stopped teacher checkpoints often transfer better [8]. Mirzadeh et al. propose a *teacher assistant* of intermediate size to bridge the gap [9]. A heuristic explanation is that the student, unable to fit the teacher's sharp distribution, is pulled by a loss it can only partially reduce, wasting capacity on high-frequency detail. For reasoning LLMs the same phenomenon has been reported: small students trained on long chain-of-thought traces from a very strong teacher can underperform those trained on shorter or weaker-teacher traces [46]. Distillation scaling laws quantify the effect: the student's loss as a function of teacher loss is non-monotone, with an interior optimum teacher strength for a given student size [53].

Remark 5.1 (Capacity gap and sharpening, schematic). Let the student class $\mathcal{S}$ have minimum forward KL to the teacher $\epsilon_{\mathcal{S}}(p) = \min_{q \in \mathcal{S}} \mathrm{KL}(p \,\|\, q)$. If the teacher is sharpened to $p_\beta \propto p^\beta$ ($\beta > 1$) then in general $\epsilon_{\mathcal{S}}(p_\beta)$ is not monotone in β but is 0 at $\beta \to \infty$ only if the argmax map is in $\mathcal{S}$. This is a heuristic remark rather than a theorem: it states that sharpening a teacher can make it *harder* to fit when the student lacks the representational resolution to match the argmax boundaries, and no general monotonicity exists. We include it to flag that "stronger teacher is better" is not a theorem.

5.6 Sequence-level distillation

For autoregressive generation the object of interest is $p(y_{1:T} \mid x)$. Kim and Rush distinguish three objectives [10]:

1. *Word-level KD*: minimize per-token KL, $\sum_t \mathbb{E}_{y_{<t} \sim \text{data}} \mathrm{KL}(p_{\mathrm{T}}(\cdot|y_{<t}) \,\|\, p_\theta(\cdot|y_{<t}))$, with prefixes taken from ground-truth data.
2. *Sequence-level KD (SeqKD)*: sample (or beam-search) $\hat{y} \sim p_{\mathrm{T}}(\cdot \mid x)$ and train the student by maximum likelihood on $(x, \hat{y})$. This approximates $\mathrm{KL}(p_{\mathrm{T}}(y|x) \,\|\, p_\theta(y|x))$ at the sequence level with a Monte Carlo (or mode) estimate.
3. *Interpolation*: mix ground-truth and teacher outputs.

SeqKD is the conceptual ancestor of black-box distillation: the student never sees the teacher's probabilities, only its text (Chapter 7). Its advantage is that it matches distributions over *complete* outputs rather than only next-token distributions at ground-truth prefixes; its limitation is the Monte Carlo noise of a few samples per prompt and the teacher's support collapse under beam search.

5.7 Exposure bias and on-policy training

If the student is trained only on prefixes from data or teacher, at inference it faces its own, different prefixes, an instance of covariate shift known in imitation learning as the compounding-error problem; DAgger addresses it by querying the expert on the learner's own state distribution [20], and the NLP analogue is exposure bias [21].

Theorem 5.1 (Compounding error, informal, after [20])**.** *If the student has per-step error rate ϵ under the* expert's *state distribution, a student that runs on its own states can accumulate cost $O(\epsilon T^2)$ over horizon T; a student trained under its own state distribution (DAgger, no-regret) obtains $O(\epsilon T)$ plus a regret term.*

For LLMs, T is the number of generated tokens, which for reasoning traces is in the thousands, so the quadratic penalty can matter; this motivates the *on-policy* methods of Chapter 6.

5.8 Summary

The theory gives four practical messages: (i) use temperature to expose relative probabilities, but remember it is a hyper-parameter coupled with the loss weighting; (ii) divergence direction is a trade-off between coverage and mode-seeking, and the trajectory distribution may matter more than the direction; (iii) intermediate matching helps when architectures are compatible and costs implementation complexity otherwise; (iv) stronger teachers are not always better, so teacher choice and student size are joint decisions, now quantified by distillation scaling laws [53].

Chapter 6

Knowledge Distillation II: Methods for Language Models

Language models differ from classifiers in three ways that matter for distillation: outputs are *sequences*, vocabularies are huge (tens to hundreds of thousands of types), and the relevant input distribution is open-ended. This chapter covers the main LLM distillation methods and their engineering: offline logit distillation at pretraining scale, on-policy distillation, the divergence choices, hidden-state and attention matching, pruning-plus-distillation, speculative-decoding draft distillation, tokenizer mismatch, multi-teacher and self-distillation, and data selection. Reasoning-trace and black-box distillation, and legal and security aspects, are in Chapter 7.

6.1 Cost accounting for distillation

Before any method, it is useful to count FLOPs. The standard estimate is $6ND$ FLOPs for training a model of N parameters on D tokens (forward plus backward) and $2ND$ for a forward pass alone [172, 173].

Proposition 6.1 (Cost of offline distillation)**.** *Distilling a student with N_S parameters on D tokens, with an online teacher of N_T parameters used in forward-only mode, costs approximately $(6N_S + 2N_T)D$ FLOPs. The overhead relative to plain pretraining of the student on the same tokens is $1 + N_T/(3N_S)$.*

Example 6.1 (A worked cost estimate, illustrative)**.** Teacher 70B, student 8B, $D = 10^{12}$ tokens. Student-only cost: $6 \cdot 8 \times 10^9 \cdot 10^{12} = 4.8 \times 10^{22}$ FLOPs. Teacher forward: $2 \cdot 70 \times 10^9 \cdot 10^{12} = 1.4 \times 10^{23}$. Total 1.88×10^{23}, i.e. 3.9× the student-only cost. The teacher dominates; this is why caching teacher outputs (impossible at this scale for full logits, see below) or distilling on fewer tokens is attractive. If the distilled student reaches the quality of a student trained on 3.9× more tokens, distillation merely breaks even. Distillation scaling laws put numbers on when this happens [53].

Storing teacher logits. Caching the full distribution is infeasible: for $V = 128{,}000$ and 10^{12} tokens at 2 bytes per probability, the cache is $\approx 2.6 \times 10^{17}$ bytes (256 PB). Keeping the top-$k = 32$ entries with a 4-byte index and 2-byte value is $10^{12} \cdot 32 \cdot 6 \approx 1.9 \times 10^{14}$ bytes (about 190 TB), still large but feasible for a lab; sampling a subset of logits per token is also used in practice. Top-k truncation biases the target: the tail mass $\epsilon_k = \sum_{j>k} p_j$ is lost and the retained probabilities must be renormalized or the tail represented by a single "other" bucket. The size of the resulting bias grows with ϵ_k: it is small at confident, low-entropy tokens (the common case) and larger at high-entropy tokens, exactly where soft targets carry the most information.

6.2 Distillation inside frontier-lab training pipelines

Public technical reports describe three uses. First, *pretraining-time distillation*: the Gemma 2 report states that the smaller models (2B and 9B) are trained by distillation from a larger teacher, using the teacher's token probabilities as targets instead of next-token one-hot labels, and argues that this lets small models effectively learn from more information per token [48]; Gemma 3 continues this practice and applies distillation in post-training [49]. Second, *pruning plus logit distillation*: Meta described the Llama 3.2 1B and 3B models as obtained by pruning from Llama 3.1 8B and then using logits of larger Llama 3.1 models as token-level targets during pretraining [51]; the Minitron line of work from NVIDIA does this systematically [61, 62]. Third, *post-training distillation*: the Qwen3 report describes a strong-to-weak pipeline in which smaller models are trained with off-policy distillation (teacher outputs in both thinking and non-thinking modes) followed by on-policy distillation [47]. The Llama 3 paper describes using larger Llama models to generate and filter synthetic post-training data [50].

6.3 On-policy distillation

6.3.1 GKD: generalized knowledge distillation

Agarwal et al. cast distillation as imitation learning: train the student on a mixture of fixed data and sequences generated by the student itself, with the teacher providing per-token targets on the student-generated prefixes [23]. The generalized objective for a divergence D is

$$\mathcal{L}_{\mathrm{GKD}}(\theta) = (1-\lambda)\,\mathbb{E}_{(x,y)\sim\mathcal{D}}\big[D(p_{\mathrm{T}}\|p_\theta)(y \mid x)\big] + \lambda\,\mathbb{E}_{x\sim\mathcal{X}}\,\mathbb{E}_{y\sim p_\theta(\cdot|x)}\big[D(p_{\mathrm{T}}\|p_\theta)(y \mid x)\big], \quad (6.1)$$

where $D(p_{\mathrm{T}}\|p_\theta)(y \mid x) = \frac{1}{|y|}\sum_t D\big(p_{\mathrm{T}}(\cdot|y_{<t},x)\,\|\,p_\theta(\cdot|y_{<t},x)\big)$ is averaged per-token divergence, and the gradient does *not* flow through the sampling process (the student samples are treated as fixed data). $\lambda = 0$ recovers supervised or word-level KD; $\lambda = 1$ is fully on-policy. The paper uses a generalized JSD with skew β and reports improvements on summarization, translation and arithmetic reasoning relative to off-policy baselines, and also shows compatibility with RL fine-tuning.

Algorithm 2 On-policy distillation (GKD-style, token-level)

Require: Teacher p_{T}, student p_θ, prompts $\mathcal{X}$, labeled data $\mathcal{D}$, on-policy fraction λ, divergence D, temperature τ_s for sampling

for step $= 1, 2, \ldots$ **do**
  Sample a batch of prompts $x \sim \mathcal{X}$
  With probability λ: sample $y \sim p_\theta(\cdot \mid x; \tau_s)$ (student rollout); otherwise draw $(x, y) \sim \mathcal{D}$
  Teacher forward on (x, y): obtain $p_{\mathrm{T}}(\cdot \mid x, y_{<t})$ for all t ▷ teacher scoring only, no sampling
  Student forward on (x, y) with gradients: $p_\theta(\cdot \mid x, y_{<t})$
  $\mathcal{L} \leftarrow \frac{1}{|y|}\sum_t D\big(p_{\mathrm{T}}(\cdot|x,y_{<t})\,\|\,p_\theta(\cdot|x,y_{<t})\big)$
  Update θ by gradient descent on $\mathcal{L}$ ▷ no gradient through sampling
end for

6.3.2 MiniLLM: reverse KL with policy gradients

MiniLLM minimizes the sequence-level reverse KL $\mathrm{KL}(p_\theta \,\|\, p_{\mathrm{T}})$ and optimizes it by policy gradient, adding three stabilizers: a single-step decomposition that computes the per-token term in closed form (so only the future-reward part needs sampling), teacher-mixed sampling that draws from $\alpha p_{\mathrm{T}} + (1-\alpha)p_\theta$ to limit reward hacking, and length normalization to counter the bias of long outputs [22]. It reports models from 120M to 13B parameters and gains in response quality and calibration over forward-KL distillation. The derivation of the gradient is in Appendix B; its key identity is

$$\nabla_\theta \,\mathrm{KL}(p_\theta \,\|\, p_{\mathrm{T}}) = -\mathbb{E}_{y\sim p_\theta}\Big[\sum_t \nabla_\theta \log p_\theta(y_t|y_{<t})\,\big(R_t - \text{baseline}\big)\Big], \quad R_t = \sum_{t'\geq t} \log \frac{p_{\mathrm{T}}(y_{t'}|y_{<t'})}{p_\theta(y_{t'}|y_{<t'})}. \tag{6.2}$$

6.3.3 DistiLLM and skewed objectives

DistiLLM replaces plain KL by *skew* KL/RKL, which mixes teacher and student distributions inside the divergence to avoid the infinite-gradient regime, and introduces an adaptive off-policy scheme that reuses student-generated outputs efficiently [24]. The practical point: pure on-policy rollouts are expensive; reuse and mixing give much of the benefit at lower cost.

6.3.4 On-policy distillation as dense-reward RL

The 2025–2026 wave recasts on-policy distillation (OPD) as RL with a *dense per-token reward* equal to the negative reverse KL, $r_t = \log p_{\mathrm{T}}(y_t|y_{<t}) - \log p_\theta(y_t|y_{<t})$. A widely read October 2025 write-up from Thinking Machines Lab describes this as sampling rollouts from the student, scoring every token with the teacher, and applying an RL-style update; it reports, for a Qwen3-based setup, that on-policy distillation lifted AIME'24 from about 60% (after SFT) to about 70% within roughly 150 steps, and cites the Qwen3 report for a three-stage pipeline of 55.0% (off-policy distillation), 67.6% (RL), and 74.4% (on-policy distillation) where the on-policy stage used 1,800 GPU-hours versus 17,920 for RL [47, 52]. These are blog-level and technical-report figures from different setups and should be taken as indicative; they support the qualitative claim that dense teacher feedback is far more sample-efficient than a sparse outcome reward.

When does OPD fail? A 2026 analysis argues that OPD succeeds when student and teacher share compatible reasoning styles and the teacher offers genuinely new abilities, and proposes remedies for failure cases [59]. A geometric analysis finds that OPD updates fewer weights than SFT or RLVR and concentrates rapidly into a low-dimensional subspace [58], which is consistent with the observation that LoRA-rank updates often suffice for RL-style training [115]; we return to this link in Chapter 8. Another line decouples *what prefixes* are used from *which KL direction* and shows four distinct objectives [57]. Practical failure modes: the student's rollouts may be degenerate early in training (so the teacher is scoring garbage); a teacher scoring a long off-distribution prefix gives noisy log-probs; and reverse KL is mode-seeking, risking loss of diversity.

Table 6.1: Reported numbers for on-policy distillation (OPD), as stated in the cited sources; different models, benchmarks and budgets, so not comparable across rows.

Source	Setting	Reported result
Thinking Machines blog [52]	AIME'24, SFT start at ≈60%	≈70% in ≈150 steps (77k prompts, 4 samples/prompt)
Same blog, citing Qwen3 [47]	AIME'24 pipeline stages	55.0% → 67.6% → 74.4%; OPD stage 1,800 GPU-h vs. RL 17,920 GPU-h
Self-distilled reasoner [54]	single model as teacher (privileged info) and student	better token efficiency than RL methods and better than off-policy distillation on math benchmarks (abstract-level claim)
Prefix OPD [56]	distill only early part of student rollouts	matches full OPD on math and OOD benchmarks while cutting training compute by 2–47× (abstract-level claim)
REOPOLD [55]	relaxed OPD with reward clipping, entropy-based sampling, staged schedule	reportedly lets a 7B student match a 32B teacher on visual reasoning

6.4 Matching hidden states and attention in LLMs

Encoder-era recipes (TinyBERT, MiniLM) extend to decoder LLMs, but at scale the practicalities change. Hidden-state losses require a projection when widths differ and add memory for caching teacher activations; layer mapping is less obvious when depth ratios are large. A common compromise is logit distillation for the bulk of training plus a light hidden-state or attention-relation term. Evidence is mixed: Minitron-style recovery after pruning reports that logit-only distillation works well, and that adding intermediate-state losses brings little additional gain for those models in the authors' ablations [61], while attention-relation losses remain useful when architectures differ greatly [14]. We treat intermediate-state matching as a tool for *initialization and stability* rather than the main source of gain.

6.5 Pruning plus distillation

Rather than starting from a random student, one can *derive* the student from the teacher by structured pruning (depth, width, attention heads, embedding channels), then recover accuracy by distillation. Minitron reports that deriving 8B and 4B models from an already pretrained larger model needs up to 40× fewer training tokens per model than training from scratch, and about 1.8× lower compute cost for a whole model family [61]; the follow-up applies the recipe to Llama 3.1 8B→4B and Mistral NeMo 12B→8B [62]. Puzzle extends the idea with a blockwise-local distillation stage and a search over per-layer alternatives (attention variants, FFN widths, removal) under hardware constraints [60]. This is a hybrid of distillation, structured pruning, and NAS (Chapter 11).

Algorithm 3 Prune-then-distill (Minitron-style pipeline, schematic)

Require: Pretrained teacher T, calibration set $\mathcal{C}$, target configuration (depth L', width d', heads h')

1: Compute importance scores for layers/channels/heads from activations on $\mathcal{C}$
2: Prune to target configuration to obtain student initialization S_0
3: **(Optional)** teacher correction: briefly fine-tune T on the distillation corpus to align distributions
4: Distill: minimize $\mathrm{KL}(p_{\mathrm{T}} \| p_\theta)$ on D' tokens, where $D' \ll D_{\text{pretrain}}$
5: Evaluate; if gap is large, iterate with different pruning axes or longer distillation

6.6 Distillation for speculative decoding

Speculative decoding uses a small *draft* model q to propose γ tokens that a large target model p verifies in one parallel pass; accepted tokens are exactly distributed as p [77, 78]. For a given context the per-token acceptance probability is $\alpha = \sum_x \min(p(x), q(x)) = 1 - \mathrm{TV}(p, q)$, and the expected number of tokens per target forward is $(1 - \alpha^{\gamma+1})/(1 - \alpha)$ [77]. Since α depends on total variation, *distilling the draft model toward the target* increases speedup. DistillSpec distils the draft from the target using on-policy data and divergence choices tuned for acceptance, and reports speedups of tens of percent over standard speculative decoding [76]. Here the student *is not* a replacement; it is an accelerator that leaves target-model outputs unchanged, a distinctive property among distillation use cases.

Example 6.2 (Acceptance rate and speedup, computed)**.** With $\gamma = 4$ drafted tokens: $\alpha = 0.6$ gives expected tokens $(1 - 0.6^5)/(0.4) = (1 - 0.0778)/0.4 \approx 2.31$; $\alpha = 0.8$ gives $(1 - 0.8^5)/0.2 = (1 - 0.3277)/0.2 \approx 3.36$; $\alpha = 0.9$ gives $(1 - 0.9^5)/0.1 = (1 - 0.5905)/0.1 \approx 4.10$. If a draft step costs a fraction $c = 0.05$ of a target step, the wall-clock improvement is about tokens$/(1+\gamma c) =$ tokens$/1.2$: roughly $1.9\times$, $2.8\times$, $3.4\times$ respectively. These are derived by formula, not measured.

6.7 Tokenizer-mismatch distillation

Classical KD assumes teacher and student share a vocabulary. With different tokenizers, logits live in different spaces and sequences segment differently. Four strategies exist.

1. *Sequence-level only*: train on teacher-generated text re-tokenized by the student tokenizer. Always possible, loses distributional information.
2. *Alignment of vocabularies*: map tokens by minimum edit distance and merge distributions, as in the knowledge-fusion line of work [73].
3. *Universal logit distillation*: compare sorted probability vectors with an optimal-transport style loss, which needs no token alignment [72].
4. *Shared-space or likelihood-matching*: project to a shared space [74], or match approximate likelihoods of aligned chunks [75].

The common challenge is that a word split into three teacher tokens and two student tokens has no one-to-one next-token distributions; chunk-level likelihood matching sidesteps this by

comparing the probability of whole aligned spans. Tokenizer mismatch matters in practice when distilling across families (for example from a large model with a 128k vocabulary into a small model with a 32k vocabulary, or to a multilingual target).

6.8 Multi-teacher and self-distillation

Multi-teacher. Ensembles of teachers can be averaged in probability space; FuseLLM aligns distributions from several source LLMs with different architectures and fuses them as targets for a target LLM [73]. The theoretical appeal is that an ensemble has lower variance than any member, as in the original distillation motivation [3]. Risks: inherited biases from several sources, license entanglement (Chapter 7), and conflicting reasoning styles.

Self-distillation. A model can teach itself: born-again training with equal-size models [6]; context distillation, where a model prompted with a long instruction or few-shot context becomes the teacher of the same model without the context [79]; self-distillation fine-tuning that rewrites targets in the model's own distribution to reduce forgetting [80]; and on-policy self-distillation, where the teacher copy sees privileged information such as a verified solution and the student sees only the question [54]. These share one idea: a model with more information (context, solution, or simply its own sampled mode) can supervise the same model without that information.

6.9 Data selection for distillation

What the student is trained on matters as much as the loss. Findings from instruction-tuning data selection transfer to distillation corpora: a small, high-quality, diverse set can match a large one [37]; gradient-based influence scores select data for targeted capabilities [38]; LLM-judged quality filtering removes low-quality teacher outputs [39]; and complexity-and-diversity scoring improves data efficiency [40]. For distillation specifically:

- **Prompt coverage.** The student only learns the teacher on the prompt distribution it sees; adversarially rare prompts (safety edge cases, long-tail languages) must be present.
- **Teacher-uncertainty weighting.** Tokens where teacher entropy is high carry more distillation signal than near-deterministic tokens; weighting or sampling by teacher entropy or student–teacher disagreement is a natural heuristic.
- **Difficulty matching to student size.** Capacity-gap results suggest that overly hard examples can hurt small students [46].
- **Deduplication and decontamination.** Teacher outputs often memorize benchmarks; test-set contamination can be detected in black-box settings [82] and must be audited, since benchmark parity is otherwise uninterpretable (Chapter 17).

Practical recipe distilled from the literature

(1) If budget allows, initialize the student by pruning a larger relative instead of random init. (2) Use forward or skewed KL on teacher logits for bulk pretraining-style distillation;

use on-policy reverse-KL-like objectives for post-training and reasoning. (3) Mix on-policy and off-policy data; pure on-policy is expensive and can be unstable early. (4) Track both perplexity and downstream metrics, since they decouple. (5) Audit prompt coverage and safety behaviour of the student explicitly.

Chapter 7

Knowledge Distillation III: Black-Box, Synthetic-Data and Reasoning Distillation

Most readers of distillation news encounter it in a different guise from Chapter 6: a smaller model fine-tuned on text produced by a stronger model, often one reachable only through an API. This chapter treats that regime. It begins with an information-theoretic comparison of text-only and logit-based supervision, then reviews the instruction-imitation era, chain-of-thought distillation, the 2025–2026 wave of distilled reasoning models, the capacity-gap and style-compatibility findings, and finally the legal, security and watermarking questions that distinguish this area from the rest of the paper. Nothing in this chapter is legal advice.

7.1 White-box, grey-box and black-box access

Table 7.1: Access regimes for the teacher.

Regime	What the student sees	Typical methods
White-box	Full logits (or hidden states) at every position	Offline KD, GKD, MiniLLM, Minitron
Grey-box	Top-k log-probabilities or logit-bias oracle	Truncated KD, probability-weighted SFT; also used in extraction attacks [69]
Black-box	Sampled text only (possibly with reasoning trace)	SeqKD [10], instruction imitation, CoT distillation, reasoning distillation

7.1.1 How much does a sample reveal?

Proposition 7.1 (Information in a hard sample versus a soft target)**.** *At a context where the teacher's next-token distribution is p over V symbols, a sampled token reveals at most $H(p)$ bits about p in expectation on the* realization*, but only $O(\log n)$ bits per parameter about p itself after n samples. To estimate a single probability p_k to additive accuracy ε with constant confidence one needs $n = \Theta(p_k(1-p_k)/\varepsilon^2)$ samples at that same context.*

Sketch. The count of token k in n i.i.d. samples is binomial with variance $np_k(1-p_k)$, so the standard error of $\hat{p}_k$ is $\sqrt{p_k(1-p_k)/n}$; setting it to ε gives the stated scaling. □

Example 7.1. Suppose $p_k = 0.05$ and we want $\varepsilon = 0.01$: $n \approx 0.05 \cdot 0.95/10^{-4} = 475$ samples *at the same context*, whereas a single logit read-out gives p_k exactly. In practice contexts rarely

repeat in open-ended generation, so the student never estimates p at a single context; it relies on generalization across contexts. This is why black-box distillation needs many more samples to pass on tail-probability structure, and why it transmits mainly the *mode* behaviour of the teacher: style, format, typical solution paths.

7.2 The instruction-imitation era (2023)

Following the release of open base models, a series of projects fine-tuned them on outputs of proprietary chat models. Stanford Alpaca used instruction-response pairs generated by OpenAI's text-davinci-003 via Self-Instruct-style prompting to tune a LLaMA 7B model [26, 27]. Orca pushed the idea toward *explanation tuning*: prompting GPT-4 and ChatGPT with system messages that elicit step-by-step reasoning and training a student on the resulting traces, rather than only final answers [28]; Orca 2 trains students to select among reasoning strategies per task [29].

Gudibande et al. then reported a sobering finding: students fine-tuned on a proprietary model's outputs imitate its *style* convincingly, so crowd and model judges rate them highly, but targeted evaluations show the capability gap on factuality, coding and problem solving is not closed; the authors argue that the way to improve open models is better base models rather than cheap imitation [30]. This is the first widely cited warning that benchmark parity and human preference can hide skill loss.

7.3 Chain-of-thought distillation

Reasoning ability seemed to be an emergent property of scale, motivating attempts to transfer step-by-step reasoning to small models.

- **Fine-tune-CoT** samples multiple reasoning chains from a large teacher per question, filters by correctness, and fine-tunes a small model on them; diverse reasoning (several chains per question) improves student performance [32].
- **Teaching small models to reason** finetunes on chains from a larger model, with reported gains on arithmetic and commonsense tasks [33]; **specializing** smaller models trades general ability for multi-step math reasoning [34].
- **Distilling step-by-step** trains the student on two tasks, predicting the label and generating the rationale, using the rationale only as auxiliary supervision so that inference does not need rationales; the authors report that smaller models can outperform larger LLMs with fewer training examples [31].
- **Question bootstrapping**: MetaMath augments questions by rewriting and backward reasoning, then distills answers [35].

A common structure underlies these: *sample* multiple teacher traces per prompt (temperature 0.7–1.0 is typical), *filter* by verifying final answers or using a judge, *deduplicate*, and *fine-tune* the student with cross-entropy on traces (and optionally answers only at inference). Algorithm 4 formalizes the loop.

Algorithm 4 Black-box reasoning-trace distillation with verification

Require: Teacher API T, prompts $\mathcal{X}$, verifier $V(x, y) \in \{0, 1\}$ (answer check, unit tests, or judge), samples per prompt m, student S_θ

$\mathcal{D} \leftarrow \emptyset$
for $x \in \mathcal{X}$ **do**
  Draw $y^{(1)}, \ldots, y^{(m)} \sim T(\cdot \mid x)$ ▷ include reasoning trace if exposed
  Keep $\{y^{(j)} : V(x, y^{(j)}) = 1\}$; optionally keep up to k shortest or most diverse
  Add retained pairs $(x, y^{(j)})$ to $\mathcal{D}$
end for
Decontaminate $\mathcal{D}$ against evaluation benchmarks; remove near-duplicates
Fine-tune S_θ by minimizing token-level cross-entropy on $\mathcal{D}$ (1–3 epochs typical)
(Optional) Follow with on-policy distillation or RL with verifiable rewards on the student

Synthetic textbooks. Related in spirit but different in construction, the Phi line trains small models on carefully generated "textbook-quality" synthetic corpora in addition to filtered web data, arguing that data quality can substitute for scale at small sizes [36, 180]. This is distillation in the broad sense, with the teacher's knowledge compiled into pretraining data rather than into per-token targets.

7.4 The distilled-reasoning-model wave (2025–2026)

In January 2025 DeepSeek released R1 and, with it, six dense models obtained by supervised fine-tuning of open base models (Qwen2.5 and Llama 3 variants from 1.5B to 70B) on roughly 800 thousand samples generated by R1 and related data (the report lists 804,745 samples in total); no RL was applied to these students in the paper [41]. The report argues qualitatively that distilling from a strong reasoner is more effective for small models than running large-scale RL on them directly. Since then the pattern has become standard.

7.4.1 What is being transferred?

Two readings coexist. The *capability* reading says long reasoning traces teach genuinely new skills such as self-verification, backtracking and decomposition. The *format/elicitation* reading, supported by the strong results of s1 and LIMO with about a thousand examples, says the base model already contains the competence and a small set of high-quality demonstrations teaches it to allocate more tokens to thinking and to organise them [42, 43]. Both can be true: the tiny-data results imply that the base model's pretraining matters more than the volume of distilled traces, whereas the improvements from million-scale curated sets such as OpenThoughts indicate that scale and data recipe still matter for harder benchmarks [44]. For a practitioner the relevant lesson is that the effect of reasoning distillation is strongly conditional on the base student.

7.4.2 Cost of producing a distilled reasoning model

The dominant cost is typically *teacher generation*, not student training, because decoding is memory-bound.

Table 7.2: Selected distilled-reasoning results as stated in the cited abstracts or pages. Rows use different base models, teachers and benchmarks and are *not* directly comparable. Exact per-model score tables for the DeepSeek-R1-Distill series were not re-verified for this survey and are omitted.

Work	Data / teacher	Student	Reported result
DeepSeek-R1 distilled family [41]	≈800k samples (804,745 listed) from R1-related generation	1.5B–70B dense (Qwen2.5, Llama)	SFT only; report argues distillation beats direct small-model RL
s1 [42]	1,000 curated questions with reasoning traces (s1K)	Qwen2.5-32B-Instruct	AIME24 50% → 57% with budget forcing at test time
LIMO [43]	small curated set (abstract: "1% of the training data" of prior work; count not restated here)	large base model	AIME24 63.3%, MATH500 95.6% (vs. 6.5% / 59.2% for earlier fine-tuned models, as stated)
OpenThoughts3 [44]	≈1.2M examples; teacher QwQ-32B; >1000 ablations	7B	AIME25 53%, LiveCodeBench 51%, GPQA-D 54%; +15.3, +17.2, +20.5 points over R1-Distill-Qwen-7B
Phi-4-reasoning [45]	curated "teachable" prompts; traces by o3-mini	14B	reported to beat R1-Distill-Llama-70B and approach full R1 on several benchmarks

Example 7.2 (Illustrative cost model)**.** Suppose 800,000 traces averaging 4,000 tokens give 3.2×10^9 generated tokens. A teacher with 37B active parameters (an MoE) needs about $2 \cdot 37 \times 10^9 \cdot 3.2 \times 10^9 \approx 2.4 \times 10^{20}$ FLOPs. At a nominal 10^{15} FLOP/s BF16 peak per accelerator, 40% utilization would take about 170 accelerator-hours, but decoding at small batch is bandwidth-bound and achieves far lower utilization; at an assumed 2% the cost is about 8,000 accelerator-hours. SFT of a 7B student for two epochs on the same tokens is $6 \cdot 7 \times 10^9 \cdot 6.4 \times 10^9 \approx 2.7 \times 10^{20}$ FLOPs, a few hundred accelerator-hours at 40% utilization. These numbers are an **illustrative estimate** with assumed sizes and utilizations; they are not measurements of any published run. The qualitative point stands: in black-box pipelines the *generation* bill, plus verification and retries, tends to rival or exceed the student training bill.

7.4.3 On-policy and RL stages after SFT

Qwen3's pipeline illustrates how SFT-style distillation and on-policy distillation combine: off-policy distillation provides a starting point, and on-policy distillation then polishes it with dense per-token feedback [47, 52]. When the teacher's logits are available, on-policy distillation is more sample-efficient than RL with outcome rewards; when only text is available, RL with verifiable rewards remains the route to move beyond the teacher.

7.5 Capacity gap and style compatibility

Li et al. report a *small-model learnability gap*: models at or below roughly 3B parameters do not consistently benefit from long chain-of-thought data or from distillation from much larger models, do better with shorter, simpler chains, and benefit from mixing long and short or large-teacher and small-teacher reasoning [46]. A 2026 analysis of on-policy distillation reaches a compatible conclusion from a different angle: success depends on student and teacher sharing compatible reasoning ("thinking-pattern") styles and on the teacher having something new to offer [59]. The general design rules are:

1. Match trace length and complexity to student size; truncating or summarizing traces can help small students.

2. Prefer teachers in the same family or with similar tokenization and style where possible.

3. Mix difficulty levels; keep some easy examples to avoid brittle overthinking.

4. Reserve long, hard traces for students large enough (empirically, several billion parameters and above) to use them.

7.6 Safety, alignment and bias in students

Distillation transmits more than task skill.

- **Hidden-signal transfer.** Cloud et al. show that a student can acquire a teacher's behavioural trait from data that is semantically unrelated, such as number sequences, even after explicit filtering, but only when teacher and student share the same base model; they warn that filtering outputs may not suffice to prevent transfer of unwanted traits [64]. The result is demonstrated in controlled settings and its prevalence in practical pipelines is unknown.

- **Narrow fine-tuning with broad effect.** Fine-tuning on a narrow task (for example insecure code) can induce broad misalignment [84], and benign fine-tuning can erode safety behaviour [83]. A distilled student fine-tuned on teacher traces is exposed to both effects.

- **Teacher bias inheritance.** Biases, refusal patterns and stylistic quirks of the teacher are transmitted; multi-teacher blends may combine them.

- **Hallucination.** Fine-tuning on facts the student does not know encourages hallucination [85]; teacher traces often contain claims the student lacks the knowledge to ground, so students may learn to produce confident-sounding steps without the underlying competence.

Safety behaviour must therefore be *re-evaluated* on the student rather than assumed (Chapter 17).

7.7 Legal, terms-of-service and IP considerations

Not legal advice

This section summarizes publicly reported facts and the general structure of the issue. Terms of service, licenses and law differ by provider, jurisdiction and time; consult counsel before using any model's outputs for training.

Terms of service. Commercial API providers commonly restrict using outputs to develop models that compete with the provider; the exact clauses vary and change. Open-weight licenses differ: some permit using outputs to improve other models, some attach conditions (for example attribution or naming requirements for derived models), and some are permissive. A practitioner must read the license of the teacher *at the time and version used.*

Reported extraction campaigns. In February 2026 Anthropic published a post alleging "industrial-scale" distillation campaigns against Claude by three labs (DeepSeek, Moonshot AI and MiniMax), reporting more than 16 million exchanges via about 24,000 fraudulent accounts, and stating that this violated its terms of service and regional access restrictions [63]. We report this as an *allegation as published and relayed by the press*, not as an adjudicated fact; the accused parties' positions are not assessed here. The episode illustrates that providers now treat large-scale output harvesting as a security and policy matter, with detection of coordinated account behaviour and restrictions on exposing reasoning traces and log-probabilities as typical countermeasures.

Model extraction research. Academic work predates the dispute. Prediction-API extraction [70], extraction of BERT-based APIs from queries [71], and recovery of part of a production language model, namely its embedding projection layer, from API access [69] show that exposing probabilities or logit-bias controls leaks structure. The more you expose (full log-probs, thought traces), the better a distiller can imitate.

Watermarks and provenance. Statistical watermarks [67] bias token selection by a keyed pseudo-random partition so that a detector can later test for the watermark. Sander et al. show that fine-tuning a student on watermarked text can make the student itself produce detectable traces ("radioactivity"), which opens the possibility of detecting that a model was distilled from a watermarked teacher given access to the student's outputs [68]. Watermarks can be weakened by paraphrasing or filtering, and robustness in realistic distillation pipelines remains an open research question.

IP and copyright. Whether model outputs are copyrightable, whether training on them creates derivative works, and whether contractual restrictions bind third parties who obtain outputs without agreeing to terms are legal questions that, at the time of writing, are unsettled and jurisdiction-dependent; the author does not take a position.

Compliance checklist for distillation projects

(1) Record the teacher, version, date, license/ToS and permitted-use clause. (2) Prefer teachers whose licenses explicitly allow training on outputs. (3) Log provenance of every synthetic sample. (4) Run decontamination against evaluation sets. (5) Re-run safety and bias evaluation on the student. (6) Plan how you would respond if a provider alleges a violation (data deletion, retraining).

7.8 Model collapse and synthetic-data loops

Training successive generations of models on the previous generation's outputs can cause the tails of the distribution to vanish: Shumailov et al. report that models trained on recursively generated data lose information about low-probability events and degrade [65], and theoretical analyses in regression show how errors accumulate [66]. In distillation the relevant mitigations are: mix real data; keep the teacher fixed and strong rather than recursing on students; use higher sampling temperature and diversity filters; and verify correctness. A toy calculation shows the mechanism.

Example 7.3 (Tail loss under resampling, toy)**.** A token with true probability $p = 10^{-3}$ is observed in a finite synthetic corpus of $n = 2000$ tokens with probability $1 - (1 - p)^n \approx 1 - e^{-2} \approx 0.86$; at $n = 500$ it is missed with probability $e^{-0.5} \approx 0.61$. If a student trained on such a corpus sees the token with zero count it assigns it very low probability, and the next generation, trained on the student's outputs, can lose it entirely. Rare-but-real facts and rare-but-important safety behaviours are exactly what is lost first. (Formula-derived toy calculation, not an empirical result.)

7.9 Summary

Black-box distillation is cheap, ubiquitous and effective at transferring style and format, and the reasoning-distillation wave shows it can transfer or elicit substantial capability in sufficiently capable bases. Its limits are informational (text-only supervision), pedagogical (capacity gap), safety-related (trait transfer and misalignment) and legal. The distilled-reasoning results mean that, for many tasks, *the teacher's API is the compression algorithm*, and the question is who is entitled to run it.

Chapter 8

Low-Rank Factorization of Weights

A dense layer $y = \boldsymbol{W}x$ with $\boldsymbol{W} \in \mathbb{R}^{m\times n}$ costs mn parameters and mn multiply-adds per token. If $\boldsymbol{W} \approx AB$ with $A \in \mathbb{R}^{m\times r}$, $B \in \mathbb{R}^{r\times n}$, both costs become $r(m+n)$. This is the oldest post-hoc compression idea for neural networks [90–92], and its applicability to LLMs has been revived by activation-aware formulations. This chapter derives the theory, compares the main methods, analyses rank allocation, and covers hybrids (low-rank plus sparse, plus quantization) and the link to LoRA. Systems aspects (when it actually runs faster) are deferred to Chapter 14.

8.1 Parameter and FLOP accounting

Proposition 8.1 (Break-even rank)**.** *A rank-r factorization of an $m\times n$ matrix saves parameters and per-token FLOPs iff $r < r^\star = \dfrac{mn}{m+n}$. For a target parameter ratio $\rho \in (0, 1]$ (fraction of parameters kept), the required rank is $r = \rho\, r^\star$. For square $n \times n$, $r^\star = n/2$.*

Example 8.1 (Llama-style shapes, illustrative)**.** For an attention projection 4096×4096, $r^\star = 2048$, so keeping 60% of the parameters requires $r = 1229$, which is 30% of the full rank 4096. For an MLP projection of shape 4096×11008 (a common 7B-class shape), $r^\star = \frac{4096\cdot 11008}{15104} \approx 2985$ and $\rho = 0.6$ gives $r \approx 1791$, about 44% of the full rank 4096. Thus a 40% parameter cut demands a rank cut of 56–70% of the matrix's maximal rank, a much harsher truncation than the ratio suggests. This is the central reason low-rank compression of dense weights is difficult.

8.2 Why plain SVD fails on LLM weights

By Theorem 3.1 truncated SVD is optimal for weight-space error. But weight-space Frobenius error is a poor surrogate for function error for two reasons.

Weights are not low-rank. Trained transformer weight matrices have slowly decaying spectra: [102] argue that the *features* (activations) of transformers are approximately low-rank while the *weights* are not, and propose to exploit the former. Our Example 3.1 and Figure 8.1 show how a power-law spectrum behaves.

Input statistics are anisotropic. LLM activations contain a few massive-magnitude channels. A weight direction can have a large singular value but be multiplied by near-zero activations (harmless to remove), or a small singular value but be aligned with an outlier channel (harmful to remove). Weight-space SVD cannot distinguish the two. Proposition 3.1 shows the correct metric: the *output error under the input second-moment matrix.*

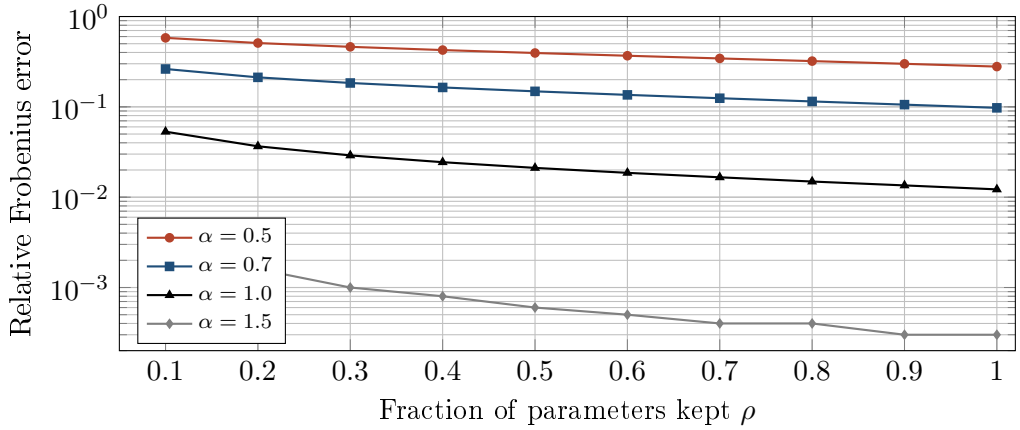


Figure 8.1: **Synthetic.** Relative Frobenius error of the best rank-r approximation of a 4096×4096 matrix whose singular values follow $\sigma_i = i^{-\alpha}$, versus parameter fraction $\rho = 2r/n$ kept. Computed exactly for these invented spectra; real LLM weight spectra differ per layer and are not power laws. Spectra with $\alpha \leq 0.7$ stay far from zero error even at $\rho = 1$, showing why a flat spectrum cannot be compressed.

A counter-observation. LASER finds that replacing some matrices, especially in later MLP layers, with a high-rank approximation (removing the highest-index singular components) can *improve* reasoning accuracy on certain tasks [101]. This suggests part of the weight spectrum encodes noise or rarely-used details; it does not contradict the failure of uniform aggressive truncation.

8.3 Data-aware low-rank methods

All methods below minimize some surrogate of $\mathbb{E}\left\|(\boldsymbol{W} - \hat{\boldsymbol{W}})x\right\|^2$ for a calibration distribution of x, and differ in the surrogate and the allocation of rank.

8.3.1 FWSVD: Fisher-weighted SVD

Hsu et al. weight each row of $\boldsymbol{W}$ by an estimate of its Fisher information, reasoning that parameters with large gradient magnitude matter more: minimize $\sum_i I_i \left\|w_i - \hat{w}_i\right\|^2$ with $I_i = \sum_j \hat{F}_{ij}$, where $\hat{F}$ is an empirical Fisher computed from gradients on task data [93]. This is a *row-weighted* weight-space objective and needs gradients and a task, so it was proposed for fine-tuned encoders rather than general LLM compression.

8.3.2 ASVD: activation-aware scaling

ASVD multiplies $\boldsymbol{W}$ on the right by a diagonal $S = \mathrm{diag}(s_1, \ldots, s_n)$ where $s_j = \left(\frac{1}{N}\sum_k |x_{jk}|\right)^\alpha$ is the mean absolute activation of input channel j raised to a power α (a hyper-parameter), factors $\boldsymbol{W}S = U\Sigma V^\top$, truncates, and absorbs S^{-1} afterwards [94]. A diagonal S treats each input channel independently and ignores correlations; it is cheap and effective at absorbing outlier channels. ASVD also uses a calibration-based search for layer-specific truncation ranks,

reports 10–30% network compression without training, and applies the idea to the channel dimension of KV activations, reporting 50% KV-cache reduction without performance loss [94].

8.3.3 SVD-LLM: truncation-aware whitening

SVD-LLM replaces the diagonal scaling by full whitening with the Cholesky factor S of $\boldsymbol{X}\boldsymbol{X}^\top$ (Proposition 3.1). The key additional result is that the truncation loss in the whitened space equals the discarded singular values exactly, giving a "direct mapping between singular values and compression loss" [95].

Theorem 8.1 (Truncation-aware whitening). *Let $\boldsymbol{X}\boldsymbol{X}^\top = SS^\top$ with S lower triangular and invertible, and $\boldsymbol{W}S = U\Sigma V^\top$. If we discard the singular values with indices in a set $\mathcal{I}$, the resulting output error satisfies $\left\|(\boldsymbol{W} - \hat{\boldsymbol{W}})\boldsymbol{X}\right\|_{\mathrm{F}}^2 = \sum_{i\in\mathcal{I}} \sigma_i^2$, with $\hat{\boldsymbol{W}} = \sum_{i\notin\mathcal{I}} \sigma_i u_i v_i^\top S^{-1}$.*

Proof. $\left\|(\boldsymbol{W} - \hat{\boldsymbol{W}})\boldsymbol{X}\right\|_{\mathrm{F}}^2 = \left\|(\boldsymbol{W} - \hat{\boldsymbol{W}})S\right\|_{\mathrm{F}}^2 = \left\|\sum_{i\in\mathcal{I}} \sigma_i u_i v_i^\top\right\|_{\mathrm{F}}^2 = \sum_{i\in\mathcal{I}} \sigma_i^2$ by orthonormality of $\{u_i\}$, $\{v_i\}$. □

Because $\sum_{i\in\mathcal{I}} \sigma_i^2$ is additive over discarded components, the ranks of every matrix can be compared on a common scale, which makes heterogeneous allocation straightforward. SVD-LLM also proposes a *parameter update with sequential low-rank approximation* after truncation to recover accuracy lost in compression [95]; the abstract does not specify the procedure and we do not reproduce it here. The abstract reports evaluation on seven models from three families at three scales and ten datasets, with the largest advantage over prior methods at high compression ratios.

Algorithm 5 Whitened low-rank compression of one linear layer (SVD-LLM style)

Require: Weight $\boldsymbol{W} \in \mathbb{R}^{m\times n}$; calibration activations $\boldsymbol{X} \in \mathbb{R}^{n\times N}$ (streamed); target rank r; jitter $\epsilon > 0$

1: $G \leftarrow \boldsymbol{X}\boldsymbol{X}^\top$ ▷ accumulate in float32/64 over calibration batches
2: $S \leftarrow \mathrm{chol}(G + \epsilon I)$ ▷ lower triangular; add damping for stability
3: $U, \Sigma, V^\top \leftarrow \mathrm{SVD}(\boldsymbol{W}S)$
4: $A \leftarrow U_{:,1:r}\,\Sigma_{1:r}^{1/2}$ ▷ $m \times r$
5: $B \leftarrow \Sigma_{1:r}^{1/2} V_{:,1:r}^\top S^{-1}$ ▷ $r \times n$; solve a triangular system rather than inverting
6: Replace layer $y = \boldsymbol{W}x$ by $y = A(Bx)$
7: **(Optional)** fine-tune A or B (or LoRA on top) on small data to heal accumulated error

Numerical care. G can be near-singular (dead channels), so damping $\epsilon \approx 10^{-4}\,\mathrm{tr}(G)/n$ is typical practice; the Cholesky is in float64 for large n. These are engineering details the author recalls from common implementations rather than from a single source.

8.3.4 SVD-LLM V2 and Dobi-SVD

SVD-LLM V2 argues that truncating the smallest singular values of a whitened matrix has a computable *theoretical truncation loss* per layer and uses it to assign different compression

ratios to different weight types, so that the total loss is minimized for a fixed budget; it reports evaluation on ten datasets and five LLMs [96]. Dobi-SVD makes the truncation position differentiable and truncates *activations* rather than weights, then reconstructs weights from the truncated activation space, addressing three challenges named in its abstract: the optimal truncation point per matrix, efficient reconstruction of weights, and an "injection" behaviour of SVD that causes information loss [97]. Both operate in the same family as Theorem 8.1; they differ in rank allocation and in how they update the factors.

8.3.5 Cross-layer basis sharing and dictionary learning

Basis Sharing stacks the same-type matrices from several layers, factors them with a common basis plus layer-specific coefficients, and studies which matrices and which layers to share [98]. It converts per-layer low-rank into a hybrid of low-rank and parameter sharing (Chapter 10). CoSpaDi replaces the low-rank assumption (all columns in one subspace) with a *union of subspaces* via sparse dictionary learning with calibration guidance, $\boldsymbol{W} \approx DC$ where D is a dictionary and C has sparse columns; the authors report that at 20–40% compression it outperforms SVD-based and structured-pruning baselines [99]. These methods add expressivity per parameter by moving from a single subspace to many.

8.3.6 SliceGPT: low-rank in disguise

SliceGPT inserts orthogonal transformations (derived from PCA of activations) that leave the output of a transformer unchanged, exploiting RMSNorm/LayerNorm invariance, then deletes rows and columns of the transformed weights, i.e. reduces the *embedding dimension* [100]. It is mathematically a low-rank projection of the residual stream, $x \mapsto Q^\top x$ with $Q \in \mathbb{R}^{d \times d'}$, applied consistently across layers. It reports removing up to 25% of parameters (including embeddings) from Llama 2 70B, OPT 66B and Phi-2 while retaining about 99%, 99% and 90% of dense zero-shot performance respectively, with inference compute for Llama 2 70B at roughly 64% (consumer 24 GB GPUs) and 66% (40 GB A100) of dense [100]. Its relevance here: unlike per-layer factorization, a *shared* projection across the residual stream reduces actual matmul dimensions in all layers at once, which is friendlier to hardware than many separate thin factors.

8.4 Error propagation and bounds

Layer-wise optimality does not imply end-to-end optimality because errors propagate. Let h_l be the hidden state after layer l and δ_l the perturbation introduced by compressing layer l. To first order,

$$\delta h_L \approx \sum_{l=1}^{L} J_{L \leftarrow l}\, \delta_l, \qquad J_{L \leftarrow l} = \frac{\partial h_L}{\partial h_l}, \tag{8.1}$$

so the final-state error is a sum of per-layer errors, each amplified by the Jacobian of the remaining network. In residual architectures $J_{L \leftarrow l} = I + \sum(\cdots)$ is close to the identity plus corrections, which explains why many layers can be perturbed with limited growth but also why errors in early layers are not damped.

Proposition 8.2 (Loss increase to second order)**.** *Let $\mathcal{L}$ be the expected cross-entropy of a model at a local optimum for data-distribution $\mathcal{D}$, and let compression perturb the final hidden state*

Table 8.1: Data-aware low-rank methods (summary of cited descriptions).

Method	Metric / transform	Rank allocation	Notes
SVD	$\|\boldsymbol{W}-\boldsymbol{B}\|_{\mathrm{F}}$	uniform	Optimal in weight space only
FWSVD [93]	row-Fisher-weighted	uniform / manual	Needs gradients; encoder-era
ASVD [94]	diagonal S from mean $\|x\|$	calibrated search per layer	Training-free; 10–30% compression; KV 50%
SVD-LLM [95]	Cholesky whitening	uniform ratio, then update	Exact loss = discarded σ_i^2
SVD-LLM V2 [96]	whitening + theoretical loss	heterogeneous by loss	Ten datasets, five LLMs
Dobi-SVD [97]	differentiable activation truncation	learned per matrix	Weight reconstruction step
Basis Sharing [98]	shared basis across layers	chosen layer groups	Low-rank + sharing hybrid
CoSpaDi [99]	sparse dictionary	per matrix	Union of subspaces
SliceGPT [100]	PCA rotation, delete channels	global width ratio	Dense smaller matrices

(pre-softmax) by δz. Then $\Delta\mathcal{L} \approx \frac{1}{2}\mathbb{E}\big[\delta z^\top F_z\, \delta z\big]$ where $F_z = \mathrm{diag}(p) - pp^\top$ is the Fisher/Hessian of cross-entropy with respect to logits, and $\Delta\mathcal{L}$ is also, to this order, $\mathbb{E}\ \mathrm{KL}(p \,\|\, p+\delta p)$.

Sketch. Cross-entropy as a function of logits is $-\log \mathrm{softmax}(z)_y$; its Hessian in z is $\mathrm{diag}(p) - pp^\top$ independent of y. At a stationary point the gradient vanishes in expectation and the second-order Taylor expansion gives the stated form; the KL form follows because both KL and cross-entropy share this Hessian. □

This justifies using output-space errors (as in Proposition 3.1) as proxies for loss and shows the cost is a *quadratic form in logit perturbation*, weighted by p: perturbations along directions that change high-probability tokens cost more. It also warns that perplexity (average log-loss) is dominated by high-probability events, while capabilities such as reasoning may depend on low-probability, high-stakes tokens, a reason perplexity can stay flat while accuracy drops [189, 190].

8.5 Rank selection and heterogeneous allocation

Let layer l have whitened singular values $\sigma_1^{(l)} \geq \sigma_2^{(l)} \geq \dots$ and size (m_l, n_l), with cost $c_l = m_l + n_l$ parameters per unit rank. Given a budget B for the total parameters, we choose ranks r_l to minimize the additive surrogate loss.

Theorem 8.2 (Greedy optimality of rank allocation)**.** *Minimize $\sum_l \sum_{i>r_l} (\sigma_i^{(l)})^2$ subject to $\sum_l c_l r_l \leq B$, $r_l \in \mathbb{Z}_{\geq 0}$. Because each $\sum_{i>r} \sigma_i^2$ is a convex, decreasing function of r (its decrements σ_r^2 are nonincreasing), the* fractional-knapsack *greedy that repeatedly keeps the component with the highest ratio $(\sigma_i^{(l)})^2/c_l$ is optimal for the continuous relaxation and within one component of optimal for the integer problem. Equivalently, there is a threshold λ such that layer l keeps all components with $(\sigma_i^{(l)})^2 > \lambda\, c_l$.*

Proof. The objective is separable; the marginal gain of increasing r_l by one is $(\sigma_{r_l+1}^{(l)})^2$ at cost c_l. Since these gains are nonincreasing in r_l, the feasible set of "items" within each layer is

already ordered by ratio, so picking items globally in decreasing ratio order and stopping when the budget is exhausted is the standard fractional-knapsack optimum. Rounding the last item costs at most one component's worth of loss. □

Allocation by this rule gives layers with flat whitened spectra (hard to compress) higher rank, and layers with fast decay lower rank; layer types differ in practice, so uniform ratios are rarely optimal. ASVD uses a sensitivity-driven search and SVD-LLM V2 a loss-driven allocation [94, 96], both approximating this principle. A caveat: the surrogate is additive and local, so it ignores the interaction of errors across layers; sensitivities measured by end-to-end perplexity or KL are costlier but more faithful (the Jacobian weights in the propagation formula).

Example 8.2 (Allocation by threshold, numerical toy)**.** Two layers with $c = 8192$ (square 4096) and $c = 15104$ (4096×11008), with $\sigma_i^2 \propto i^{-1.4}$ for layer 1 and $\propto 3\, i^{-1.4}$ for layer 2 (layer 2 carries three times the energy). The keep condition $\sigma_i^2 > \lambda c$ gives $i < (\kappa/\lambda c)^{1/1.4}$: ranks scale as $(\kappa/c)^{0.714}$, so layer 2's rank is $(3 \cdot 8192/15104)^{0.714} = (1.627)^{0.714} \approx 1.42$ times layer 1's rank. Uniform ranks would be suboptimal; the toy shows the direction (more rank to energetic, cheaper layers) rather than realistic magnitudes.

8.6 Healing and recovery

After truncation the factors A, B are a natural initialization for further training. Three recovery strategies exist: (i) fine-tune *all* factor entries on a small corpus; (ii) freeze A, B and add LoRA [105], which reintroduces a second low-rank term and thus parameters; (iii) distill from the original model (teacher) into the factored student, using logits or hidden states (Chapter 6). Option (iii) is natural: the original dense model is available as teacher at zero acquisition cost, and on-policy self-generated data avoids the need for external data. SVD-LLM includes an update step of this kind in its design [95]. The author's expectation (not a result established here) is that healing is most valuable exactly where one-shot truncation has crossed the threshold at which downstream skills degrade.

8.7 Low-rank plus sparse

Robust PCA decomposes $\boldsymbol{W} = L + S$ with L low-rank and S sparse by convex relaxation $\min \|L\|_* + \mu \|S\|_1$ [89]. For LLMs the sparse part captures outliers that a low-rank basis would waste rank on. LoSparse prunes the sparse component iteratively while training and keeps a low-rank part that approximates the coherent structure of the expressive neurons [110]. OATS combines outlier-aware scaling (as in ASVD/Wanda-style activation weighting) with an alternating-thresholding solver for sparse plus low-rank, applied post-training [111]. The cost model is $\rho = r(m+n)/(mn) + s$ for sparse density s; with unstructured sparsity the sparse part needs index storage and sparse kernels, and with 2:4 sparsity only certain GPUs accelerate it, so the *wall-clock* benefit depends strongly on hardware (Chapter 14).

8.8 Low-rank plus quantization

Quantizing the bulk of $\boldsymbol{W}$ and keeping a small high-precision low-rank correction recovers much of the error from aggressive quantization. Let $\boldsymbol{W} \approx Q + LR$ with Q low-bit.

- **LoftQ** alternates between quantizing $\boldsymbol{W} - LR$ and refitting LR by SVD of the quantization residual so that LoRA initialization compensates quantization error [107].
- **LQ-LoRA** performs an iterative low-rank plus quantized decomposition with Fisher-weighted objective and a mixed-quantization configuration under a bit budget, then fine-tunes the low-rank part while the quantized part stays frozen [108].
- **QLoRA** is the special case with L, R initialized at zero [106].
- **CALDERA** approximates $\boldsymbol{W} \approx Q + LR$ with Q, L and R *all* quantized, optimizing the calibration-weighted error $\left\|(Q + LR - \boldsymbol{W})\boldsymbol{X}^\top\right\|_{\mathrm{F}}^2$; its abstract reports superiority to existing post-training methods below 2.5 bits per parameter on Llama 2 (7B, 13B, 70B) and Llama 3 (8B), together with approximation-error upper bounds from a rank-constrained regression framework, and the factors can be further fine-tuned with LoRA [109].

Proposition 8.3 (Effective bits per weight)**.** *If Q uses b_Q bits per entry and L, R use b_{LR} bits per entry at rank r, the effective bits per original weight is $b = b_Q + b_{LR}\,\dfrac{r(m+n)}{mn}$ (ignoring scales and metadata).*

Example 8.3. For a 4096×4096 matrix with $b_Q = 2$ bits, rank $r = 64$ factors in 16-bit: $b = 2 + 16 \cdot \frac{64 \cdot 8192}{4096^2} = 2 + 16 \cdot 0.03125 = 2.5$ bits. Quantizing the factors to 4 bits reduces the overhead from 0.5 to 0.125 bits, for $b \approx 2.125$. The overhead is an *additive* cost that is only worthwhile if the low-rank correction recovers more quality than spending the same bits on Q (for instance, raising b_Q to 2.5 bits).

8.9 LoRA, adapters and low-rank structure of updates

LoRA parameterizes the *update* $\Delta\boldsymbol{W} = BA$ with rank $r \ll \min(m, n)$ [105], motivated by intrinsic-dimension findings that fine-tuning happens in a low-dimensional subspace [117, 118]. It is compression of the *fine-tuning delta*, not of the base model, and in that sense LoRA, adapters [116] and delta-compression (Chapter 11) belong to one family. Empirical studies sharpen the picture: LoRA learns less and forgets less than full fine-tuning, with the gap larger for knowledge-intensive tasks such as continued pretraining on code or math [114]; and for RL-style post-training, a small-capacity LoRA may match full fine-tuning because policy-gradient updates carry few bits per episode [115]. The latter fits the geometry finding that on-policy distillation updates collapse into a low-dimensional subspace [58]: when the *information content of the update* is low, low-rank adapters suffice. GaLore applies the low-rank idea to *gradients* to save optimizer memory during training rather than parameters [113]; AdaLoRA allocates rank across matrices with SVD-parameterized adapters [112], an adaptive-rank idea parallel to Theorem 8.2.

LoRA is the low-rank "delta"; SVD compression is the low-rank "base"

Low-rank structure is much more plausible in *updates* (task-specific, low information) than in *pretrained weights* (broad knowledge, high information). This asymmetry explains why LoRA succeeded at scale and why post-hoc SVD of base weights remains hard.

8.10 Low-rank structure in activations and the KV cache

Because activations are more compressible than weights, compressing them is often more fruitful. DeepSeek-V2's Multi-head Latent Attention (MLA) caches a compressed latent vector from which keys and values are reconstructed through learned up-projections, reducing KV-cache size; this is low-rank structure built into the architecture and trained from scratch [104]. Post-hoc analogs: ASVD's KV channel reduction [94] and Palu, which applies low-rank projection to KV-cache with grouped decomposition and cooperating kernels [103]. The common lesson is that when the compressible object is an activation, one can fold the up-projection into the attention computation (absorb matrices), so the compressed dimension is used end-to-end without reconstruction.

8.11 Summary of the low-rank landscape

1. Use whitened (activation-aware) truncation; plain SVD is the wrong objective.
2. Allocate rank heterogeneously by a global additive loss and verify with end-to-end metrics.
3. Expect compression ratios of tens of percent at modest degradation; beyond that, healing or hybridization (quantization, sparsity, shared basis) is needed.
4. The memory-versus-speed question depends on rank and kernels, covered in Chapter 14.
5. Low-rank is natural for updates (LoRA) and activations (KV latents); it is least natural for pretrained weight matrices themselves.

Chapter 9

Structured Matrices, Tensor Decompositions, Hashing and Embedding Compression

Low-rank is one point in a large family of *structured* parameterizations $\boldsymbol{W} = \mathcal{M}(\theta)$ with $\dim \theta \ll mn$ and a fast matrix–vector product. This chapter covers Kronecker, Toeplitz/circulant, butterfly and Monarch families, tensor-network formats (CP, Tucker, tensor train), the hashing trick, and the special case of embedding tables and vocabularies, which hold a large share of the parameters of small LLMs. A recurring theme: these parameterizations shine when used *from the start of training*; applying them post hoc to a trained dense LLM is generally lossy because the pretrained weights do not live near the chosen structure.

9.1 Framework: structure, parameters, and matvec cost

A structured family is characterized by (i) the number of free parameters p, (ii) the cost of $x \mapsto \boldsymbol{W}x$, and (iii) its *expressivity*: which matrices it can represent exactly. Table 9.1 summarizes the families treated here for an $n \times n$ matrix.

Table 9.1: Structured families for $n \times n$ matrices (standard complexities).

Family	Parameters	Matvec cost	Notes
Dense	n^2	$O(n^2)$	Baseline
Low-rank (rank r)	$2nr$	$O(nr)$	Subspace assumption
Circulant	n	$O(n \log n)$ via FFT	Very restrictive [124]
Toeplitz / low displacement rank	$O(n)$–$O(rn)$	$O(rn \log n)$	Structured transforms [125]
Kronecker $A \otimes B$	$2n$ (for $\sqrt{n}$-sized factors)	$O(n^{1.5})$	Rank-1 in a rearranged space
Butterfly (product of $\log n$ factors)	$O(n \log n)$	$O(n \log n)$	Expresses FFT-like transforms [120]
Monarch ($b = \sqrt{n}$)	$2n^{3/2}$	$O(n^{3/2})$	Hardware-friendly [119]
Tensor train (rank r, d modes)	$O(dr^2 n^{2/d})$	$O(dr^2 n^{1+1/d})$	Compact, kernels are costly [128]
Hashing (budget K)	K	$O(n^2)$	Memory only; no FLOP reduction [136]

9.2 Kronecker products

For $A \in \mathbb{R}^{m_1 \times n_1}$, $B \in \mathbb{R}^{m_2 \times n_2}$, $A \otimes B \in \mathbb{R}^{m_1 m_2 \times n_1 n_2}$ with blocks $a_{ij}B$. The key identity is

$$(A \otimes B)\,\mathrm{vec}(X) = \mathrm{vec}(B\,X\,A^\top), \qquad X \in \mathbb{R}^{n_2 \times n_1}, \tag{9.1}$$

so multiplication costs $O(m_2 n_2 n_1 + m_2 n_1 m_1)$ rather than $O(m_1 m_2 n_1 n_2)$. Parameters drop from $m_1 m_2 n_1 n_2$ to $m_1 n_1 + m_2 n_2$.

Theorem 9.1 (Nearest Kronecker product, Van Loan–Pitsianis)**.** *Let $\mathcal{R}(\boldsymbol{W}) \in \mathbb{R}^{m_1 n_1 \times m_2 n_2}$ be the rearrangement whose rows are the vectorized $m_2 \times n_2$ blocks of $\boldsymbol{W}$. Then $\|\boldsymbol{W} - A \otimes B\|_{\mathrm{F}} = \big\|\mathcal{R}(\boldsymbol{W}) - \mathrm{vec}(A)\mathrm{vec}(B)^\top\big\|_{\mathrm{F}}$, so the best Kronecker approximation is given by the leading singular triplet of $\mathcal{R}(\boldsymbol{W})$.*

Proof. Each block of $A \otimes B$ is $a_{ij}B$; its vectorization is $a_{ij}\mathrm{vec}(B)$, so $\mathcal{R}(A \otimes B) = \mathrm{vec}(A)\mathrm{vec}(B)^\top$, and $\mathcal{R}$ is a permutation of entries and thus preserves the Frobenius norm. The claim follows from Theorem 3.1 with $r = 1$. □

The generalization to a *sum* of k Kronecker products corresponds to a rank-k approximation of $\mathcal{R}(\boldsymbol{W})$ and gives a graded family between pure Kronecker and dense. Kronecker products compress recurrent networks by 15–38× on small IoT-oriented models in the authors' report [126], and Kronecker adapters offer a parameter-efficient alternative to LoRA [127]. For LLMs the difficulty is expressivity: a single Kronecker product imposes that all blocks are scalar multiples of one another, which the weights of a pretrained transformer do not satisfy; $\mathcal{R}(\boldsymbol{W})$ typically has a slowly decaying spectrum.

9.3 Circulant, Toeplitz and displacement structure

A circulant matrix $C = \mathrm{circ}(c)$ is diagonalized by the DFT, so $Cx = \mathcal{F}^{-1}(\mathcal{F}c \odot \mathcal{F}x)$ in $O(n \log n)$ time with n parameters. Cheng et al. proposed circulant projections for fully connected layers [124]; Sindhwani et al. generalized to Toeplitz-like matrices with low *displacement rank* r, i.e. matrices M for which $M - ZMZ^\top$ has rank r for a shift Z, giving a tunable family between very restrictive and dense [125]. Related FFT-based random-feature constructions such as Fastfood replace dense Gaussian projections by products of diagonal, Hadamard and permutation matrices [123]. Such families are valuable on resource-constrained speech and vision models; for transformers, circulant structure is attractive as an *operator* (long convolutions) rather than as a replacement for the FFN weights.

9.4 Butterfly and Monarch

9.4.1 Butterfly factorizations

Many fast linear transforms (DFT, Hadamard, DCT) factor as products of $\log_2 n$ sparse "butterfly" matrices, each with two non-zeros per row. Dao et al. showed that a product of butterfly factors with learnable entries can represent a broad class of structured matrices with $O(n \log n)$ parameters and operations [120]; Kaleidoscope matrices (products of butterfly and

its transpose) represent all structured matrices with near-optimal parameters [121]. In practice butterfly's access pattern is not hardware-friendly (scattered memory); Pixelated Butterfly uses a flat, block-sparse relaxation that maps to GPU block-sparse kernels [122].

9.4.2 Monarch matrices

Definition 9.1 (Monarch matrix [119])**.** For $n = b^2$, a Monarch matrix is $\boldsymbol{M} = \boldsymbol{P}^\top \boldsymbol{L}\boldsymbol{P}\boldsymbol{R}$ where $\boldsymbol{L}$ and $\boldsymbol{R}$ are block-diagonal with b blocks of size $b \times b$, and $\boldsymbol{P}$ is the "transpose" permutation that views a length-n vector as a $b \times b$ grid and transposes it.

Multiplying by $\boldsymbol{M}$ is two batched small matmuls interleaved with a transpose, which maps to existing batched-GEMM kernels. Parameters: $2b \cdot b^2 = 2n^{3/2}$; FLOPs: $2n^{3/2}$ multiply-adds.

Example 9.1 (Monarch parameter count)**.** For $n = 4096$, $b = 64$: parameters $= 2 \cdot 64 \cdot 64^2 = 524{,}288$ against 16,777,216 dense, about 3.1% (a 32× reduction), and FLOPs fall by the same factor. Generalizations with different block counts interpolate between dense ($b = 1$) and extreme sparsity. The price is expressivity: a Monarch matrix has at most $O(n^{3/2})$ degrees of freedom.

Dao et al. show Monarch matrices are expressive enough to represent many transforms exactly (including the DFT and convolutions), demonstrate training of GPT-2-scale models and ViT variants with reduced training time at comparable quality, and give a Monarch *projection* that finds the best Monarch approximation to a given dense matrix by solving a collection of independent rank-one problems; this enables sparse-to-dense fine-tuning where dense pretrained weights are first projected onto Monarch form [119]. For LLM compression the post-hoc projection route has the same limitation as SVD: the quality of the projection depends on how Monarch-like the pretrained weights are. To the author's knowledge, no widely deployed open LLM at the time of writing uses butterfly or Monarch weights for its main linear layers; the technique is more visible in research on efficient attention and long-convolution models.

9.5 Tensor decompositions

Tensors generalize matrix factorization to multilinear structure [130]. Three classical formats are used for compression.

Definition 9.2 (CP, Tucker and tensor-train formats)**.** For an order-d tensor $\mathcal{T} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$:

- **CP (canonical polyadic)**, rank R: $\mathcal{T} = \sum_{j=1}^{R} a_j^{(1)} \circ \cdots \circ a_j^{(d)}$; $R \sum_k n_k$ parameters.
- **Tucker**, multilinear rank $(r_1, \ldots, r_d)$: $\mathcal{T} = \mathcal{G} \times_1 U_1 \times_2 \cdots \times_d U_d$; $\prod r_k + \sum n_k r_k$ parameters.
- **Tensor train (TT)**, ranks $(r_0{=}1, r_1, \ldots, r_d{=}1)$: $\mathcal{T}_{i_1 \cdots i_d} = G_1[i_1] G_2[i_2] \cdots G_d[i_d]$ with core slices $G_k[i_k] \in \mathbb{R}^{r_{k-1} \times r_k}$; $\sum_k r_{k-1} n_k r_k$ parameters [129].

CP and Tucker decompositions were applied to convolution kernels [131, 132]. For fully connected layers, *tensorizing* reshapes the $n \times n$ matrix, with $n = n_1 \cdots n_d$, into a tensor with index pairs (i_k, j_k) and represents it in TT format ("TT-matrix" or matrix product operator) [128, 133].

Example 9.2 (TT-matrix parameter count)**.** Reshape a 4096×4096 matrix into $d = 4$ modes with $n_k = 8$ (since $8^4 = 4096$) on each side. Each core has shape $r_{k-1} \times 8 \times 8 \times r_k$, so $64\, r_{k-1} r_k$ entries. With uniform ranks $(1, r, r, r, 1)$: parameters $= 64(r + r^2 + r^2 + r) = 64(2r + 2r^2)$. At $r = 16$ this is $64 \cdot 544 = 34{,}816$, versus 16,777,216: a 482× reduction. At $r = 64$: $64 \cdot (128 + 8192) = 532{,}480$, a 31.5× reduction. Whether the achievable ranks suffice for a given layer is a property of the weights, not of the format.

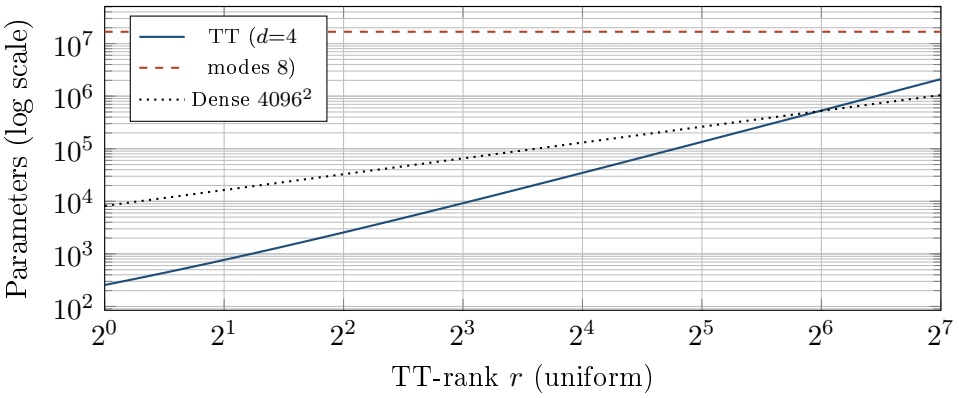


Figure 9.1: Exact parameter counts as functions of rank for a 4096×4096 layer (formulas of Example 9.2 and Proposition 8.1). Even at $r = 128$ the TT count (≈ 2.1M) is about 8× below dense. TT-rank and matrix rank mean different things, so curves are not comparable at equal r, and the *ranks that suffice* for real weights are weight-dependent and not shown.

Theorem 9.2 (TT-SVD quasi-optimality [129])**.** *Let $\mathcal{A} \in \mathbb{R}^{n_1 \times \cdots \times n_d}$ and $\mathcal{A}_{best}$ the best TT approximation with ranks at most (r_k) in Frobenius norm. The TT-SVD algorithm (sequential truncated SVDs of unfoldings) yields $\mathcal{A}_{TT}$ with the same ranks bounds and $\|\mathcal{A} - \mathcal{A}_{TT}\|_{\mathrm{F}} \leq \sqrt{d-1}\, \|\mathcal{A} - \mathcal{A}_{best}\|_{\mathrm{F}}$.*

Thus TT-SVD is within a factor $\sqrt{d-1}$ of optimal, in contrast to CP where best low-rank approximation may not exist. In deep-network applications the question is less about approximating a fixed tensor and more about training in TT form: Tensorizing Neural Networks trains TT-layers by back-propagating through the cores, reporting large compression on fully connected layers of image classifiers [128].

Why tensorization is hard for LLMs. (i) The reshaping of an n-dimensional axis into $(n_1, \ldots, n_d)$ has no semantic basis for transformer features; different factorizations give different TT-ranks, and ordering of modes matters. (ii) TT contraction is a chain of small matmuls with poor arithmetic intensity on GPUs, so FLOP reductions need not become speedups (Chapter 14). (iii) As with SVD, post-hoc TT decomposition of pretrained LLM weights loses accuracy at high ratios and needs substantial retraining. TT is more established for *embedding tables* and recommendation models, discussed next [134, 135].

9.6 Hashing

HashedNets reduce a virtual weight matrix of size $m \times n$ to $K \ll mn$ real parameters by assigning each position (i, j) to a bucket $h(i, j) \in \{1..K\}$ via a hash function and sharing the parameter,

optionally with a sign hash $\xi(i,j) \in \{\pm 1\}$ to make collisions unbiased: $\boldsymbol{W}_{ij} = \xi(i,j)\, w_{h(i,j)}$ [136]. The forward pass is a dense matmul on the virtual matrix (or a custom kernel that gathers); backward pass accumulates gradients into shared buckets. The idea derives from feature hashing [137].

Proposition 9.1 (Unbiasedness of signed hashing)**.** *Let $x, y \in \mathbb{R}^n$ and let $\phi(x)_k = \sum_{j:h(j)=k} \xi(j) x_j$ with independent uniform hash $h : [n] \to [K]$ and independent signs. Then $\mathbb{E}\langle \phi(x), \phi(y)\rangle = \langle x, y\rangle$, and* $\mathrm{Var} \le \frac{1}{K}\big(\|x\|^2 \|y\|^2 + \langle x, y\rangle^2\big)$.

Sketch. $\langle \phi(x), \phi(y)\rangle = \sum_k \sum_{j,j':h(j)=h(j')=k} \xi(j)\xi(j') x_j y_{j'}$. The cross terms $j \neq j'$ have zero expectation because $\mathbb{E}[\xi(j)\xi(j')] = 0$; the diagonal terms give $\sum_j x_j y_j$. The variance bound follows by computing second moments of the collision indicator, which equal $1/K$ for pairs. □

This justifies hashing as a randomized sketch with variance decaying as $1/K$. Hashing offers pure memory compression, with no FLOP reduction unless combined with other structure, and random collisions are blind to which weights matter, so hashed models usually need training from scratch. Hash embeddings combine several hash functions with learned importance weights to represent huge vocabularies [138], and compositional embeddings use complementary partitions (quotient–remainder) so that each category gets a unique combination of a few small tables [139].

9.7 Embedding tables and vocabularies

9.7.1 The share of parameters in embeddings

With vocabulary V and hidden size d, the input embedding has Vd parameters; if the output head is untied it has another Vd. For small models this is a large fraction.

Table 9.2: Embedding parameters for hypothetical configurations (computed as Vd; illustrative shapes, not specific named models). "Share" is the fraction of a nominal total parameter count.

V	d	Vd (M)	Untied total (M)	Nominal total (M)	Share (untied)
32,000	4096	131	262	7,000	3.7%
128,000	4096	524	1,049	8,000	13%
128,000	2048	262	524	1,200	44%
128,000	1024	131	262	400	66%
256,000	2304	590	1,180	2,600	45%

For sub-billion-parameter models the embedding matrices can exceed the transformer body, so *tying* input and output embeddings [142, 143] is the most effective single compression move at that scale, and is used by on-device-oriented designs; MobileLLM reports combining embedding sharing with deep-and-thin architectures and layer sharing for sub-billion models [154].

9.7.2 Factorized embeddings

ALBERT factorizes the embedding as $E = E_1 E_2$ with $E_1 \in \mathbb{R}^{V \times e}$, $E_2 \in \mathbb{R}^{e \times d}$ ($e \ll d$), reducing parameters from Vd to $Ve + ed$ [140]. For $V = 128{,}000$, $d = 4096$, $e = 512$: $Ve = 65.5$M and

$ed = 2.1$M, a total of 67.6M versus 524M, a 7.8× reduction of the table. The factorization is a rank-e constraint on the embedding matrix; its costs are an extra $e \times d$ projection per token and potentially degraded rare-token representations.

9.7.3 Tensorized and hashed embeddings

TT-embeddings represent the $V \times d$ table as a TT-matrix and reconstruct rows on the fly; TT-Rec reports large memory savings for recommendation models and Khrulkov et al. evaluate TT embeddings for NLP tasks [134, 135]. For LLMs, token embeddings are accessed densely during output projection (the LM head touches *all* rows), so any scheme that reconstructs rows on demand must be amortized over the full vocabulary matmul, which limits the benefit for the output head but not for the input lookup.

9.7.4 Vocabulary and tokenizer size as compression choices

Tokenization is itself lossless compression of text into token IDs. A larger vocabulary shortens sequences (fewer tokens per document, so lower attention and FFN cost per document and a shorter KV cache) but enlarges the embedding table and softmax; Tao et al. study scaling laws with vocabulary size and argue that optimal vocabulary size grows with model size and that many current models are under-vocabularied relative to their compute budgets [141]. The practical implication for compression is that *vocabulary reduction* for a specific domain or language (trimming unused tokens, restricting the output head) lowers parameters and softmax cost at no cost for text covered by the retained vocabulary, a tool particularly useful for speculative-decoding draft models and on-device models, but one that complicates tokenizer compatibility with the teacher (Chapter 6).

9.8 Evidence and assessment

1. **Where structure has won.** Embeddings (weight tying, factorization), recommendation tables (TT, hashing, compositional), and speech and edge models (Kronecker, circulant). Also long-convolution sequence models where structured operators replace attention.

2. **Where it has not (yet).** Replacing the main dense layers of a pretrained LLM with Monarch or TT cores. Post-hoc projection loses accuracy; training from scratch forfeits the pretrained model; and kernels are less mature than dense GEMM.

3. **Hardware lens.** Structures that decompose into *batched dense blocks* (Monarch, block-sparse butterfly) fit tensor cores. Structures that need FFTs, gathers, or long chains of tiny matmuls (circulant, TT, hashing) do not.

Takeaway

Structured and tensorized parameterizations are powerful tools for *designing* efficient models and for compressing embeddings, but they are not currently the leading way to compress a *pretrained* dense LLM. Their usefulness hinges on training-time adoption and on kernels that turn parameter savings into speed.

Chapter 10

Parameter Sharing and Recurrence

Distillation and factorization reduce *computation and storage together*. Parameter sharing is different: it reduces the number of *distinct* parameters while leaving the number of operations unchanged, because the same weights are applied several times. This chapter formalizes that trade, reviews cross-layer sharing from ALBERT to relaxed recursive transformers, covers looped and recurrent-depth language models and Mixture-of-Recursions, and places them against mixture-of-experts on a common parameters-versus-FLOPs map.

10.1 Forms of sharing

1. **Weight tying**: input and output embeddings share a matrix (Chapter 9) [142, 143].
2. **Cross-layer sharing**: layers l and l' use the same block Φ. ALBERT shares all layers' parameters (or attention or FFN only) and, with factorized embeddings, obtains a model with fewer parameters than BERT-large while reporting state-of-the-art results on GLUE, RACE and SQuAD in its abstract [140]. Takase and Kiyono compare sharing patterns, in particular sequential, cyclic and reverse-cyclic assignment of K unique blocks to L layers, and find that cyclic assignments are generally preferable to sequential ones for the same parameter budget [146].
3. **Recurrence in depth**: a block is applied until a halting criterion holds or for a fixed number of iterations (Universal Transformer with adaptive computation time [144]; Deep Equilibrium Models, which iterate to a fixed point and differentiate implicitly [145]).
4. **Shared bases**: layers share low-rank bases with layer-specific coefficients [98] (Chapter 8).

10.2 Accounting: what sharing saves and what it does not

Let a transformer of L layers have per-layer parameter count p and per-token FLOPs $f \approx 2p$. Share K unique blocks, cycled $R = L/K$ times.

Proposition 10.1 (Costs of cyclic sharing)**.** *Compared with the unshared L-layer model: (i) stored parameters fall from Lp to Kp (a factor R); (ii) FLOPs per token are unchanged at $\approx 2Lp$; (iii) KV-cache size per token is unchanged (one cache per layer* application*), unless keys and values are explicitly shared; (iv) bytes of weights read from HBM per decoded token are unchanged at batch size one, unless the shared block fits in on-chip memory and is reused from there.*

Sketch. (i) and (ii) are immediate. (iii): each application produces distinct hidden states, hence distinct keys and values. (iv): a decode step executes L block applications in sequence; between

two applications of the same block other layers' weights stream through caches, and a block of hundreds of megabytes exceeds the tens of megabytes of on-chip cache of current accelerators. □

Example 10.1 (Sharing does not speed up memory-bound decoding)**.** Take a 7B-parameter-class model, $L = 32$, $p \approx 0.2$B (BF16: 0.4 GB per layer). Cycle $K = 8$ unique blocks four times. Stored weights: $8 \times 0.4 = 3.2$ GB instead of 12.8 GB, which is a clear capacity win (it fits on a smaller GPU). But decode at batch size one still reads $32 \times 0.4 = 12.8$ GB of weights per token (each application streams its block from HBM), so the bandwidth-bound latency is unchanged. Sharing converts a *capacity* problem into a non-problem and leaves the *latency* problem intact. At large batch sizes, weights are amortized over many tokens and the model is compute-bound; there too FLOPs are unchanged. (Illustrative arithmetic.)

The sharing trade

Sharing saves storage and resident memory (and can make training cheaper in optimizer state), but it does not save FLOPs, KV cache or streamed bytes. It is most valuable when the binding constraint is *capacity* (on-device, distribution size) or when extra compute per parameter is itself useful (reasoning, test-time scaling).

10.3 Relaxed Recursive Transformers

Bae et al. convert a pretrained transformer into a *Recursive Transformer* with K unique layer blocks looped to reach the original depth, and then *relax* the constraint with depth-wise LoRA modules so that each loop can deviate slightly from the shared weights [147]. In symbols, the ℓ-th application of the block uses

$$\Phi_\ell = \Phi_{\ell \bmod K} + B_\ell A_\ell, \qquad \operatorname{rank}(B_\ell A_\ell) \le r, \tag{10.1}$$

where Φ_k are shared and $B_\ell A_\ell$ is an unshared low-rank correction per application. Total unique parameters are $Kp + L \cdot r(m+n) \cdot$ (number of matrices). They initialize from the pretrained model (rather than training from scratch), using a truncated SVD of the residual between each original layer's weights and the shared weights to initialize LoRA [147]. The paper reports that recursive models initialized this way outperform similarly sized vanilla pretrained models and distillation baselines (it names a recursive Gemma 1B against TinyLlama 1.1B and Pythia 1B) and recover much of full-size Gemma 2B's performance. It also proposes *continuous depth-wise batching* with early exiting, and argues from analysis (not measurement) that this could give up to 2–3× throughput [147].

The algorithm shows that relaxed recursion is *cross-layer sharing + low-rank per-layer residual*: the dense layer weights are written as $\Phi_\ell^0 \approx \Phi_k + B_\ell A_\ell$, a low-rank-plus-shared decomposition across layers rather than within a matrix. This is the same structural idea as basis sharing [98], and it can be read as the answer to a natural question: how much do layers of a trained transformer differ from one another? If the answer is "by a low-rank amount", sharing plus LoRA recovers most of the capacity.

Algorithm 6 Uptraining a pretrained model into a relaxed recursive transformer (schematic)

Require: Pretrained L-layer model with layer weights $\{\Phi^0_\ell\}$, number of unique blocks K ($L = KR$), LoRA rank r

for $k = 1, \dots, K$ **do**
  $\Phi_k \leftarrow \mathrm{Init}(\{\Phi^0_{k+jK}\}_{j=0}^{R-1})$ ▷ e.g. average of tied layers, or select a subset
  for $j = 0, \dots, R-1$ **do**
    $\Delta \leftarrow \Phi^0_{k+jK} - \Phi_k$; $(U, \Sigma, V) \leftarrow \mathrm{SVD}(\Delta)$; $B \leftarrow U_{:r}\Sigma^{1/2}_{:r}$, $A \leftarrow \Sigma^{1/2}_{:r}V^\top_{:r}$ ▷ LoRA init
  end for
end for
Uptrain on a modest token budget (distillation from the original model is natural here)

10.4 Looped transformers and recurrent depth

10.4.1 Theory: depth without parameters

Looped transformers apply a k-layer block L times. Theoretical results show that a looped transformer can simulate computation: with appropriate construction it can emulate programmable computers [152] and learn in-context learning algorithms with far fewer parameters than a non-looped transformer [153]. Saunshi et al. argue that many reasoning problems need depth more than parameters; on synthetic tasks (addition, p-hop induction, math problems) a k-layer transformer looped L times nearly matches a kL-layer non-looped model and clearly beats a plain k-layer one; in language modeling they report looped models competitive with or better than kL-layer models on downstream reasoning, a trade-off with memorization, and they prove that looped models can simulate T chain-of-thought steps with T loops, i.e. generate *latent thoughts* [151].

10.4.2 Recurrent depth and test-time scaling

Geiping et al. train a 3.5B-parameter model on 800B tokens whose recurrent block can be unrolled to arbitrary depth at inference time; they report that reasoning-benchmark scores improve, sometimes substantially, as test-time recurrence is increased, up to a compute load they equate to a 50B-parameter model [150]. Scaling is in latent space and requires no special data and little context, in contrast to chain-of-thought tokens.

10.4.3 Ouro

The Ouro family (1.4B and 2.6B parameters, trained on 7.7T tokens) is a pretrained looped language model; the authors report that these models match the results of dense LLMs up to 12B parameters on a range of benchmarks and attribute the advantage to better knowledge *manipulation* rather than greater knowledge *capacity* [149]. This aligns with the reasoning-versus-memorization trade-off of [151]: looping buys effective depth, not stored facts.

10.5 Mixture-of-Recursions

Mixture-of-Recursions (MoR) unifies parameter sharing with *adaptive* depth: a shared stack of layers is applied up to N_r times and lightweight routers decide per token how many recursions it receives, so easy tokens exit early while hard tokens are processed deeper [148]. Attention in each recursion is computed only among the tokens still active, and only their keys and values are stored.

Routing. Two routers are studied. *Expert-choice* routing treats each recursion depth as an expert that selects its top-k tokens (so the active set narrows with depth and compute is fixed in advance, but training needs an auxiliary mechanism to avoid causality leakage). *Token-choice* routing assigns each token a recursion depth up front via a top-1 router (no leakage, but load-balancing is needed).

Sharing strategy. The paper compares Cycle (layers reused cyclically), Sequence (each layer reused consecutively), and Middle-variants that keep unique first and last layers; *Middle-Cycle* performed best in ablations and is used throughout.

KV caching. *Recursion-wise caching* stores keys and values only for tokens routed to a given depth, reducing KV memory and IO to roughly $(N_r+1)/(2N_r)$ of vanilla; *recursive KV sharing* caches once at the first recursion and reuses it at later depths, reducing memory to about $1/N_r$ at the cost of weaker attention-FLOP savings and some quality loss.

Reported results (models of 135M to 1.7B parameters) [148]: non-embedding parameters of 167M, 118M and 98M for 2, 3 and 4 recursions versus 315M for the vanilla baseline; at equal training compute (16.5×10^{18} FLOPs) the 2-recursion expert-choice model reaches 43.1% average few-shot accuracy versus 42.3% for vanilla with nearly 50% fewer parameters; at equal data (20B tokens) it uses 25% fewer training FLOPs, cuts training time by 19% and reduces peak memory by 25%; token-choice at 3 recursions scores 40.0% versus 42.6% for expert-choice; and with continuous depth-wise batching MoR with 4 recursions reaches up to 2.06× throughput over vanilla at the largest batch, with a slight quality drop. These are results at small scale, from one group, and should not be extrapolated to frontier scale without independent replication.

Table 10.1: Reported small-scale results for Mixture-of-Recursions [148] (as stated in the source page; not independently verified).

Quantity	Vanilla	MoR (as reported)
Non-embedding params	315M	167M / 118M / 98M (2/3/4 recursions)
Avg. few-shot acc. at equal FLOPs	42.3%	43.1% (expert-choice, 2 recursions)
Training FLOPs at equal data (20B tok.)	100%	75% (2 recursions)
Peak memory	100%	−25%
Throughput (max batch, cont. depth-wise batching)	1×	up to 2.06× (4 recursions)

10.6 Mixture-of-experts versus dense versus recursive: a common map

Define for each architecture the pair (total parameters P, active FLOPs per token $\propto F$). Dense models lie on $F \approx 2P$. Mixture-of-experts (MoE) models route each token to a subset of E_a of E experts, with $P \approx P_{\text{shared}} + EP_{\text{exp}}$ and $F \approx 2(P_{\text{shared}} + E_a P_{\text{exp}})$, so $F \ll 2P$: *more parameters per FLOP* [156–158]. Recursion lies on the opposite side: $F \approx 2RP$, *more FLOPs per parameter*.

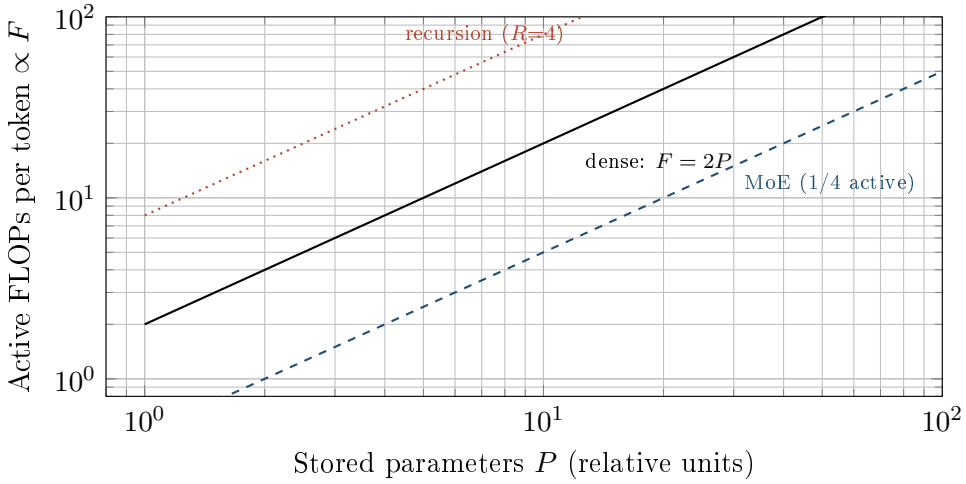


Figure 10.1: **Conceptual schematic** of the parameters/FLOPs plane. Slopes are idealized (MoE active fraction 1/4, recursion factor $R = 4$); real MoE has shared attention parameters and routing overhead, and recursion adds router and LoRA parameters. The vertical axis is relative; no measured data.

This framing clarifies what "compression" means in each case.

- **MoE** improves quality per FLOP but is *expensive in memory*: all experts must be resident (or offloaded), so it is the opposite of capacity compression. Unified scaling laws for routed models show that quality gains from more experts diminish with size [159].
- **Recursion** improves quality per parameter and per unit of memory but costs FLOPs; it is attractive when the deployment is memory-limited and compute-cheap, or when depth is what the task needs.
- **Distillation and factorization** move down the diagonal (smaller P and smaller F together).

A MoE with recursive experts, or a recursive model whose routers choose both depth and expert, spans both axes; Mixture-of-Depths, which routes tokens to skip layers to reduce FLOPs at fixed parameters, is the dual of recursion [155].

10.7 Assessment

- **Evidence for**: consistent small-scale results that recursion plus a little unshared capacity (LoRA, routing) recovers much of the quality of larger dense models; theoretical and synthetic

results that depth, not parameters, limits some reasoning tasks; Ouro and Huginn as larger proof points [147–151].

- **Evidence against or missing**: no independent large-scale replication at frontier size; hardware kernels and serving stacks (continuous depth-wise batching, heterogeneous exit depths) are not yet standard; sharing does not reduce streamed bytes or KV cache by default (Proposition 10.1); looped models might trade away knowledge capacity.
- **Where it fits**: on-device and edge deployment where capacity binds; reasoning-focused small models; research on adaptive compute. For a generic server deployment targeting lowest cost per token, a distilled dense or MoE model is currently the lower-risk choice.

Chapter 11

Model Families: Merging, Delta Compression, Small-Model Scaling and NAS

This chapter treats two views that sit outside single-model compression. First, a *family* of models derived from a common base can be stored and served at far less than the sum of its members: merging, task vectors, delta quantization and multi-tenant adapter serving. Second, the strongest alternative to compressing a large model is often *training a small one well*, so we review inference-aware scaling, over-training, small-model families and hardware-aware architecture search as the baseline that every method in this paper must beat.

11.1 Task vectors and merging

Let θ_0 be a pretrained model and θ_i a fine-tuned version for task i. The *task vector* is $\tau_i = \theta_i - \theta_0$ [161]. Operations in weight space then act on behaviours:

$$\theta = \theta_0 + \sum_i \lambda_i \tau_i, \tag{11.1}$$

where positive λ_i adds a skill and negative values subtract it. Model soups average several fine-tuned models from the same pretrained start, improving accuracy without increasing inference cost [160]; Fisher-weighted averaging weights each parameter by its estimated importance [164].

Why averaging can work. Fine-tuned models that start from a common pretrained point typically remain in a shared low-loss basin. For task i with loss $\mathcal{L}_i$, a second-order expansion around θ_i gives

$$\mathcal{L}_i\Big(\theta_0 + \sum_j \lambda_j \tau_j\Big) \approx \mathcal{L}_i(\theta_i) + \tfrac{1}{2}(\delta_i)^\top H_i\, \delta_i, \qquad \delta_i = (\lambda_i - 1)\tau_i + \sum_{j \neq i} \lambda_j \tau_j, \tag{11.2}$$

with H_i the Hessian of $\mathcal{L}_i$ at θ_i. The cross terms $\lambda_j\, \tau_i^\top H_i \tau_j$ $(j \neq i)$ measure *interference*: they vanish if the task vectors are orthogonal in the H_i metric, and are large when different tasks modify the same high-curvature directions.

Proposition 11.1 (Averaging two models, convex case)**.** *If $\mathcal{L}$ is convex then $\mathcal{L}(\frac{1}{2}(\theta_1 + \theta_2)) \leq \frac{1}{2}\mathcal{L}(\theta_1) + \frac{1}{2}\mathcal{L}(\theta_2)$. For a μ-strongly convex $\mathcal{L}$ the gap is at least $\frac{\mu}{8}\,\|\theta_1 - \theta_2\|^2$.*

Neural losses are not convex, so the proposition is only suggestive; its content is that the *distance* between the endpoints controls the potential gain and that the loss along the segment can have barriers if the models are not in the same basin.

Reducing interference. TIES-Merging trims small-magnitude entries of each task vector, resolves sign conflicts by electing a majority sign per parameter, and averages only agreeing entries [162]. DARE randomly drops a large fraction p of delta entries and rescales the survivors by $1/(1-p)$, preserving the expectation: for each coordinate, $\mathbb{E}[\tilde{\tau}_k] = (1-p)\cdot\tau_k/(1-p) = \tau_k$. The authors report that a large proportion of fine-tuning deltas can be removed without harming the fine-tuned model's abilities when deltas are small [163]. Surveys and toolkits are available [165, 166].

Merging as compression. If N expert models can be replaced by one merged model, the storage is reduced by $N\times$. However the merged model does not reproduce each expert exactly; merges tend to lose the sharper capabilities of individual specialists. It is a compression of the *family*, with quality loss that depends on interference.

11.2 Delta compression

Because τ_i is typically small relative to θ_0 and has low information content, a fine-tuned model can be stored as $(\theta_0, \hat{\tau}_i)$ with $\hat{\tau}_i$ compressed aggressively.

11.2.1 BitDelta: one bit per delta weight

BitDelta quantizes each delta matrix to its sign and a single scale: $\hat{\Delta} = \alpha\,\mathrm{sign}(\Delta)$ [167].

Proposition 11.2 (Optimal scale for 1-bit deltas)**.** *For $\Delta \in \mathbb{R}^N$ the scale minimizing $\|\Delta - \alpha\,\mathrm{sign}(\Delta)\|_2^2$ is $\alpha^\star = \frac{1}{N}\sum_k |\Delta_k|$, with residual $\|\Delta\|_2^2 - N\alpha^{\star 2}$.*

Proof. $\|\Delta - \alpha s\|^2 = \|\Delta\|^2 - 2\alpha\sum_k |\Delta_k| + N\alpha^2$ since $s_k\Delta_k = |\Delta_k|$ and $s_k^2 = 1$. Setting the derivative to zero gives $\alpha = \frac{1}{N}\sum_k |\Delta_k|$, and substituting yields the residual. □

BitDelta then *distills* the scale factors: with the 1-bit signs frozen it fine-tunes the per-matrix scales to match the logits of the original fine-tuned model on a small calibration set [167]. The abstract reports more than 10× GPU-memory reduction for multi-tenant serving (one high-precision base plus 1-bit deltas per tenant) and states that memory savings can translate to lower generation latency in multi-tenant settings.

Example 11.1 (Storage for 100 fine-tunes of a 7B model, illustrative)**.** Full copies in BF16: $100\times 14 = 1{,}400$ GB. BitDelta-style: base 14 GB $+ 100\times(7\times 10^9/8)$ B $= 100\times 0.875 = 87.5$ GB, a total of about 101.5 GB (scales are negligible), i.e. 13.8× smaller. LoRA rank 16 on all linear layers of a Llama-7B-shaped model: $16\cdot(4\cdot 8192 + 3\cdot 15104)\cdot 32 \approx 40\times 10^6$ parameters $= 80$ MB in BF16, so 100 adapters take 8 GB: $14 + 8 = 22$ GB total, 64× smaller than full copies. The comparison is nuanced: LoRA adapters are *trained as* low-rank and constrain the fine-tune; BitDelta compresses an arbitrary full fine-tune post hoc.

11.2.2 Mixed-precision and low-rank deltas

Delta-CoMe observes that the delta's singular values have a long-tail distribution and assigns higher precision to singular vectors with larger singular values and lower precision to others,

a training-free mixed-precision delta compression [168]. DeltaZip compresses full-model-tuned deltas and co-designs a serving system for many such variants [169]. Together with DARE's sparsification and BitDelta's 1-bit sign code, these illustrate the three levers: sparsify, low-rank approximate, and quantize the delta.

11.2.3 Multi-tenant adapter serving

For LoRA-style deltas the serving problem is batching requests for different adapters with a shared base: the base GEMM is shared and the low-rank terms $B_i A_i x$ are computed per request. Punica introduces a batched kernel (SGMV) for multi-adapter computation and reports about 12× higher throughput than state-of-the-art serving systems on a fixed cluster at about 2 ms added per-token latency [170]. S-LoRA adds unified paging of adapter weights and KV cache, and reports up to 4× throughput over HuggingFace PEFT and vLLM with naive LoRA support, while serving thousands of adapters [171]. These speed-ups are relative to specific baselines at the time of those papers.

Algorithm 7 Batched multi-adapter forward for one linear layer (conceptual)

Require: Base weight $\boldsymbol{W}$, adapters $\{(A_i, B_i)\}$ in a pool, batch of requests $\{(x_k, i_k)\}$ with adapter ids

$Y \leftarrow X\boldsymbol{W}^\top$ ▷ one large GEMM shared by all requests
for each group of requests with the same adapter i (or via gathered batched kernel) **do**
  $Z_i \leftarrow X_i A_i^\top$ ▷ $d \to r$
  $Y_i \leftarrow Y_i + s\, Z_i B_i^\top$ ▷ $r \to d$; scale $s = \alpha/r$
end for
return Y

Family compression is the most mature "compression" in production serving

Storing a base once and variants as small deltas turns an N-fold memory cost into $1 + N\varepsilon$ with $\varepsilon \ll 1$. It requires no change to the model's training for LoRA, and little for BitDelta-like codes. The risk is quality drift of lossy deltas on out-of-distribution inputs.

11.3 The baseline: train a small model well

11.3.1 Compute-optimal and inference-aware scaling

Chinchilla fitted the loss of dense transformers as $L(N, D) = E + A/N^\alpha + B/D^\beta$ for N parameters and D training tokens, concluding that for a fixed training budget parameters and tokens should be scaled in similar proportion (about 20 tokens per parameter in the headline summary) [172, 173]. This ignores inference. Sardana et al. add the cost of serving: for a given quality and expected inference demand, the optimum moves to smaller models trained on far more tokens, and in their experiments quality kept improving at tokens-per-parameter ratios as high as 10,000 [174]. Gadre et al. show over-trained models still follow reliable scaling relations for loss and downstream tasks [175].

Proposition 11.3 (Inference-aware optimum)**.** *Minimize total FLOPs* $C = 6ND + 2ND_{\text{inf}}$ *subject to* $L(N, D) = L^\star$ *with* $L(N, D) = E + AN^{-\alpha} + BD^{-\beta}$*. Along the constraint,* $D(N) =$

$\left(B/(L^\star - E - AN^{-\alpha})\right)^{1/\beta}$, *and the stationarity condition* $\mathrm{d}C/\mathrm{d}N = 0$ *becomes* $6D + 2D_{\text{inf}} + 6N\,D'(N) = 0$ *(with* $D' < 0$*). Larger* D_{inf} *shifts the optimum to smaller* N *and larger* D*.*

Example 11.2 (Worked numeric example, using published fitted constants)**.** Using the fitted constants reported by Hoffmann et al. ($E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$, $\beta = 0.28$) and a target loss $L^\star = 2.0$, Table 11.1 lists $D(N)$ and costs assuming a lifetime of $D_{\text{inf}} = 2\times10^{13}$ served tokens. The *extrapolation of a fitted law to these regimes is itself uncertain* (Sardana et al. note that laws fit at ordinary ratios mis-predict extreme ratios [174]), so the table illustrates the mechanism, not a recommendation. Without inference ($D_{\text{inf}} = 0$) the minimum-cost size is about 15B parameters; with 2×10^{13} inference tokens it moves to about 5–7B, and with 2×10^{14} to about 3B.

Table 11.1: Computed from $L(N, D) = 1.69 + 406.4N^{-0.34} + 410.7D^{-0.28}$ at $L^\star = 2.0$ (constants from [173]); FLOPs in units of 10^{23}. Illustrative extrapolation, not a measurement.

N (B)	D (T tokens)	tokens/param	Train $6ND$	Inference $2ND_{\text{inf}}$	Total
4.0	12.16	3040	2.92	1.60	4.52
5.4	5.63	1043	1.82	2.16	**3.98**
7.0	3.39	485	1.42	2.80	4.22
10.0	1.97	197	1.18	4.00	5.18
15.4	1.20	78	1.11	6.16	7.27
30.0	0.69	23	1.25	12.00	13.25
70.0	0.43	6	1.82	28.00	29.82

Practice. The Llama 3 8B model was trained on about 15T tokens, i.e. $15 \times 10^{12}/8 \times 10^9 \approx 1{,}900$ tokens per parameter, roughly two orders of magnitude beyond the 20 tokens-per-parameter headline [50]. TinyLlama trained a 1.1B model on about 3T tokens [178]; SmolLM2 trained 1.7B parameters on about 11T tokens [179]; Phi-3-mini (3.8B) used heavily filtered and synthetic data [180]; MobileLLM optimizes deep-and-thin sub-billion architectures with embedding sharing and layer sharing [154]. The Densing-law analysis defines capacity density as the ratio of a model's effective parameter size (the size of a reference model with equal downstream performance) to its actual size and reports that on common benchmarks it doubles approximately every three months across recent open-source base models [176]. If that trend holds, a fixed-footprint compressed older model quickly competes with newly trained smaller models; compression results are therefore perishable.

11.3.2 Distillation versus training from scratch

Distillation scaling laws study when a distilled student beats a student trained with supervised pretraining at equal compute: with a ready teacher or when many students are produced, distillation wins up to a compute level that grows with student size; if a single student is to be distilled and the teacher must also be trained, supervised pretraining is generally better [53]. The crossover is a function of the teacher's cost amortization, which is why distillation dominates in organizations that already pay for a frontier teacher (and use it for a family of students [48, 61]).

11.3.3 Pruning, NAS and hardware-aware search

Neural architecture search (NAS) fits the architecture to a hardware target. Once-for-all trains a supernet from which sub-networks are extracted without retraining [181]; HAT searches hardware-aware transformers with latency measured on the target device [182]; MnasNet includes measured mobile latency in the reward [183]; Primer searches transformer variants for language modeling and finds, for example, squared ReLU and depthwise convolutions in attention helpful [184]. For LLMs, Puzzle combines blockwise local distillation of alternative blocks with a mixed-integer search over per-layer choices under throughput and memory constraints; the abstract reports that the two derived models (about 51B and 49B parameters, from a Llama-70B-class parent) achieve a 2.17× inference throughput speedup, fit on a single H100, preserve 98.4% of the parent's benchmark accuracy, and use at most 45B training tokens versus 15T for the parent [60]. This is a template for "compression as heterogeneous architecture search under distillation".

Table 11.2: Selected data points on the alternative to compression: small models and pruning-plus-distillation. Values are as stated in the cited sources.

Model / method	Size and data	Note
Llama 3 8B [50]	8B; ≈15T tokens	≈1,900 tokens/param (derived)
TinyLlama [178]	1.1B; ≈3T tokens	open small model
SmolLM2 [179]	1.7B; ≈11T tokens	data-centric small model
Minitron [61]	pruned+distilled families	up to 40× fewer tokens per model; 1.8× lower family cost (abstract)
Puzzle [60]	≈50B from 70B parent	2.17× throughput; 98.4% accuracy; ≤45B tokens
Densing law [176]	capacity density	doubles about every 3 months (reported trend)

11.4 Putting the baseline to work

1. **A fair comparison fixes total cost**, not just footprint: include training compute for the small model (or distillation compute) and amortized inference.

2. **Footprint-matched comparison.** For a fixed memory budget, compare (a) a quantized large model, (b) a pruned-and-distilled model, (c) a recursive/shared model, and (d) a small model trained to saturation. Published comparisons rarely include all four.

3. **Perishability.** Because capacity density rises quickly [176], compressing an old model competes against a moving target; compress the model that is current when you start and re-evaluate against new small releases.

4. **Where compression wins.** When you cannot retrain (frozen, licensed or fine-tuned weights), when you need exact behaviour preservation (lossless), when you have many variants (delta compression), or when the large model's data is unavailable.

Chapter 12

Lossless Compression of Weights

Every method so far changes the model. Lossless compression does not: the decoded tensor is bit-identical to the original, so every output, benchmark score, safety behaviour and numerical quirk is preserved. This is a distinctive and valuable guarantee, and the price is a hard ceiling set by entropy and a decoding cost. This chapter derives where the redundancy in floating-point weights lives, how coders exploit it, what the systems problem of decoding on a GPU looks like, and how lossless coding combines with quantization. Several of the quantitative results cited come from 2024–2026 papers [200–205].

12.1 Where the redundancy lives

A BF16 weight has 1 sign bit, 8 exponent bits and 7 mantissa bits (Chapter 3). If the three fields were i.i.d. uniform there would be nothing to gain. Trained weights are not uniform: they are concentrated near zero with approximately bell-shaped (sometimes heavy-tailed) marginals, so the exponent, which encodes the *binade* $\lfloor \log_2 |w| \rfloor$, takes few values with substantial probability.

12.1.1 A derivation for bell-shaped weights

Let w have a density f symmetric about zero and continuous at 0. The exponent is $E = \lfloor \log_2 |w| \rfloor$ and

$$P(E = k) = 2 \int_{2^k}^{2^{k+1}} f(x)\, dx \approx 2 f(0)\, 2^k \qquad (\text{for } 2^{k+1} \ll \text{scale of } f). \tag{12.1}$$

Thus on the small-magnitude side the probabilities form a *geometric sequence with ratio* $1/2$: each binade down has half the mass of the one above. On the large-magnitude side the mass falls off according to the tail of f (super-geometrically for a Gaussian). The entropy $H(E) = -\sum_k P(E = k) \log_2 P(E = k)$ is therefore a few bits regardless of the scale σ, because scaling $w \mapsto cw$ merely shifts E by $\log_2 c$.

Proposition 12.1 (Scale invariance of exponent entropy)**.** *If $c = 2^j$, then $E(cw) = E(w) + j$ and $H(E)$ is unchanged. For a general c the distribution of E is a re-binning of a shifted continuous variable $\log_2 |w| + \log_2 c$; for smooth f its entropy varies only slightly with c.*

Example 12.1 (Computed for Gaussian weights)**.** For $w \sim \mathcal{N}(0, \sigma^2)$ computing $P(E = k)$ numerically for $\sigma = 0.02$ (a typical order for LLM weights) gives the distribution of Figure 12.1, with $H(E) = 2.545$ bits, 22 binades with probability above 10^{-6}, the best 7 consecutive binades covering 97.9% of weights and the 12 most probable covering 99.9%. The value of $H(E)$ is the same to three decimals for $\sigma \in \{0.005, 0.01, 0.02\}$ and changes by 0.003 bits at $\sigma = 0.05$ and 0.1. For a Laplace distribution of equal variance $H(E) = 2.83$ bits, and for a Student-t with

3 degrees of freedom 2.77 bits: heavier tails spread more mass over more binades. These are exact computations for *synthetic* distributions; real LLM weights depart from them.

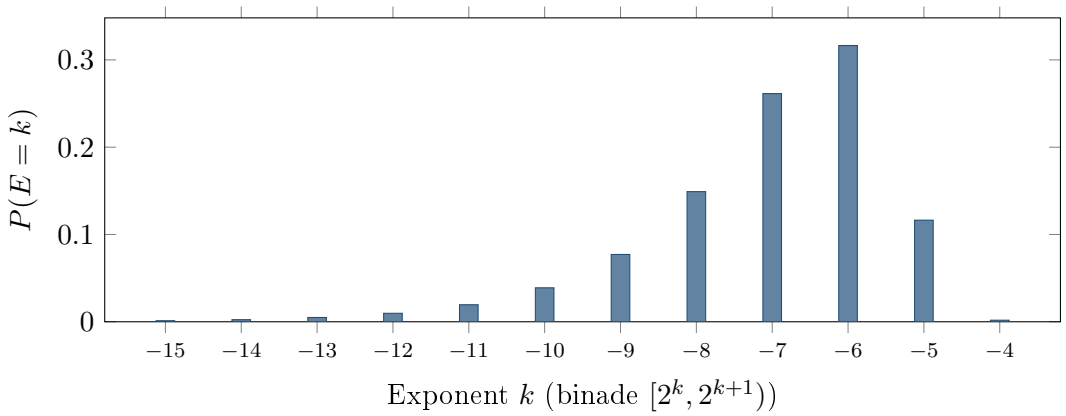


Figure 12.1: **Computed for a synthetic Gaussian** $\mathcal{N}(0, 0.02^2)$: probability of each BF16 exponent value. Mass halves per binade on the small side (geometric tail) and drops sharply above the peak; $H(E) = 2.545$ bits. Real LLM weights give similar shapes, with about 40 of 256 exponent values observed in practice according to [200, 201].

Empirical confirmation. DFloat11 reports an exponent entropy of about 2.6 bits (of 8 allotted) across the LLMs it studies, with only about 40 of the 256 exponent values used and sign and mantissa entropy close to their nominal widths [200]. ZipNN reports the same pattern: about 40 values used, the top 12 covering nearly 99.9% of parameters, and attributes the skew to weights remaining near $[-1, 1]$ with optimizer noise setting a floor [201]. The "exponent concentration" analysis argues that SGD noise has heavy-tailed increments whose accumulation approaches symmetric α-stable laws; under that model the exponent is two-sided geometric and its entropy is bounded between $\alpha/(1 + 2^{-\alpha})$ and $\alpha/(1 - 2^{-\alpha})$ bits (for $\alpha = 2$, between 1.6 and 2.67 bits), which brackets our Gaussian computation of 2.545 bits [203]. NeuZip likewise reports exponent entropy under 3 bits [202].

12.1.2 The ceiling

Proposition 12.2 (Entropy ceiling for BF16 exponent coding)**.** *If sign and mantissa are incompressible and the exponent has entropy* H_E *bits, no lossless scheme that treats weights as i.i.d. symbols can use fewer than* $8 + H_E$ *bits per weight on average, so the compressed fraction is at least* $(8 + H_E)/16$*. With* $H_E = 2.55$ *the bound is* 0.659*, i.e. at most 34% savings; with* $H_E = 2.83$*,* 0.677*.*

This matches reported results: ZipNN about 66% for BF16 models [201]; DFloat11 67.6%–69.5% of original size, with average widths of 10.81–11.12 bits (Llama 3.1 405B: 811.71 GB to 551.22 GB) [200]; ZipServ "up to 30%" reduction [204]; and a report of BF16 ratios up to 62% (the abstract does not say whether this is compressed-over-original; the figure may include structure beyond i.i.d. coding such as clean or converted models) [205].

Mantissa. The mantissa is not exactly uniform: within a binade the density of a Gaussian decreases, so smaller mantissas are slightly more likely. Our computation gives $H(\text{mantissa} \mid$

$E) = 6.918$ bits out of 7 for the Gaussian above, an extra 0.08 bits (0.5% of 16 bits). Worth noting but rarely worth the decode cost.

Non-i.i.d. structure. Conditioning on context (neighbouring weights in a row, the layer, the channel scale) could in principle lower the entropy further, since weights are correlated through shared scales. Published systems largely use i.i.d. or per-tensor models; the headroom from richer context models is, to the author's knowledge, not yet established for dense BF16 weights, and any context model must be decodable at GPU speed.

12.2 Entropy coders

General-purpose floating-point compressors such as fpzip (for scientific arrays) and Gorilla (for time series) exploit smoothness or temporal correlation between neighbouring values [206, 207]; that assumption does not hold for the nearly independent entries of a weight matrix, which is why the neural-network-specific coders below separate fields and model only the exponent.

Huffman coding assigns shorter codes to likelier exponents [196]. For our Gaussian exponent distribution the Huffman average length is 2.591 bits against $H = 2.545$, a redundancy of 0.046 bits, and 2.864 versus 2.832 for the Laplace case. The ceiling in Proposition 12.2 is thus nearly reached. ZipNN uses Huffman only, without Lempel–Ziv matching, because repeated byte strings in weights are short and random; it separates the exponent bits into their own stream (*byte grouping*) and compresses each stream separately [201]. DFloat11 compresses only exponents with Huffman and leaves sign and mantissa as raw 8-bit packed bytes [200].

Asymmetric numeral systems (ANS) reach the entropy bound more closely than Huffman at similar speed and are GPU-friendly in interleaved forms [198]; NeuZip uses ANS on exponents [202], and NVIDIA nvCOMP and DietGPU provide GPU ANS. The advantage over Huffman on this source is small (tens of millibits), because the alphabet has dozens of symbols and a relatively smooth distribution.

Fixed-length structured codes. ZipServ avoids variable-length decoding altogether with a Tensor-Core-Aware Triple Bitmap Encoding: the 7 most common *consecutive* exponents receive 3-bit codewords 001–111, codeword 000 marks a "fallback" weight stored at full precision, and each 8×8 tile stores three 64-bit bitmaps plus value buffers [204]. This gives a slightly worse rate than Huffman but is decodable in parallel.

Proposition 12.3 (Rate of the fixed 3-bit code)**.** *Let c be the probability mass of the 7 consecutive exponents given short codes. Per weight, a non-fallback weight costs $3 + 8 = 11$ bits (code plus sign and mantissa) and a fallback weight costs $3 + 16 = 19$ bits, so the average is $11 + 8(1 - c)$ bits and the compressed fraction is $(11 + 8(1 - c))/16$.*

For $c = 0.979$ (our Gaussian calculation) this gives 11.17 bits, a fraction of 0.698: about 30% savings, in line with the 30% reported [204]. Compared with the Huffman ideal of 0.662 for the same source, the fixed code gives up about 3.6 percentage points of ratio in exchange for decodability. This is the central trade-off of the chapter: *rate versus decode parallelism.*

Table 12.1: Lossless BF16/FP8 weight coders compared (design as described in the cited papers; ratios are compressed/original).

System	Coding	Reported ratio	Target use
ZipNN [201]	byte grouping + Huffman on exponent stream; CPU, multi-threaded	BF16 ≈66%; FP32 ≈83% (regular)	Storage, transfer; up to 80 GB/s multi-threaded decompression reported
NeuZip [202]	ANS on exponent; optional lossy mantissa truncation	ideal ratio stated as 1.501 (achieved 1.505); lossy mode: 7.70 GiB at 3 mantissa bits	Training and inference memory
DFloat11 [200]	Huffman on exponent; hierarchical LUT decode kernel	67.6–69.5%	GPU inference; block-level decompress
ECF8 [203]	Huffman on FP8 exponents; fused-decode CUDA	FP8: 9.8–14.8% savings (LLMs); 26.9% for a video model	FP8 GPU inference
ZipServ [204]	TCA-TBE fixed 3-bit exponent code; fused ZipGEMM	up to 30% smaller	Decode-stage fused GEMM

12.3 Decoding on a GPU: the systems problem

12.3.1 Why variable-length codes are awkward

Variable-length codes must be decoded sequentially: symbol boundaries are unknown until the previous symbols are decoded, lookups are data dependent, and warps diverge. ZipServ measures that on an L40S the ANS-based DietGPU and Huffman-based DFloat11 reach only 43.7% and 76.5% of peak memory bandwidth in decompression, and that decoupled decompression alone takes 1.56–3.44× the time of the core inference computation in some regimes [204]. DFloat11's kernel uses hierarchical lookup tables (the Huffman tree is split into subtrees of height 8, so each LUT has 256 entries that fit in shared memory), a "gap" array storing bit offsets per thread, and a two-phase decode with a prefix sum so that threads know their output positions [200].

12.3.2 Traffic analysis

Let W be the BF16 size of a weight tensor and $\rho < 1$ its compression fraction.

Proposition 12.4 (Memory traffic of decode strategies)**.** *At batch size one where each weight is used once per token:*

1. Baseline*: reads W from HBM.*

2. Decoupled decompress-then-GEMM*: reads ρW, writes W (decompressed), reads W again,*

for a total of $(2+\rho)W$*, strictly worse than the baseline unless the decompressed buffer is amortized over multiple uses.*

3. Block-level decompression amortized over B uses*:* $\rho W + W + BW$ *for* B *GEMM reads versus* BW *for the baseline, an overhead of* $(1+\rho)W$ *spread across the* B *uses.*

4. Fused decode-in-registers*: reads* ρW *only; the bandwidth-bound speedup is at most* $1/\rho$*.*

Thus, in the memory-bound regime (decode at small batch), decoupled decompression *cannot* help if the model already fits; it helps only (a) when the alternative is something worse (offloading weights over PCIe, or not fitting at all), or (b) when decompressed weights are reused across many tokens (large batch, prefill), or (c) with a fused kernel. DFloat11's headline throughput gains of 2.3–46.2× are against *CPU offloading* of an uncompressed model, and its 5.7–14.9× longer generation lengths are under a fixed GPU memory budget; both reflect scenario (a) [200]. In its design the compressed matrices of a transformer block are decompressed together just before the block's forward pass and discarded, so the overhead is constant per forward and is amortized by larger token batches [200]. ZipServ's fused ZipGEMM decodes to registers that feed Tensor-Core *mma* instructions, uses the fused path in the memory-bound decode stage and a decoupled path in the compute-bound prefill stage, and reports up to 2.21× kernel speedup over cuBLAS and an average 1.22× end-to-end speedup over vLLM on consumer and workstation GPUs (RTX 4090, L40S, RTX 5090) [204].

Reading the 2.21× against the $1/\rho$ bound

For $\rho \approx 0.70$ the bandwidth-bound ceiling on a fused kernel is $1/0.70 \approx 1.43\times$. A kernel-level speedup of 2.21× over cuBLAS therefore indicates that the cuBLAS baseline on those (skinny) shapes is itself well below the bandwidth roofline, not that decoding beats physics. The 1.22× end-to-end average, below the ceiling, is the more representative number. (Author's interpretation of the cited figures.)

12.3.3 When does lossless coding pay off?

Define the *regimes*:

12.4 Training-time and checkpoint compression

Training state is larger than weights: optimizer moments, gradients, and periodic checkpoints. ZipNN reports that during fine-tuning of a BF16 RoBERTa the model compresses to about 66%, optimizer state to about 54% and gradients to about 47%, and that compressing the *difference* between successive checkpoints or similar models gives much higher ratios, because few bits change per step [201]. This is delta coding (Chapter 11) in its lossless form. Earlier work compressed checkpoints for recommendation-model training [208] and used joint weight-momentum shrinking for lossy checkpoint compression [209]. NeuZip applies ANS to exponent bits for *lossless training*: for Llama-3 8B it reports peak training memory falling from 30.97 GiB (vanilla) to 15.25 GiB with identical loss, at a throughput cost versus the memory-efficient baseline (0.78 to 0.45 iterations per second in its table), and offers a lossy inference mode that truncates mantissa bits, reaching 7.70 GiB at perplexity 9.93 against 9.89 for the 15.08 GiB

Table 12.2: Regimes for lossless weight coding (author's synthesis).

Regime	Benefit	Condition / caveat
Storage / hub / transfer	≈1.4–1.5× fewer bytes moved	Decode at load time only; cheap; ZipNN-type CPU coders reach tens of GB/s [201]
Model fits but only barely	enables larger KV cache or batch	Needs GPU decoder; overhead per forward
Model does not fit (offload alternative)	large (reported 2.3–46× vs CPU offload)	Scenario-dependent [200]
Bandwidth-bound decode, model fits	up to $1/\rho$ if fused	Needs fused kernel for the exact hardware [204]
Compute-bound (large batch / prefill)	none or negative	Decode adds work; fused path not helpful
Training memory	lossless training memory reduction	NeuZip: overhead in throughput [202]

original with three retained mantissa bits [202]. The lossy mode is a quantization method in all but name and is compared in that paper against INT8, FP4 and NF4 baselines.

12.5 Lower-precision formats: less to gain

The headroom shrinks with format width because the exponent field is smaller. For FP8 E4M3 or E5M2, computing the exponent entropy of a Gaussian under per-tensor absmax scaling yields about 2.5 bits, i.e. an ideal compressed fraction of 0.82 (E4M3, 4 exponent bits) and 0.69 (E5M2); for FP16 (5 exponent bits) the ideal fraction is 0.85. These are synthetic computations with idealized i.i.d. Gaussian weights; actual FP8 checkpoints use per-channel or block scales and show 9.8–14.8% savings for LLMs with ECF8, and up to 26.9% on a video-generation model [203]; another report gives FP8 ratios up to 83% [205]. ECF8's analysis argues that a theoretical limit for lossless exponent coding corresponds to about 4.67 bits per weight (1 sign + about 2.67 exponent bits + 1 mantissa bit, per their bound), but treats FP4.67 or FP5 as impractical on current hardware [203].

FP4 and block formats. For 4-bit formats with block scales the remaining redundancy is mostly in the scale metadata and in non-uniform usage of the 16 code values; whether entropy coding is worthwhile depends on how equiprobable the quantizer's bins are.

12.6 Entropy-coded quantized weights

Quantization produces integer indices whose distribution is non-uniform; entropy coding recovers that slack. Deep Compression combined magnitude pruning, trained quantization and Huffman coding to reduce AlexNet and VGG by 35× and 49× respectively [185]. For a uniform scalar quantizer with step Δ on a continuous source at high resolution, the entropy of the indices is $H(Q) \approx h(X) - \log_2 \Delta$.

Theorem 12.1 (Entropy-coded scalar quantization gap [199, 210]). *At high resolution, a uniform scalar quantizer followed by ideal entropy coding is within* $\frac{1}{2}\log_2(\pi e/6) \approx 0.254$ *bits per sample of the rate–distortion function for squared error, for any smooth source.*

Example 12.2 (Computed: how much can entropy coding save on INT-b weights?). For $X \sim \mathcal{N}(0,1)$ and a uniform quantizer with 16 levels (nominal 4 bits) and step 0.4 (range about $\pm 3.2\sigma$), the index entropy is 3.37 bits, 15.7% below 4 bits. With step 0.5 it is 3.06 bits. For 3-bit uniform with step 0.8 the entropy is 2.40 bits (20% below 3). For 2-bit with step 1.3 it is 1.71 bits (15% below 2). For comparison, the Gaussian rate–distortion function at the distortion $\Delta^2/12 = 0.0133$ achieved by step 0.4 is $0.5\log_2(1/0.0133) \approx 3.12$ bits, so the gap is about 0.26 bits, consistent with Theorem 12.1. These are synthetic i.i.d. Gaussian computations.

Two caveats limit the practical value. First, modern weight-only quantizers use *non-uniform grids designed to be roughly equiprobable* (for example NormalFloat-style levels, whose bins carry equal Gaussian mass) or are optimized for task loss rather than MSE; for an equiprobable codebook the index entropy is already the nominal bit width, and entropy coding gains nothing. Second, group-wise scales and outlier handling change the statistics; real gains must be measured per quantizer. Hence "entropy-coded quantized weights" is attractive for *uniform-grid* schemes at low bit widths and on storage-bound paths, and less so for equiprobable codebooks.

12.7 Lossless versus lossy: a decision frame

1. If exactness is required (audit, reproducibility, regression guarantee) the only option here is lossless; expect ≈30–34% savings on BF16 and ≈10–18% on FP8.
2. If a 4-bit quantizer meets quality, it gives 4× over BF16 at some quality cost, a much larger reduction than any lossless scheme; lossless coding of the *quantized* model adds a further ≤15–20% for uniform-grid indices only.
3. Lossless and lossy compose: store a quantized model entropy-coded for distribution and decode to the runtime format at load time (no runtime overhead) or keep it coded and decode on the fly.
4. The most practical near-term wins are in *distribution and storage* (load-time decode, CPU-side, little risk) and in memory-constrained serving with fused kernels on specific hardware.

Entropy-decoding overhead and format lock-in

A coder is only as good as its kernel on *your* GPU: published speedups come from specific hardware (consumer and workstation GPUs in [204]; H100 clusters in [203]) and specific serving stacks. Decode overhead can erase gains in compute-bound regimes, and variable-length formats complicate tensor-parallel sharding and weight updates. Treat reported end-to-end speedups as upper-bound hints until reproduced on target hardware.

Chapter 13

Language Models as Compressors, MDL and Generalization Bounds

The previous chapter compressed the *weights* of a model. This chapter turns the lens around: a language model is itself a compressor of *data*, and the theory of compression (Kolmogorov complexity, minimum description length, PAC-Bayes) gives both a conceptual account of why learning works and, increasingly, quantitative generalization bounds for LLMs. It also supplies the language in which the trade-off between model size and quality is a rate–distortion frontier.

13.1 Prediction is compression

Given a probabilistic model q over sequences, arithmetic coding turns it into a lossless code: a sequence $x_{1:T}$ can be encoded in at most $-\log_2 q(x_{1:T}) + 2$ bits, by narrowing an interval of $[0, 1)$ according to the conditional probabilities $q(x_t \mid x_{<t})$ [195, 197]. For an autoregressive LM the conditionals are exactly the next-token distributions, so the code length is the model's total negative log-likelihood. Conversely any lossless compressor induces a distribution via $q(x) = 2^{-\ell(x)}$ (Kraft's inequality).

Algorithm 8 Lossless text compression with a language model and arithmetic coding

Require: Model q with deterministic next-token probabilities, text $x_{1:T}$ (token IDs), precision P bits

low $\leftarrow 0$, range $\leftarrow 2^P$
for $t = 1, \ldots, T$ **do**
    $p \leftarrow q(\cdot \mid x_{<t})$ quantized to integer frequencies f_v summing to $2^{P'}$ (every $f_v \geq 1$)
    low $\leftarrow$ low + range $\cdot$ cdf$(x_t - 1)/2^{P'}$; range $\leftarrow$ range $\cdot f_{x_t}/2^{P'}$
    Renormalize and emit leading bits when the interval is determined
end for
Flush enough bits to identify a point in the final interval

Decoding runs the same model on the *decoded prefix* and inverts the interval narrowing.

Practical constraints. Encoder and decoder must compute *bit-identical* probabilities; floating-point non-determinism across hardware, kernels or batch shapes breaks decoding, so deterministic inference (or integer arithmetic) is a requirement. Decoding costs one forward pass per token, so LM compression is slow. The model size must either be amortized (shared by sender and receiver) or counted in the compressed size.

13.2 Empirical results on LMs as compressors

- **Language modeling is compression.** Delétang et al. show that large LMs are strong general-purpose compressors: Chinchilla 70B, trained mostly on text, compresses ImageNet patches to 43.4% of raw size and LibriSpeech samples to 16.4%, beating PNG (58.5%) and FLAC (30.3%) [211]. They also discuss the *adjusted* compression rate that charges the model's own size to the compressed output, which is essential when comparing against compressors with no large side information.

- **LLMZip** uses LLaMA-7B as next-token predictor with a lossless coder, reports a new asymptotic upper bound on the entropy of English below earlier estimates, and in limited preliminary experiments beats BSC, ZPAQ and paq8h [212].

- **FineZip** fine-tunes the LM online and uses batched prediction to make compression practical: about 54× faster than LLMZip (about 4 hours versus about 9.5 days for 10 MB of text) with comparable compression, and about 50% better ratio than conventional compressors [213].

- **Compression as a proxy for capability.** Across 31 public LLMs and 12 benchmarks, average benchmark score correlates almost linearly with the ability to compress external text corpora [214].

A cruder illustration of the same principle is classification with an off-the-shelf compressor and nearest neighbours, reported to be competitive with some neural baselines on text classification [215] (some evaluation details of that work were later debated, so it is cited here only as an illustration of the compression–similarity link). On the theory side, in-context learning has been argued to be a form of prequential coding whose objective jointly favours simple and well-fitting explanations [232].

These results are the empirical basis for the slogan "prediction equals compression". They also warn: a model that compresses a corpus well may have seen the corpus, so compression of public text is contamination-sensitive, and the advertised ratio excludes the model's own weight cost unless adjusted.

13.3 Kolmogorov complexity, MDL and the prequential view

Kolmogorov complexity and Solomonoff induction. The Kolmogorov complexity $K(x)$ is the length of the shortest program printing x on a universal machine; it is uncomputable but defined up to an additive constant [221]. Solomonoff's universal prior $M(x) = \sum_{p:U(p)=x*} 2^{-|p|}$ yields predictions whose cumulative loss is bounded by $K(\mu)\ln 2$ for any computable data-generating μ [220, 222]. This is the ideal that practical LMs approximate: a mixture over simple explanations of data.

Minimum description length. MDL chooses the model minimizing the *two-part* code length $L(M) + L(\mathcal{D} \mid M)$ [216, 217]: bits to describe the model plus bits to describe the data given the model. For neural networks, Hinton and van Camp proposed minimizing the description length of the weights, with noisy weights and bits-back coding [218].

Prequential (online) code length. Blier and Ollivier note that deep learning models cannot be efficiently two-part coded, because the weights are expensive to specify at the precision needed, but *online coding* is natural: encode each datum x_t using the model trained on $x_{<t}$, giving

$$L_{\text{preq}}(\mathcal{D}) = \sum_{t=1}^{T} -\log_2 p_{\theta_{t-1}}(x_t), \tag{13.1}$$

which is simply the area under the training-loss curve in bits, and which does *not* require transmitting weights [219]. They argue that the prequential code length of standard networks is far shorter than naive two-part codes, which supports the view that deep learning compresses data well despite huge parameter counts.

Example 13.1 (Weights are small relative to the data code, illustrative)**.** Suppose a 7B-parameter model with BF16 weights ($\approx 1.1 \times 10^{11}$ bits) is trained for one epoch on $D = 3 \times 10^{12}$ tokens, with an average online loss of 1.5 bits/token (an assumed round number). The prequential code length is 4.5×10^{12} bits; the final weights are 2.5% of that. In the compression view the learner has "spent" most of its description on data it has already predicted, and in the regime $D \gg N$ the final weights are a small side-channel. This framing explains why an LLM can trade parameters for tokens (Chinchilla trade-offs) and why distillation, which transmits the *function* rather than the data, is cheap relative to training.

Compression of the weights in the MDL picture. Everything in Chapters 8, 9 and 12 reduces $L(M)$: low-rank, structured and quantized parameterizations and entropy coding of the weights make the model part shorter. MDL says this is worthwhile only if $L(\mathcal{D} \mid M)$ does not grow by more. That is a precise statement of the rate–distortion trade-off, with distortion measured in extra bits of data code length (equivalently, increased loss).

13.4 Generalization bounds via compression

13.4.1 Occam and PAC-Bayes

An Occam bound states that for a prefix code on hypotheses and losses in $[0, 1]$, with probability $\geq 1-\delta$, $L(h) \leq \hat{L}(h) + \sqrt{(\ell(h) \ln 2 + \ln(1/\delta))/(2n)}$ (Chapter 3). The PAC-Bayes generalization [228]: for prior P chosen before seeing data and any posterior Q,

$$\mathbb{E}_{h\sim Q} L(h) \leq \mathbb{E}_{h\sim Q} \hat{L}(h) + \sqrt{\frac{\mathrm{KL}(Q \,\|\, P) + \ln \frac{2\sqrt{n}}{\delta}}{2n}}. \tag{13.2}$$

The key: the bound depends on $\mathrm{KL}(Q \,\|\, P)$ in nats, i.e. the number of bits needed to specify Q given P, not on the raw parameter count. A compressible solution has a small KL.

Example 13.2 (How many bits can we afford?)**.** With $n = 10^9$ i.i.d. samples, a loss bounded in $[0, 1]$, and $\mathrm{KL}(Q \,\|\, P) = 10^6$ nats, Equation (13.2) adds $\sqrt{(10^6 + \ln(2\sqrt{10^9}/0.05))/(2 \cdot 10^9)} \approx \sqrt{5 \times 10^{-4}} \approx 0.022$ to the empirical risk. At 10^9 nats the term is ≈ 0.71, vacuous for a $[0, 1]$ loss. So non-vacuous bounds need specifications of order 10^6 nats (≈ 180 KB) for 10^9 samples, extremely small compared with a model of billions of parameters, which is why extreme compression is the path to nontrivial bounds. (Arithmetic example, not an empirical result.)

13.4.2 From small networks to LLMs

Dziugaite and Roy first obtained non-vacuous PAC-Bayes bounds for stochastic neural networks with many more parameters than training points by optimizing the bound directly [223]. Arora et al. derived bounds via an explicit compression of the trained network [224]; Zhou et al. extended non-vacuous bounds to ImageNet scale using a PAC-Bayesian compression approach [225]; Lotfi et al. made them much tighter, tight enough in some settings to explain generalization, by combining PAC-Bayes with quantization and intrinsic-dimension-style subspace training [226]. For *language models*, Lotfi et al. construct the first non-vacuous generalization bounds for pretrained LLMs with nearly a billion parameters, using SubLoRA, a low-dimensional non-linear parameterization combining LoRA with subspace training; handling the unbounded log-likelihood via prediction smoothing; a subsampling extension to compute the bound on massive datasets; and the finding that *larger models get better bounds and are more compressible than smaller ones* [227].

Why this matters for compression practice

The bound-based finding that larger models are *more compressible* meshes with practice: large models tolerate quantization, pruning and low-rank truncation better than small ones, a recurrent empirical observation in compression studies. It also suggests that the reason compression works may be intrinsically linked to why the model generalizes: the function it computes has a short description.

13.4.3 Intrinsic dimension

Li et al. define the *intrinsic dimension* of an objective landscape as the smallest subspace dimension d such that training only $\theta = \theta_0 + P\,z$ ($z \in \mathbb{R}^d$, P a fixed random projection) reaches a target performance [118]. Aghajanyan et al. find that pretrained language models have strikingly low intrinsic dimension for fine-tuning; for RoBERTa on MRPC, 200 trainable parameters randomly projected into the full space reach about 90% of full fine-tuning performance, and larger or longer-pretrained models tend to have lower intrinsic dimension [117]. This motivates LoRA (Chapter 8) and underlies the SubLoRA bound parameterization. *Caution*: low intrinsic dimension of *fine-tuning* does not imply that the pretrained weights themselves are low-rank.

13.5 Information bottleneck

The information bottleneck (IB) principle seeks a representation T of input X that minimizes $I(X;T) - \beta I(T;Y)$, i.e. is maximally compressed while keeping information about the label [229]. Shwartz-Ziv and Tishby reported two phases of training (fitting and compression) in the information plane of small networks [230]; Saxe et al. argued that the compression phase depends on activation function and is not general, and that estimates of mutual information in deterministic networks are problematic [231]. The author regards IB as a useful *design heuristic* (representations that discard nuisance information transfer well; a distillation student has less capacity to retain nuisance details) rather than an established explanation of LLM behaviour.

Relation to distillation. The student's task is to reproduce a teacher's conditional distribution; the *information about Y that the student must carry* is $I(T;Y)$ measured against the teacher. Soft targets carry more bits per example than hard labels (Proposition 7.1), so for a fixed student capacity they supply a richer signal for allocating that capacity. This is an informal, non-rigorous link.

13.6 Scaling, precision and Pareto frontiers

Compression methods trade *memory* against *loss*. The efficient frontier is the lower envelope of loss as a function of resident bits across all (model size, precision, method) combinations. Two published facts shape it. First, precision-aware scaling laws: training in low precision reduces a model's effective parameter count, and post-training quantization degrades models more as they are trained on more data, to the point that extra pretraining can be harmful [177] (fitted on over 465 runs with predictions checked up to 1.7B parameters and 26B tokens, so extrapolation to frontier scale is not established). Second, over-training makes small models stronger but, by the same law, harder to quantize. The interplay means the best compressed model for a fixed memory is not simply "the biggest model at the lowest bits".

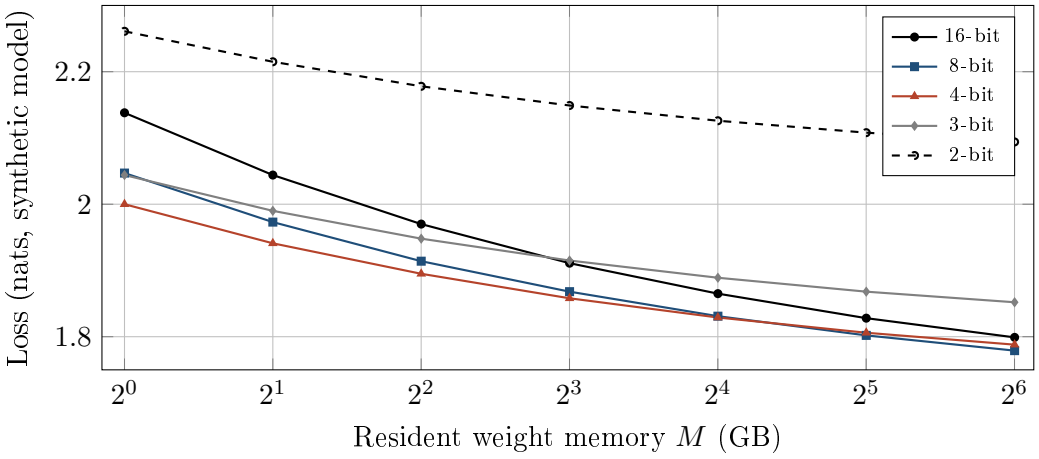


Figure 13.1: **Fully synthetic illustration of a memory–loss Pareto frontier.** Loss is $1.69 + 406.4\,N^{-0.34} + \delta(b)$ with $N = 8M/b$ parameters and an *invented* per-precision penalty $\delta(16, 8, 4, 3, 2) = (0, 0.003, 0.03, 0.10, 0.35)$ nats; the N-dependence uses the Chinchilla-form fit of [173] with data term dropped. The shape (4-bit best at small M, 8-bit overtaking at large M, 2-bit dominated) is the point; the penalties are not measurements.

Pareto methodology. A sound frontier comparison (i) fixes the evaluation suite, (ii) reports *actual resident bytes* including scales, indices and metadata, (iii) includes the cost of producing the model, and (iv) compares across families (quantized large, distilled small, factorized, recursive). Chapter 14 discusses evaluation hygiene.

13.7 Summary

1. Language models are compressors; the code length of data under a model is its cross-entropy loss, and large LMs compress even non-text data well [211].
2. In the MDL picture, weight compression shortens the model part, and the criterion for acceptable compression is a bounded increase in data code length.
3. Compression-based bounds have reached LLM scale at the price of extreme parameterizations (SubLoRA) and show that bigger models are more compressible [227].
4. Theory supports, but does not replace, empirical frontier mapping.

Chapter 14

Systems Considerations and Evaluation Methodology

A compressed model is only useful if it is both *good enough* and *faster or smaller where it counts*. This chapter first analyses when the compression families of this paper turn into wall-clock gains (roofline reasoning, factorized-layer kernels, format overhead, KV cache) and then lays out evaluation practice that avoids the most common errors.

14.1 Roofline reasoning for LLM inference

For a kernel that performs F FLOPs and moves Q bytes, the attainable time is $\max(F/\Pi,\, Q/\beta)$, where Π is peak compute and β is memory bandwidth [235]. The *ridge point* Π/β (FLOP per byte) separates memory-bound from compute-bound regimes. For a linear layer applied to B tokens with weights stored at b_w bytes per element, $F = 2Bmn$ and $Q \approx b_w mn$ (weights dominate), so the arithmetic intensity is $2B/b_w$ FLOP/byte.

Proposition 14.1 (Crossover batch size)**.** *The layer is memory-bound when* $B < B^\star = \dfrac{b_w}{2}\dfrac{\Pi}{\beta}$. *Reducing* b_w *(quantization, lossless coding of the streamed bytes)* lowers $B^\star$*: a compressed model becomes compute-bound at smaller batch.*

Example 14.1 (Illustrative numbers)**.** For an accelerator with $\Pi \approx 10^{15}$ FLOP/s dense BF16 and $\beta \approx 3\times10^{12}$ B/s (rounded values typical of a recent high-end part), $\Pi/\beta \approx 330$ FLOP/byte. For BF16 weights ($b_w = 2$): $B^\star \approx 330$ tokens. For 4-bit weights ($b_w = 0.5$): $B^\star \approx 80$. Single-stream decode ($B = 1$) is deep in the memory-bound regime for any format, so bytes streamed set the latency; large-batch serving and prefill are compute-bound and gain nothing from fewer bytes. (Approximate hardware figures; specific products differ.)

This single idea sorts the families:

- *Memory-bound decode* benefits from anything that reduces streamed bytes *or* reads (quantization, low-rank with small r, lossless coding with fused decode, smaller student).

- *Compute-bound prefill and large-batch serving* benefits only from FLOP reductions: smaller students, low-rank/structured layers with high-efficiency kernels, MoE sparsity; sharing and lossless coding give nothing.

14.2 When does low-rank win wall-clock?

14.2.1 The model

Replace $y = \boldsymbol{W}x$ by $y = A(Bx)$. Let $\rho = r(m+n)/(mn)$ be the parameter fraction. The memory-bound time of the dense layer is $T_d = t_m + t_\ell$ with $t_m = b_w mn/\beta$ and t_ℓ a fixed launch/synchronization overhead; the factorized layer needs two dependent kernels and moves ρ of the bytes:

$$T_{lr} = \rho\, t_m + 2t_\ell + \epsilon, \qquad S = \frac{T_d}{T_{lr}} = \frac{t_m + t_\ell}{\rho\, t_m + 2t_\ell + \epsilon}, \tag{14.1}$$

where ϵ covers the extra read and write of the $B \times r$ intermediate (negligible at small B).

Proposition 14.2 (Break-even parameter fraction)**.** *Ignoring ϵ, $S > 1$ iff $\rho < 1 - t_\ell/t_m$. With $t_\ell \to 0$ the condition is $\rho < 1$ (the parameter-count break-even of Proposition 8.1); a nonzero overhead moves the break-even to a* smaller *ρ, and $S \to (1 + t_\ell/t_m)/(2t_\ell/t_m)$ as $\rho \to 0$, bounded by the launch overhead.*

Example 14.2 (A 4096×4096 BF16 layer, assumed parameters)**.** At $\beta = 3$ TB/s, $t_m = 2 \cdot 4096^2/3 \times 10^{12}\,\mathrm{s} \approx 11.2\,\mu\mathrm{s}$. With a $5\,\mu$s per-kernel overhead (an assumed figure typical of unfused eager execution; CUDA graphs and fusion reduce it), the break-even is $\rho < 1 - 5/11.2 = 0.55$: the factorized layer must keep *less than 55%* of the parameters to be faster at all, and at $\rho = 0.5$ the speedup is only $S = 1.04$; at $\rho = 0.3$, $S = 1.21$. With a $2\,\mu$s overhead the break-even is $\rho < 0.82$ and $S(0.5) = 1.38$. In the ideal $t_\ell = 0$ limit $S = 1/\rho$. Because Example 8.1 showed a 40% parameter cut requires dropping 56–70% of the rank, many published low-rank configurations (10–30% compression) sit in the zone where the speedup is marginal or negative at batch size one. (Computed from the formula with assumed overheads; not a measurement.)

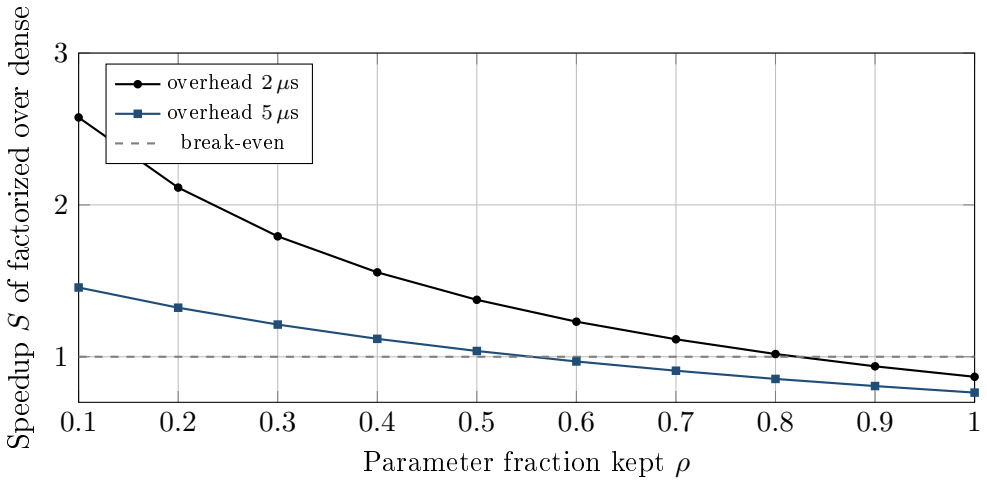


Figure 14.1: **Computed from the model of Proposition 14.2** for a 4096×4096 BF16 layer at 3 TB/s, with *assumed* per-kernel overheads. The ideal ($t_\ell = 0$) curve is $1/\rho$ and is off scale at small ρ. Illustrative only.

Dense baselines are themselves highly optimized: IO-aware fused attention [233], mixed-precision GEMM kernels for low-bit weights that approach the bandwidth bound at small batch [234], and well-understood partitioning strategies for large-model inference [237]. A

compression method must be compared against these tuned baselines, not against naive eager execution.

14.2.2 Prefill and compute-bound regimes

When compute-bound, ideal FLOP reduction is $1/\rho$ but thin GEMMs (inner dimension r small) are less efficient on tensor cores: tile sizes (typically multiples of 16–128) mean ranks should be padded to multiples of 8 or 64 (and a rank like 1229 wastes work); the two-GEMM form has a skinny intermediate. Fusing $A(Bx)$ into one kernel keeps the $B \times r$ intermediate on chip. The practical rule: choose ranks that are multiples of the tile size, evaluate on the *actual serving batch distribution*, and measure.

14.2.3 Other considerations

- **Tensor parallelism.** Column- and row-parallel sharding of $y = A(Bx)$ needs B and A sharded along compatible axes; the rank dimension is a natural shard axis but the intermediate must be all-reduced or all-gathered, adding communication that dense sharding avoids.

- **Batched small matmuls.** Monarch and block-diagonal formats map to batched GEMM, which is efficient when blocks are large enough (Chapter 9).

- **Absorbed projections.** In attention, up-projections of KV latents can be absorbed into the query path, as in MLA [104], so that low-rank structure reduces both memory and FLOPs without reconstruction; plain post-hoc factorization of W_K, W_V must reconstruct unless kernels are co-designed [103].

14.3 Memory-format overhead

A nominal bit width ignores metadata. Table 14.1 lists typical overheads, computed from the layout.

14.4 KV cache and long contexts

Weight compression is irrelevant when the KV cache dominates. The cache holds $2\,L\,n_{kv}\,d_h\,b$ bytes per token (keys and values, L layers, n_{kv} KV heads, head size d_h, b bytes per element).

Example 14.3 (KV bytes per token)**.** For $L = 32$, $n_{kv} = 8$ (grouped-query attention), $d_h = 128$, BF16: $2 \cdot 32 \cdot 8 \cdot 128 \cdot 2 = 131{,}072$ B $= 128$ KiB per token. A 128,000-token context needs ≈ 16 GiB per sequence, comparable to the weights of an 8B model in BF16 (16 GB). Serving 8 such sequences at once costs 128 GiB of cache. Lossless weight compression by 30% saves ≈ 5 GB, which is only about 0.3 of one such sequence's cache. Techniques that act on the cache (latent KV compression [94, 103, 104], KV sharing in recursive models [148], paged allocation [236]) matter more for long contexts. (Arithmetic example.)

Table 14.1: Effective bits per weight including metadata (computed from layout; typical implementations).

Format	Bits/weight	Computation
BF16	16	baseline
INT4, group 128, FP16 scale	4.125	$4 + 16/128$
INT4, group 128, FP16 scale + zero	4.25	$4 + 32/128$
INT4, group 32, FP16 scale	4.5	$4 + 16/32$
INT2, group 64, FP16 scale	2.25	$2 + 16/64$
2:4 sparse FP16 (2 bits metadata per kept value)	9	$(16 + 2)/2$ per original weight
Low-rank r=64 FP16 on 4096^2	0.5 added	$16 \cdot 64 \cdot 8192/4096^2$
DFloat11 (Huffman exponent)	10.8–11.1	as reported [200]; plus LUTs and gap arrays
TCA-TBE (3-bit exponent code)	≈11.2	$11 + 8(1 - c)$ with $c \approx 0.98$

14.5 System costs of distillation and recurrent models

- **On-policy distillation** interleaves generation (memory-bound, needs an inference engine) and scoring/updates (compute-bound). Teacher and student can share a cluster, but idle time arises from generation stragglers (variable-length reasoning traces). Reusing rollouts across steps and truncating them [24, 56] reduces cost.

- **Teacher serving.** A large teacher scoring student rollouts needs only forward passes with logits restricted to top-k or to the tokens sampled, saving logit bandwidth.

- **Recursive and early-exit models** require continuous depth-wise batching to realize throughput gains because sequences exit at different depths [147, 148]; mainstream serving engines are designed for uniform depth.

- **Delta serving** depends on batched multi-adapter kernels and adapter paging [170, 171].

14.6 Evaluation methodology

14.6.1 What to measure

1. **Fidelity to the original model.** For compression of an existing model: next-token KL divergence on held-out text, top-1 agreement, and the *flip rate*: the fraction of questions whose correctness changes. Dutta et al. show that compressed models can match accuracy while changing many individual answers ("flips"), so accuracy alone can mislead; they propose distance metrics between model outputs [189].

2. **Capability coverage.** Knowledge-intensive tasks, multi-step reasoning, coding, long-context retrieval, multilingual tasks, instruction following, tool use. The LLM-KICK benchmark argues that perplexity-based evaluations hide failures of knowledge-intensive tasks under pruning and quantization [190].

3. **Trustworthiness and safety.** Decoding Compressed Trust reports that compression can change dimensions such as toxicity, stereotype bias, privacy and robustness in non-obvious ways, with moderate compression often benign and further compression sometimes harmful [191].

4. **Efficiency, measured honestly.** Resident bytes (not nominal parameters), peak memory including KV cache and activations, time-to-first-token, time-per-output-token, throughput at a fixed latency SLO, batch-size sweeps, cost per million tokens, energy. State the hardware, software stack and kernel versions.

5. **Distillation-specific.** Held-out *prompt distribution shifts*; diversity (pass@k, self-BLEU); calibration; student–teacher agreement per domain; safety behaviour relative to the teacher; contamination checks.

14.6.2 Statistical power

Differences of one or two points on a benchmark are often within noise. For accuracy p on n i.i.d. questions the standard error is $\sqrt{p(1-p)/n}$.

Example 14.4 (How many questions to detect a 1-point drop?)**.** At $p = 0.70$ and $n = 1000$: $\mathrm{SE} = \sqrt{0.21/1000} = 1.45$ points, so a 95% interval is ± 2.8 points; a 1-point change is invisible. To detect a difference $\delta = 0.01$ between *two independent* accuracy estimates at 5% significance and 80% power requires $n \approx (1.96+0.84)^2 \cdot 2p(1-p)/\delta^2 = 7.84 \cdot 0.42/10^{-4} \approx 3.3 \times 10^4$ questions per arm. A *paired* test on the same questions (McNemar-style), exploiting that most answers agree, needs far fewer: with a disagreement rate d (fraction of questions where exactly one model is right), the required n scales as $\approx (1.96 + 0.84)^2\, d/\delta^2$; for $d = 0.10$ and $\delta = 0.01$ this is $7.84 \cdot 0.10/10^{-4} \approx 7.8 \times 10^3$. Sampling-based generation also adds variance across seeds; report multiple runs. (Arithmetic from standard formulas.)

14.6.3 Contamination, judges and imitation traps

- **Contamination.** Models, and especially teachers whose outputs are used for distillation, may have seen benchmark items. Fresh variants of benchmarks reveal drops (GSM1k reports performance declines for several model families relative to GSM8K) [81], and black-box tests can detect contamination statistically [82]. Decontaminate distillation corpora and prefer held-out or freshly written evaluation sets.

- **LLM-as-judge.** Judges exhibit position, verbosity and self-preference biases; use calibrated, randomized-order, multi-judge protocols and validate against human or programmatic checks [194]. Imitation students look good to judges because style transfers easily while capability may not [30].

- **Harness differences.** Prompt format, few-shot count, answer extraction and decoding settings change scores by amounts larger than many compression effects; use a standardized harness and report the configuration [192, 193].

14.6.4 A reporting checklist

Table 14.2: Minimum reporting for a compression or distillation claim.

Item	What to report
Baselines	Original model, a quantized model of equal bytes, a small model of equal footprint, and (if relevant) a distilled alternative
Size	Resident bytes including metadata, KV cache policy, peak memory
Quality	Perplexity *and* downstream tasks *and* flips/agreement; confidence intervals; paired tests
Safety / bias	Refusal rate, toxicity, bias probes relative to the original
Speed	Hardware, batch sizes, prompt/output lengths, TTFT and TPOT, kernel stack; both memory-bound and compute-bound regimes
Cost to produce	GPU-hours, tokens, teacher queries and their cost, human effort
Data	Calibration/training data provenance, licenses, decontamination procedure
Reproducibility	Seeds, code, checkpoints, harness version

Chapter 15

Recent Advances: A Timeline from 2022 to October 2026

This chapter orders the works cited in this paper by date, groups them into trends, and notes inflection points. Dates are those of first public release (usually arXiv) or the stated publication; months are given where the author is confident. Items from 2026 were located by web search in October 2026 and, except where noted, only their abstracts or pages were read, so their conclusions are reported at abstract level and are not independently assessed.

15.1 Timeline table

Table 15.1: Timeline of selected developments (only works cited in this paper).

Date	Area	Development
2022-03	Scaling; merging	Compute-optimal scaling fits (Chinchilla) [173]; model soups [160]
2022-04	Structured	Monarch matrices and projection from dense weights [119]
2022-09	Distillation	Context distillation: teach a model to behave as if the prompt were present [79]
2022-10/11	PTQ; decoding	GPTQ [186]; speculative decoding and speculative sampling (Nov 2022 / Feb 2023) [77, 78]
2022-12	Merging	Task arithmetic [161]
2023-03	Imitation	Alpaca-style instruction imitation from API outputs [26]
2023-05	Imitation critique	"False promise of imitating proprietary LLMs" [30]; step-by-step distillation [31]; QLoRA [106]; LIMA [37]
2023-06	On-policy KD; synthetic data	MiniLLM [22], GKD [23]; Orca [28]; Textbooks Are All You Need [36]; LLMZip [212]
2023-09	Theory	Language modeling is compression [211]
2023-10/11	Serving	Punica [170], S-LoRA [171]; DARE [163]
2023-12	Low-rank	ASVD [94]; LASER [101]; non-vacuous LLM bounds (SubLoRA) [227]
2024-01	Scaling; fusion	Beyond Chinchilla-optimal [174]; FuseLLM [73]
2024-02	Deltas; small LMs	BitDelta [167]; MobileLLM [154]; DistiLLM [24]; universal logit distillation [72]

Date	Area	Development (continued)
2024-03	Low-rank; trust	SVD-LLM [95]; Decoding Compressed Trust [191]; stealing part of a production LM [69]; over-training scaling [175]
2024-05	Low-rank + quant; KV	CALDERA [109]; DeepSeek-V2 and MLA [104]
2024-07	Pruning + KD; models	Minitron [61]; Llama 3 [50]; Gemma 2 [48]; model collapse in *Nature* [65]
2024-09	Small models	Llama 3.2 1B/3B via pruning and logit distillation [51]
2024-10	Recursion; lossless	Relaxed Recursive Transformers [147]; NeuZip [202]; Basis Sharing [98]
2024-11	Lossless; NAS; scaling	ZipNN [201]; Puzzle [60]; Scaling laws for precision [177]
2024-12	Scaling	Densing law [176]
2025-01	Reasoning distillation	DeepSeek-R1 and six distilled dense models (about 800k samples) [41]; s1 [42]
2025-02	Reasoning; scaling	LIMO [43]; Distillation scaling laws [53]; recurrent-depth latent reasoning (Huginn) [150]; Dobi-SVD [97]; "looped transformers reason" [151]; "small models struggle to learn from strong reasoners" [46]
2025-03	Models; low-rank	Gemma 3 [49]; SVD-LLM V2 [96]
2025-04	Lossless; reasoning	DFloat11 [200]; Phi-4-reasoning [45]
2025-05	Models	Qwen3 with strong-to-weak distillation [47]
2025-06	Data	OpenThoughts data recipes [44]
2025-07	Recursion; safety	Mixture-of-Recursions [148]; subliminal learning [64]
2025-08/09	Lossless; low-rank	Lossless compression of weights, checkpoints, KV in low precision [205]; CoSpaDi [99]
2025-10	Lossless; loops; OPD	ECF8 exponent concentration [203]; Ouro looped LMs [149]; "On-policy distillation" write-up (27 Oct) [52]
2026-01	OPD	On-policy self-distillation with privileged information [54]
2026-02	OPD; legal	Prefix on-policy distillation [56]; allegations of industrial-scale distillation campaigns against a commercial model (23 Feb) [63]
2026-03	OPD; lossless	Relaxed OPD (REOPOLD) [55]; ZipServ fused lossless GEMM [204]
2026-04	OPD analysis	Rethinking on-policy distillation [59]
2026-05/06	OPD theory	Decoupling KL and trajectories [57]; geometry of OPD updates [58]

15.2 Trends and inflection points

15.2.1 From imitation to principled on-policy distillation

In 2023 distillation news was dominated by instruction imitation from API outputs, followed almost immediately by the critique that imitation transfers style more than skill [30]. The same year saw the theoretical repair: casting distillation as imitation learning with student-generated prefixes [22, 23]. The 2025–2026 period elevated this to a first-class post-training stage: Qwen3's pipeline and the October 2025 write-up on dense per-token reverse-KL rewards [47, 52], followed by a stream of 2026 refinements concerning cost (prefix-only rollouts [56]), stability (reward clipping, entropy-based sampling [55]), self-teachers with privileged information [54], failure analysis [59], unified objectives [57] and update geometry [58]. The author's reading is that on-policy distillation is converging on a standard recipe, with the open questions about where it stops working (student/teacher mismatch) rather than whether it works.

15.2.2 Distilled reasoning as the dominant "compression" story

R1's release of distilled dense models in January 2025 reframed the field: a cheap SFT on teacher traces moves a small model far along the reasoning curve [41]. Within months, minimal-data variants (s1, LIMO), open data recipes (OpenThoughts) and curated teachable-prompt approaches (Phi-4-reasoning) refined which traces matter [42–45], while the capacity-gap result counselled caution at the small end [46].

15.2.3 Post-hoc low-rank: from SVD to activation-aware to hybrid

Plain SVD of LLM weights was ineffective; activation-aware scaling (ASVD), then Cholesky whitening with exact loss accounting (SVD-LLM), then heterogeneous allocation and differentiable truncation (SVD-LLM V2, Dobi-SVD), and finally hybrids (shared bases, sparse dictionaries, low-rank plus quantization) [94–99, 109]. The hybrids' trend, the author notes, is toward models in which low-rank is one component among several rather than the sole compression mechanism.

15.2.4 Recursion returns through reasoning

Parameter sharing (ALBERT, Universal Transformers) lay dormant for LLMs until relaxed recursion and looped models revived it, motivated by reasoning rather than by memory: depth-not-parameters arguments, recurrent-depth test-time scaling, Ouro, and Mixture-of-Recursions [147–151]. Adoption in frontier open-weight models, as far as the author has been able to verify, remains limited.

15.2.5 Lossless coding becomes a GPU-kernel topic

ZipNN (2024) established the exponent-skew finding and storage use; DFloat11 (April 2025) brought on-GPU decode for inference; ECF8 extended to FP8; ZipServ (March 2026) moved to hardware-aware fixed-length codes with a fused GEMM [200, 201, 203, 204]. The trajectory is from *variable-length, high-ratio, offline* to *fixed-length, slightly lower ratio, fused.*

15.2.6 Security and legal turn

Subliminal-learning results and the February 2026 allegation about distillation campaigns mark the moment at which distillation became a governance issue as much as a technique [63, 64]. Model-collapse theory and the Nature paper contributed the data-side warning [65].

15.2.7 A sketch

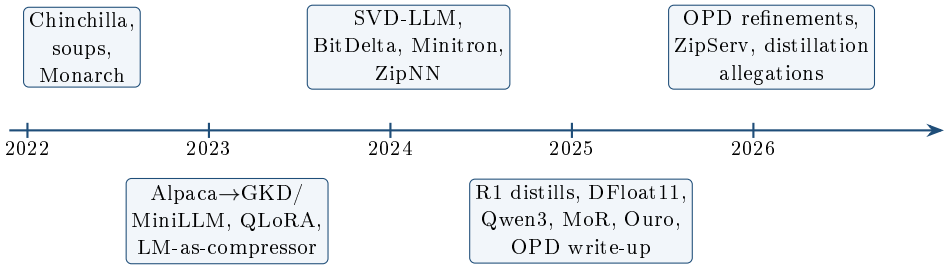


Figure 15.1: Schematic timeline of the main waves (not to scale; selected items).

What the timeline suggests

The fastest-moving pieces are those with a strong feedback loop to production training pipelines (on-policy and reasoning distillation). The slowest-moving are those that require new kernels and serving stacks (recursion, structured matrices). Lossless coding sits in between: the theory is trivial, the engineering is not.

Chapter 16

Capabilities: What Is Achievable Today

This chapter collects the quantitative claims from the sources read for this survey, organized by family, and adds the author's estimates where sources give none. Every row states its provenance. Rows are *not* comparable across tables: models, baselines, benchmarks, hardware and dates differ. Where a source gave no number the cell says so rather than filling it from memory.

16.1 Distillation and derived small models

Reading the table. Three patterns recur. (i) Derived models retain a very high fraction of a parent's benchmark accuracy at a small fraction of the training tokens when starting from pruned weights (Minitron, Puzzle). (ii) For reasoning, the cost of eliciting the skill is tiny (a thousand examples) but depends on a capable base. (iii) On-policy distillation reports large cost advantages over RL on the same student, in self-reported blog and abstract-level numbers.

16.2 Low-rank, structured and tensor methods

Reading the table. Post-hoc LLM factorization results are in the tens-of-percent range with caveats, in line with the analysis of Example 8.1. Order-of-magnitude reductions appear mainly for small or older models, or require training in the structured format. SliceGPT's result is the most hardware-friendly among them because it shrinks dense dimensions.

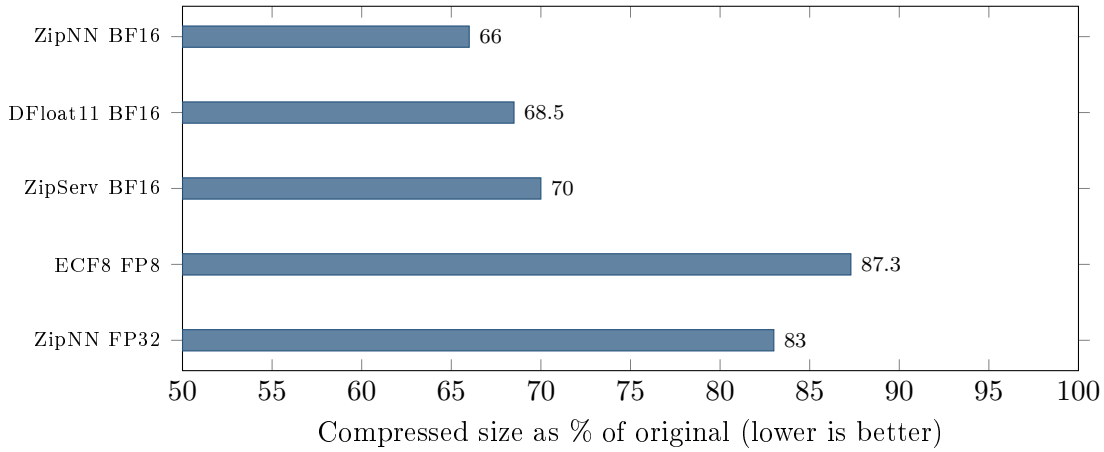


Figure 16.1: Compressed size fractions from the sources in Table 16.5. DFloat11 value is the midpoint of the reported 67.6–69.5%; ZipServ is $100-30$ from "up to 30%" (so best case); ECF8 is the midpoint of 9.8–14.8% savings (85.2–90.2%). Bars are reconstructions of reported ranges, not new measurements.

16.3 Sharing, recursion and looping

16.4 Delta compression, merging and serving

16.5 Lossless compression

16.6 Cost to produce

16.7 What practitioners can count on, with ranges

The shape of what is achievable

Exact-preserving methods reach about 1.4×. Approximate post-hoc methods without training reach about 1.3–2× with measurable degradation. Methods that include training (distillation, prune+distill, uptraining) reach 2× and beyond with high retention but cost compute and carry the risks of the next chapter. The gap between 1.4× and beyond is where entropy limits end and learned approximation begins.

Table 16.1: Reported results for distillation and derived models (as stated in the cited sources).

Source	Claim	Provenance / caveat
DistilBERT [11]	40% smaller, about 60% faster, retains about 97% of BERT language-understanding performance	Paper-level claim on GLUE-type tasks (encoder)
TinyBERT [12]	about 7.5× smaller and 9.4× faster	Paper-level claim; encoder, GLUE
Minitron [61]	up to 40× fewer training tokens per derived model; 1.8× lower compute to train the family	Abstract statement; 15B→8B/4B family
Puzzle [60]	2.17× throughput, 98.4% of parent's benchmark accuracy, ≤45B tokens (parent 15T), fits one H100	Abstract; parent Llama-70B-class, derived ≈49–51B
Qwen3 pipeline stages [47, 52]	AIME'24: 55.0% (off-policy KD) → 67.6% (RL) → 74.4% (on-policy KD); OPD stage 1,800 GPU-h vs. RL 17,920 GPU-h	As quoted in the Thinking Machines write-up from the Qwen3 report; one 8B-class student setting
On-policy distillation write-up [52]	AIME'24 about 60% → about 70% in about 150 steps; estimated 9–30× cheaper in FLOPs than extrapolated SFT to 2M prompts (about 18× in GPU hours)	Blog-level estimates by the authors
Prefix OPD [56]	2–47× training compute reduction vs. full OPD at matched accuracy on math and OOD	Abstract
REOPOLD [55]	7B student matches 32B teacher on visual reasoning	Abstract-level
R1 distilled family [41]	six dense students from about 800k samples	Per-model scores not re-verified here
s1 [42]	1,000 examples; AIME24 50%→57% with budget forcing	Abstract; base Qwen2.5-32B-Instruct
LIMO [43]	AIME24 63.3%, MATH500 95.6%	Abstract; vs. 6.5%/59.2% for earlier fine-tuned models
OpenThoughts3-7B [44]	AIME25 53%, LCB 51%, GPQA-D 54%; +15.3/ + 17.2/ + 20.5 over R1-Distill-Qwen-7B	Abstract; teacher QwQ-32B; 1.2M examples
Distillation scaling laws [53]	Distillation beats supervised pretraining only up to a compute level that grows with student size, if a	Qualitative abstract claim

Table 16.2: Reported results for factorization-type methods.

Source	Claim	Provenance / caveat
ASVD [94]	10–30% compression without training; 50% KV-cache reduction without performance loss	Abstract; "without loss" is the authors' statement
SliceGPT [100]	up to 25% of parameters removed; about 99% (Llama 2 70B, OPT 66B) and 90% (Phi-2) of dense zero-shot performance; Llama 2 70B inference compute about 64–66% of dense	Abstract
CALDERA [109]	outperforms prior post-training methods below 2.5 bits per parameter (Llama 2 7B/13B/70B, Llama 3 8B)	Abstract; no numbers quoted
CoSpaDi [99]	outperforms SVD-based and structured pruning baselines at 20–40% compression	Abstract
SVD-LLM / V2 [95, 96]	better than prior SVD methods, especially at high compression; seven models/ten datasets (V1), five LLMs/ten datasets (V2)	Abstracts; no numbers re-verified
Kronecker RNNs [126]	15–38× compression	Title-level claim; small IoT-oriented RNNs
Deep Compression [185]	35× (AlexNet), 49× (VGG-16) with pruning + quantization + Huffman	Original abstract; vision models
TT/Monarch parameter counts	482× (TT, r=16) and 32× (Monarch) fewer parameters on a 4096^2 layer	**Computed** from formulas (Examples 9.1 and 9.2); says nothing about accuracy

Table 16.3: Reported results for parameter sharing and recurrence.

Source	Claim	Provenance / caveat
ALBERT [140]	best model has fewer parameters than BERT-large with state-of-the-art GLUE/RACE/SQuAD at the time	Abstract; no exact numbers read
Relaxed Recursive Transformers [147]	recursive Gemma 1B outperforms TinyLlama 1.1B and Pythia 1B; recovers much of Gemma 2B; theoretical 2–3× throughput from continuous depth-wise batching	Abstract; throughput is analytical
Mixture-of-Recursions [148]	non-embedding parameters 98–167M vs 315M; 43.1% vs 42.3% few-shot at equal FLOPs; 25% fewer FLOPs, 19% less time, 25% less peak memory at equal data; up to 2.06× throughput	As read from the paper's page; scales 135M–1.7B
Huginn [150]	3.5B parameters, 800B tokens; reasoning improves with recurrence up to compute equal to a 50B model	Abstract
Ouro [149]	1.4B and 2.6B on 7.7T tokens match results of LLMs up to 12B on a range of benchmarks	Abstract

Table 16.4: Reported results for family compression and multi-tenant serving.

Source	Claim	Provenance / caveat
BitDelta [167]	>10× GPU-memory reduction in multi-tenant serving via 1-bit deltas	Abstract; latency gain unquantified in abstract
Punica [170]	about 12× throughput over state-of-the-art serving on a fixed cluster; about 2 ms added per-token latency	Abstract
S-LoRA [171]	up to 4× throughput over HF PEFT and naive vLLM; thousands of adapters	Abstract
Delta-CoMe [168]	training-free mixed-precision delta compression by singular value	Abstract; no numbers
100 fine-tunes of a 7B base	1,400 GB → about 101.5 GB (BitDelta-style) or 22 GB (LoRA r=16)	**Computed** (Example 11.1)

Table 16.5: Reported results for lossless weight compression (compressed size relative to original unless stated).

Source	Claim	Provenance / caveat
ZipNN [201]	BF16 ≈66%; regular FP32 ≈83%; ≈62% faster than zstd; up to 80 GB/s decompress with threads	Page text; storage/transfer oriented
DFloat11 [200]	67.6–69.5% of BF16; Llama 3.1 405B 811.71→551.22 GB; 2.3–46.2× throughput vs CPU offload; 5.7–14.9× longer generation under fixed memory	Paper page; baselines are offloading, not an in-memory dense model
ECF8 [203]	9.8–14.8% memory savings for LLM FP8; up to 26.9% (video model); up to 177.1% throughput under fixed memory (12 GB GPU)	Paper page; throughput is relative to a baseline that did not fit
ZipServ [204]	up to 30% smaller; kernel up to 2.21× vs cuBLAS; mean 1.22× end-to-end vs vLLM	Abstract and page; consumer/workstation GPUs
NeuZip [202]	Llama-3 8B training memory 30.97→15.25 GiB, same loss; lossy inference 15.08→7.70 GiB at perplexity 9.93 (vs 9.89)	Abstract-level text
Heilper–Singer [205]	ratios up to 62% (BF16) and 83% (FP8)	Abstract does not state direction of the ratio

Table 16.6: Cost classes for producing a compressed or distilled model. Entries with "reported" come from the cited sources; "estimate" marks the author's order-of-magnitude judgment.

Method	Cost	Basis
Lossless coding	seconds–minutes per GB on CPU; kernel integration effort	Reported coder speeds (ZipNN); estimate for integration
Delta 1-bit (BitDelta)	minutes; scale distillation on small calibration set	Estimate from method description
Whitened SVD (SVD-LLM style)	calibration forward passes plus one SVD per matrix: minutes to a few hours on one GPU for 7B-class	**Estimate** (author's), no source figure
Low-rank healing	short fine-tune (hours–days on a few GPUs)	Estimate
Prune + distill (Minitron)	up to 40× fewer tokens than from scratch	Reported
Heterogeneous NAS + KD (Puzzle)	≤45B tokens on a 15T-token parent lineage	Reported
Reasoning SFT from traces	10^3 to 10^6 examples; teacher generation often dominant	Reported dataset sizes; Example 7.2
On-policy distillation	hundreds of steps; 1,800 GPU-h for an 8B-class stage in Qwen3 as quoted	Reported via [52]
Pretraining small from scratch	$6ND$ with $D \sim 10^{12}$–10^{13} tokens	Computed formula; Llama 3 8B ≈15T tokens [50]
Recursive uptraining	modest token budget relative to pretraining	Qualitative [147]

Table 16.7: Author's summary of achievable outcomes. Ranges are judgments informed by the cited sources, not measurements; "quality" means broad benchmark retention for a typical 7B-class model unless exact preservation is stated.

Technique	Size reduction	Quality	Speed effect
Lossless BF16 coding	1.4–1.5×	exact	Faster only if fused or offload-bound
Lossless FP8 coding	1.1–1.2×	exact	As above
Delta 1-bit / LoRA family	10–60× per additional variant	near-lossless to small drop	Multi-tenant throughput gains
Whitened SVD, 20–30%	1.25–1.4×	small to moderate drop, healing helps	Marginal at batch 1 unless fused
Width slicing (SliceGPT)	1.3× (25%)	90–99% of zero-shot	Dense shrink: real gains
Prune + distill	1.5–3×	high retention (e.g., 98% in Puzzle)	Real FLOP and byte cuts
Distilled student of different family	2–100×	varies strongly by task and student size	Real
Recursion / sharing	2–3× fewer unique params	competitive at small scale (reported)	Not by itself; needs adaptive depth

Chapter 17

Pros, Cons and Risks

17.1 Strengths and weaknesses by family

Table 17.1: Pros and cons of the families in this paper (author's synthesis from the preceding chapters).

Family	Pros	Cons
Distillation (logit, on-policy)	Real FLOP and byte reduction; student can be any architecture; strong results at modest cost with a good teacher; on-policy methods are sample-efficient	Needs training pipeline and teacher; distribution-limited; capacity gap; teacher bias inheritance; legal exposure with third-party teachers
Black-box / reasoning distillation	No teacher internals needed; effective for style and reasoning elicitation; open data recipes exist	Text-only supervision is information-poor; ToS and IP issues; contamination; safety trait transfer; capacity gap for small students
Post-hoc low-rank	Training-free or short healing; works on frozen weights; composes with quantization	Weights not low-rank; harsh rank cuts for modest ratios; wall-clock gains uncertain; brittle past a threshold
Structured / tensor	Orders-of-magnitude parameter reductions are possible; sound theory	Needs training in format; kernels immature; little deployment evidence for LLM body layers
Hashing	Simple; pure memory saving	No FLOP saving; blind collisions; training from scratch
Sharing / recurrence	Fewer unique parameters; extra depth helps reasoning; adaptive compute possible	No FLOP, KV or streamed-byte savings by default; needs special serving; limited large-scale evidence
Merging / deltas	Excellent for many variants; no retraining; mature serving kernels	Lossy deltas; interference in merges; requires shared base
Lossless coding	Zero behaviour change; simple guarantees; storage and transfer wins	Ceiling about 1.4× (BF16) or 1.1–1.2× (FP8); decode overhead; kernel dependence
Small model from scratch	Clean, no inheritance; benefits from rising capacity density	Expensive pretraining; needs data; no relation to the large model's idiosyncrasies

17.2 Risk register

Each risk below gives a mechanism, evidence, early-warning signals and mitigations. A summary matrix with the author's subjective ratings closes the section.

17.2.1 Capability and skill loss hidden by benchmark parity

Mechanism and evidence

Aggregate accuracy and perplexity are dominated by common cases. Compression can preserve the bulk while eliminating rare skills. Imitation students achieve high preference ratings while a capability gap remains [30]; perplexity can stay flat while knowledge-intensive tasks fail [190]; accuracy can be unchanged while many individual answers flip [189]. Proposition 8.2 explains why: loss is a probability-weighted quadratic form, so perturbations on low-probability tokens are cheap in loss but may be decisive for reasoning chains.

Warning signs: unchanged perplexity with degraded long-form or multi-step tasks; high flip rates; divergence on out-of-domain probes. *Mitigations*: evaluate fidelity (KL, agreement, flips), include knowledge-intensive and long-horizon tasks, and maintain a private, task-specific regression suite (Chapter 14).

17.2.2 Distribution shift

A distilled or healed model is accurate on the distribution used for distillation or calibration. If inference prompts come from a distribution q' with density ratio $w(x) = q'(x)/p(x) \leq w_{\max}$ with respect to the distillation prompt distribution p, then for any nonnegative per-prompt error $e(x)$,

$$\mathbb{E}_{q'}[e] \leq w_{\max}\, \mathbb{E}_p[e], \tag{17.1}$$

so a student with a small error on p can have an error up to $w_{\max}$ times larger under shift, and $w_{\max}$ is unbounded for prompts outside the support of p. Calibration sets for SVD-type methods carry the same issue: directions with low variance on the calibration set are removed, and they may be exactly those used by out-of-distribution inputs. *Mitigations*: diverse and domain-covering calibration/distillation sets; on-policy sampling that exposes the student's own failure modes [23]; monitoring of production traffic against the training distribution.

17.2.3 Hallucination increase

Fine-tuning on facts a model does not know encourages hallucination [85]. In distillation, teacher outputs include claims the student cannot ground, so the student learns to imitate the form of confident knowledge. Reasoning traces compound this: long traces provide many opportunities for ungrounded steps. *Mitigations*: verify and filter teacher data where possible (answer checking, retrieval support), prefer on-policy objectives that penalize student mass where the teacher has none (reverse KL), and measure factuality and calibration separately from accuracy.

17.2.4 Safety and alignment transfer or loss

Safety behaviour is not guaranteed to survive, or to stay behind

Benign fine-tuning can erode safety behaviours [83]; narrow fine-tuning can induce broad misalignment [84]; compression changes trustworthiness metrics in non-monotone ways [191]; and distillation can transmit hidden behavioural traits between models sharing a base even through filtered, semantically unrelated data [64]. Conversely, a student trained from teacher outputs that include refusals may inherit refusal *style* without the underlying robustness, and jailbreak resistance can differ.

Mitigations: re-run the full safety evaluation on every compressed or distilled model; include red-team prompts in distillation data; distill from a teacher whose safety-relevant behaviour was audited; avoid same-base teacher–student pairs for untrusted teachers or filter more aggressively (the subliminal-learning result applies when base models match); keep safety classifiers or system-level guards independent of the student.

17.2.5 License, ToS and IP exposure

Using a provider's outputs to train a competing model can breach its terms; open-weight licenses carry their own conditions; and extraction campaigns may trigger account bans and legal action [63]. Copyright and contract questions are unsettled (Chapter 7). *Mitigations*: compliance checklist and provenance logs; use permissively licensed teachers; in-house teachers; legal review for commercial use.

17.2.6 Teacher-bias inheritance

A student inherits the teacher's demographic, cultural, political and stylistic biases and its blind spots; multi-teacher blends mix them. On-policy distillation can *amplify* teacher biases through mode-seeking (reverse KL emphasizes teacher's modal behaviour). *Mitigations*: bias probes comparing student to teacher; diversity-promoting data mixtures; periodic audit.

17.2.7 Model collapse from synthetic data

Training repeatedly on model-generated data loses tail information [65, 66]; in distillation pipelines the risk appears when students become teachers for later students, or when synthetic data replace real data wholesale. Example 7.3 gives the mechanism. *Mitigations*: anchor with real data; avoid recursion beyond one or two generations without fresh data; track tail metrics (rare-token perplexity, long-tail QA).

17.2.8 Benchmark contamination

A teacher that has seen test items yields distilled data with leaked answers; students then appear to "reason" on benchmarks they have effectively memorized. Fresh-variant benchmarks reveal declines for some model families [81], and statistical tests can flag contamination in black-box models [82]. *Mitigations*: n-gram and embedding decontamination of distillation sets; evaluate on post-cutoff or privately authored items; compare with the teacher on fresh variants.

17.2.9 Low-rank brittleness

Layer-wise errors compound. Consider an L-layer residual network with per-layer relative perturbation ε and perturbations propagated with gain at most $1+\varepsilon$ per layer (a crude worst-case); the final relative error bound is $(1+\varepsilon)^L - 1$. For $\varepsilon = 0.05$ and $L = 32$ this is $(1.05)^{32} - 1 \approx 3.76$, i.e. a vacuous bound. If instead errors are independent and add in quadrature the typical figure is $\sqrt{L}\,\varepsilon \approx 0.28$. Reality lies between, and the transition is often sharp: error grows slowly until the rank falls below the effective rank of some critical layers, then sharply. This is why SVD-type methods report their largest advantages at *high* compression ratios, where prior methods degrade [95]. (Toy computation, not an empirical law.) *Mitigations*: heterogeneous allocation (Theorem 8.2), sensitivity analysis, healing, and a safety margin from the knee.

17.2.10 The no-wall-clock-speedup trap

Parameter counts fall but latency does not. Mechanisms: launch overhead and thin GEMMs for factorized layers (Example 14.2); sharing leaves FLOPs, KV and streamed bytes unchanged (Proposition 10.1); decoupled decompression increases traffic (Proposition 12.4); sparse or structured formats lack kernels. *Mitigations*: measure on target hardware at the target batch size and context length before committing; prefer methods whose savings map to dense, large GEMMs.

17.2.11 Entropy-decoding overhead

Decode work and variable-length boundaries reduce effective bandwidth (43.7% and 76.5% of peak for two GPU coders on one device [204]); decoupled schemes add traffic and per-forward constant overhead [200]. Gains in the cited papers are against specific baselines (offloading, fixed-memory budget, a single serving stack). *Mitigations*: use load-time decoding for storage and transfer; use fused kernels for runtime; validate on the target accelerator; consider the interaction with tensor-parallel sharding.

17.2.12 Additional risks

- **Capacity-gap mismatches** waste distillation compute [46, 53].
- **Perishability**: capacity density rises quickly [176], so a compressed older model can be dominated by newer small models within months.
- **Reproducibility**: deterministic decoding and bit-exactness needed for lossless claims; GPU kernel nondeterminism can break arithmetic-coding pipelines (Chapter 13).
- **Security of compressed formats**: custom formats and decoders expand the attack surface (malformed compressed streams) and complicate supply-chain verification.
- **Evaluation noise**: sub-point differences are below the noise floor of typical benchmark sizes (Example 14.4).

Table 17.2: Subjective risk matrix. Likelihood and impact are the author's judgments on a 1–5 scale for a typical production attempt of the named method, informed by the cited evidence; they are not measured quantities.

Risk	Likelihood	Impact	Most exposed methods
Hidden skill loss behind parity	4	4	all lossy; distillation, low-rank
Distribution shift	4	3	distillation, SVD with narrow calibration
Hallucination increase	3	4	black-box distillation
Safety trait loss/transfer	3	5	distillation, fine-tuned students
License / ToS / IP exposure	3	5	black-box distillation from APIs
Teacher-bias inheritance	4	3	distillation
Model collapse	2	4	recursive synthetic pipelines
Benchmark contamination	4	3	distillation from unknown teachers
Low-rank brittleness	4	3	post-hoc low-rank, aggressive ratios
No wall-clock speedup	4	3	low-rank, sharing, structured, hashed
Entropy-decode overhead	3	2	lossless coding in compute-bound serving

17.3 Risk matrix

Risk-minimizing defaults

(1) Choose the method whose gains map to dense GEMMs and that you can evaluate end-to-end. (2) Make exactness the default where possible (lossless) and approximation an explicit decision with measured fidelity. (3) Treat any distillation from a third-party teacher as a legal and security project, not only a modelling one. (4) Re-run safety and bias evaluation on every artifact.

Chapter 18

Practical Decision Guide

This chapter turns the preceding analysis into a procedure. It begins with a flowchart, then decision tables keyed on the binding constraint, the access you have to a teacher, and the serving regime, and finishes with worked scenarios using arithmetic from earlier chapters. The guide is the author's synthesis; it is a starting point to be validated by measurement on the user's own workload and hardware.

18.1 Flowchart

18.2 Decision tables

Table 18.1: Choose by binding constraint.

Binding constraint	First choices	Why / what to avoid
Capacity (does not fit)	Quantization; prune+distill; recursion/sharing for edge; lossless only if gap $< 30\%$	Lossless gives at most about $1.4\times$; avoid low-rank at small ratios (no net gain)
Decode latency (bandwidth)	Smaller student; quantization; fused lossless; speculative decoding with distilled draft	Sharing does not cut streamed bytes; decoupled decompression adds traffic
Throughput at large batch / prefill	Smaller student; MoE; efficient low-rank only with good kernels	Byte-saving methods give nothing once compute-bound
Long-context KV cache	KV-latent designs (MLA-like), KV quantization/low-rank, shared KV	Weight compression barely helps (Example 14.3)
Storage / distribution	Lossless coding; delta coding of variants	Cheap, exact, load-time decode
Many fine-tunes	LoRA/adapter serving; BitDelta-style deltas	Merge only if some interference is acceptable
Exactness / audit	Lossless only	Everything else changes outputs
Training budget absent	Post-hoc: quantize, whitened low-rank, slicing, lossless	Distillation needs training

Table 18.2: Choose by teacher access.

Access	Method	Notes
Own teacher, full logits	Offline KD for bulk, on-policy distillation for post-training; prune+distill	Best case; cache top-k or run teacher online
Open-weight teacher, license permits	As above; check tokenizer match	Tokenizer mismatch requires alignment or universal losses [72, 75]
Top-k log-probs only	Truncated KD on retained mass	Beware renormalization bias
Text only (API)	Trace SFT with verification, then RL; mix easy and hard traces	Check ToS; capacity-gap and contamination checks
No teacher	Train small from scratch, long	Baseline to beat [174]

Table 18.3: Choose by serving regime. B = tokens per step; $B^\star$ from Proposition 14.1.

Regime	Likely to help	Unlikely to help
$B \ll B^\star$ (single stream decode)	Quantization, fused lossless, small student, low-rank with small r and fused kernels	Sharing, big dense low-rank with launch overhead
$B \approx B^\star$	Mixed approaches; test everything	Variable-length decoders without fusion
$B \gg B^\star$ (batched, prefill)	Smaller student, MoE sparsity, kernel-efficient factorization	Anything that only saves bytes
Edge / on-device	Quantization + small student, embedding tying/-factorization, sharing	Large vocab embeddings (Table 9.2)
Serving thousands of variants	Deltas and adapter batching	Full copies

18.3 Worked scenarios

18.3.1 Scenario A: a 70B model on one 80 GB GPU

BF16 weights occupy 140 GB. Lossless coding at about 0.66–0.70 gives 92–98 GB: still over capacity. 8-bit weights need about 70 GB, leaving about 10 GB for KV and activations: with 80 layers, 8 KV heads and $d_h = 128$ (a hypothetical shape) the cache costs $2{\cdot}80{\cdot}8{\cdot}128{\cdot}2 = 320$ KiB per token, so about 31,000 tokens in total across all sequences. 4-bit weights need about 35–37 GB with group scales. **Conclusion**: exactness is impossible on this card; use quantization (4-bit) and, if possible, prune and distill to a 30–40B-class student (Puzzle-like [60]); lossless coding of the *quantized* model is optional and adds at most 15–20% for uniform grids (Example 12.2).

18.3.2 Scenario B: a 1–3B model for on-device use

Embeddings dominate (Table 9.2); tie input/output embeddings, consider factorized embeddings and vocabulary trimming; train or distill a deep-and-thin student [154]; quantize to 4-bit; consider layer sharing if flash capacity is the limit and the extra compute is affordable. Distil from a larger in-family teacher; evaluate battery and latency on the device.

18.3.3 Scenario C: 500 customer fine-tunes of a 7B base

Full copies: $500 \times 14 = 7{,}000$ GB. With LoRA-style adapters (rank 16, about 80 MB each, Example 11.1): $14 + 500 \times 0.08 = 54$ GB: it fits on a single 80 GB GPU, and multi-adapter batching serves them concurrently [170, 171]. For full fine-tunes, BitDelta-style 1-bit deltas cost 0.875 GB each: $500 \times 0.875 = 437.5$ GB plus base, so they must be paged or sharded. **Conclusion**: adapters where training can be constrained; deltas where it cannot.

18.3.4 Scenario D: distributing 140 GB of weights to 1,000 nodes

Raw transfer is 140 TB. At ZipNN-like BF16 ratios (0.66) this is about 92 TB, saving about 48 TB per rollout; decode at load time on CPU at tens of GB/s is negligible relative to transfer [201]. Delta-coding successive checkpoints can save more when only fine-tuned layers change. **Conclusion**: lossless coding is the obvious, nearly free choice here.

18.3.5 Scenario E: a small reasoning model from an API-only teacher

Check ToS first. If permitted, generate several verified traces per prompt (Algorithm 4), start with a base of at least several billion parameters (capacity gap), mix long and short traces [46], decontaminate, SFT, and then run RL with verifiable rewards or self-distillation. Budget teacher generation (Example 7.2); evaluate safety and hallucination.

18.4 Rules of thumb

1. *Compression ratio is not a speedup.* Calculate bytes streamed and FLOPs for each regime.
2. *Lossless first for storage and distribution; lossy only with measured fidelity.*
3. *If you can train, distill; if you cannot, quantize.* Low-rank sits third.
4. *Do not distill from a teacher you cannot legally or securely use.*
5. *Compare against a smaller model trained well.*
6. *Always re-evaluate safety.*

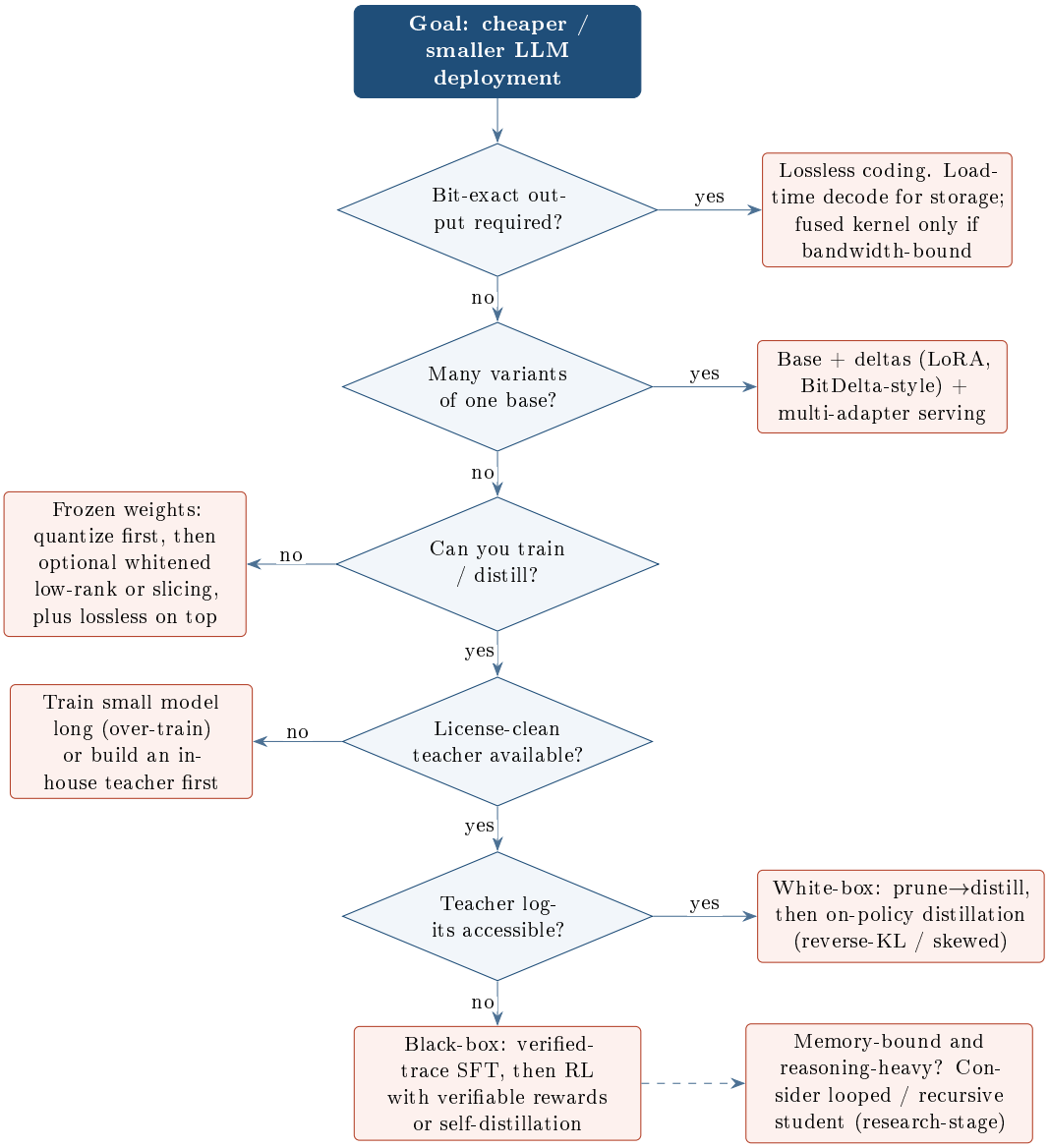


Figure 18.1: Decision flowchart. Quantization and pruning are treated in companion papers; they appear as the default first step for frozen weights. Lossless coding is composable with every branch.

Chapter 19

Estimated Priority in LLM Engineering

Read this first

The scores in this chapter are the author's *subjective judgments*, informed by the evidence cited throughout this paper. They are **not measured data**, not survey results, and not derived from any dataset of adoption. Different readers, with different workloads and risk tolerances, would score differently, which is why the chapter reports sensitivity to the weights. The purpose is transparency: to make the author's reasoning inspectable and contestable.

19.1 Rubric

Each sub-topic is scored on five criteria from 1 (low) to 5 (high). Orientation is chosen so that *higher is always better for priority.*

Table 19.1: Rubric and anchors. Weights are the author's base choice; alternatives are used for sensitivity analysis.

Criterion	Base weight	Meaning	Anchors (1 / 3 / 5)
Adoption	0.25	Production adoption in LLM pipelines	No known production use / used in niches or specific stacks / routine in major open and frontier pipelines
Momentum	0.20	Research activity and growth, 2025–2026	Few recent papers / steady stream / many new papers and rapid refinement
Hardware tailwind	0.15	Kernel, format, hardware and framework support improving	Headwind or none / mixed / strong ecosystem support and growth
Benefit left	0.20	Marginal benefit remaining if more effort is invested	Largely exhausted / moderate / large untapped gain
Safety (low risk)	0.20	Inverse of risk: legal, safety, quality, operational	High risk / moderate risk / low risk

The priority score is $P = \sum_c w_c s_c \in [1, 5]$. The base weights favour present-day adoption slightly but give equal weight to future potential (momentum, benefit left) and to risk. Four alternative weightings test robustness: *Equal* (0.2 each), *Adoption-heavy* (A 0.40, M 0.10, H 0.20, B 0.10, S 0.20), *Research-heavy* (A 0.10, M 0.35, H 0.10, B 0.30, S 0.15), *Risk-averse* (A 0.20, M 0.10, H 0.15, B 0.15, S 0.40).

19.2 Scores

Table 19.2: Scores (1–5) by sub-topic and base-weight priority, sorted by P (ties are listed in arbitrary order; ranks 1–2, 8–9, 10–13 and 16–17 are ties). The last column gives the rank under the Equal / Adoption-heavy / Research-heavy / Risk-averse weightings. All numbers are subjective.

#	Sub-topic	A	M	H	B	S	P	Alt. ranks
1	On-policy / logit distillation	5	5	4	4	3	4.25	1 / 3 / 1 / 5
2	Small-model over-training (baseline practice)	5	4	4	3	5	4.25	3 / 2 / 3 / 1
3	LoRA / adapters and multi-tenant serving	5	4	5	3	4	4.20	2 / 1 / 4 / 2
4	Prune-then-distill and distillation-based NAS	4	4	4	4	4	4.00	4 / 4 / 2 / 3
5	Reasoning-trace / black-box distillation	5	4	4	3	2	3.65	7 / 5 / 7 / 9
6	Low-rank KV / latent activations (MLA-style)	3	4	3	4	4	3.60	5 / 8 / 5 / 6
7	Draft-model distillation (speculative decoding)	3	3	4	3	5	3.55	6 / 6 / 8 / 4
8	Model merging / soups	4	3	3	2	3	3.05	9 / 9 / 15 / 12
9	Embedding tying, factorization, vocabulary reduction	4	2	3	2	4	3.05	14 / 7 / 17 / 8
10	Lossless coding for storage and transfer	2	3	3	2	5	2.95	12 / 10 / 12 / 7
11	Tokenizer-mismatch / multi-teacher distillation	2	3	3	4	3	2.95	11 / 11 / 10 / 13
12	Cross-layer sharing / looped / recursive	1	5	2	4	3	2.95	10 / 16 / 6 / 15
13	Delta compression (BitDelta-style)	2	3	3	4	3	2.95	8 / 12 / 11 / 14
14	Lossless coding fused into inference kernels	1	4	3	3	4	2.90	13 / 14 / 9 / 10
15	Low-rank + quantization hybrids	2	3	3	3	3	2.75	15 / 13 / 14 / 16
16	Post-hoc activation-aware SVD	2	3	2	3	3	2.60	16 / 15 / 16 / 17
17	LM-as-compressor, MDL, PAC-Bayes (theory)	1	3	1	3	5	2.60	17 / 17 / 13 / 11
18	Structured matrices and tensor trains	1	2	2	3	3	2.15	18 / 18 / 18 / 18

19.3 Justification of the scores

The table below states, for each sub-topic, the evidence that most influenced the scores. "Adoption" judgments are the author's reading of public technical reports and open-source ecosystems; no proprietary information was used, and absence of public evidence is not evidence of absence.

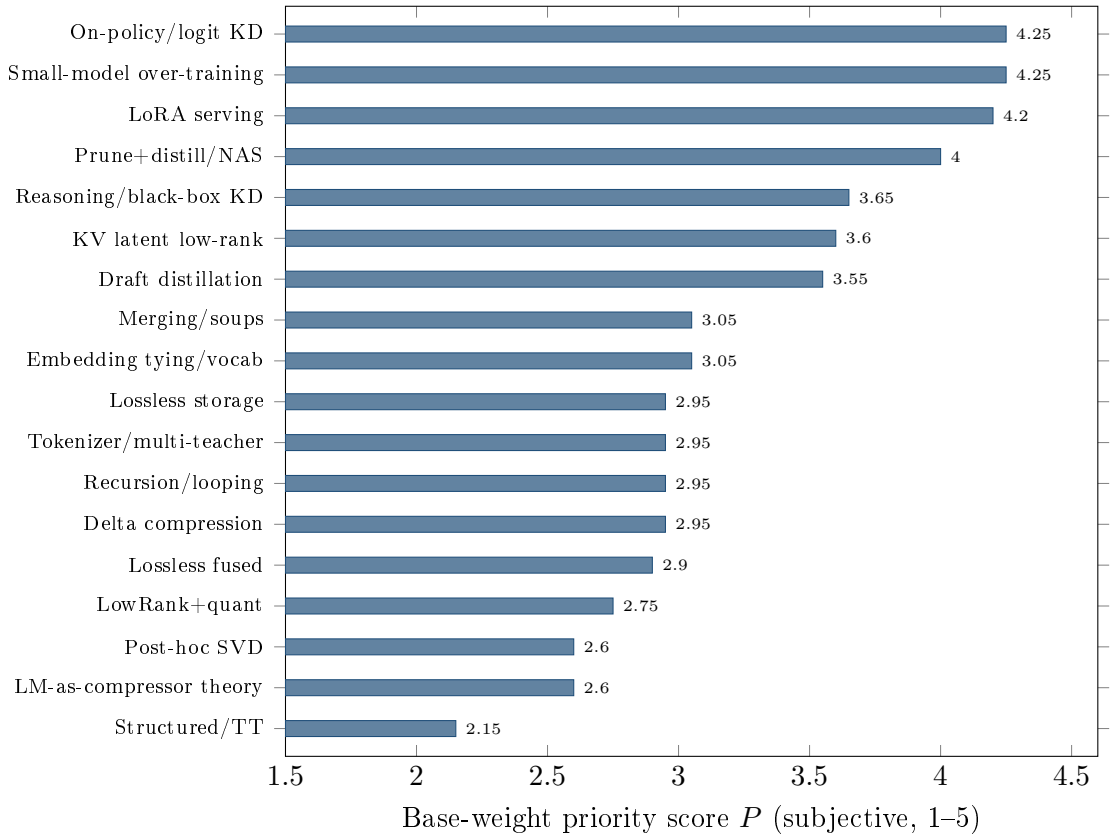


Figure 19.1: Base-weight priority scores from Table 19.2. These are subjective scores computed from the author's 1–5 ratings and the stated weights; they are not measurements.

Table 19.3: Evidence behind the scores.

Sub-topic	Reasoning
On-policy / logit distillation	Described in the Gemma 2/3, Llama 3.2 and Qwen3 materials as part of how smaller models are built [47–49, 51]; a dense stream of 2026 papers [54–59]; cost advantage reported over RL [52]. Safety score reduced for teacher-bias and trait-transfer concerns [64].
Small-model over-training	Llama 3 8B at about 15T tokens [50]; inference-aware scaling [174]; capacity density doubling [176]. Low risk. Listed because it is the baseline that everything else must beat.
LoRA / adapters and serving	Ubiquitous; mature multi-adapter kernels [105, 170, 171]. Benefit left moderate: rank and capacity questions [114, 115].
Prune + distill / NAS	Minitron, Puzzle, Llama 3.2 [51, 60, 61]; large token savings; low risk if teacher is in-house.
Reasoning / black-box distillation	Pervasive in the open ecosystem [41, 44, 45]; but legal, contamination and safety risks lower the safety score to 2 [63, 81]; capacity-gap limits [46].
Low-rank KV / latent activations	MLA in DeepSeek-V2 [104]; post-hoc variants [103]; KV dominates at long context (Example 14.3).

Sub-topic	Reasoning (continued)
Draft-model distillation	Speculative decoding is widely deployed [77]; distilled drafts improve acceptance [76]; outputs unchanged, hence safe.
Model merging / soups	Strong open-community use [166]; research is mature [165]; benefit left limited.
Embedding tying / vocab	Standard in small models [142, 154]; limited upside beyond what is done.
Lossless coding (storage)	Simple, safe, exact [201]; ceiling about 1.4×; adoption unclear to the author (scored 2 for A).
Tokenizer-mismatch / multi-teacher	Active niche [72, 73, 75]; useful when families differ.
Recursion / looping	Highest research momentum [147–151]; no confirmed frontier adoption known to the author; serving support lacking.
Delta compression	Clear value for multi-tenant serving [167–169]; production adoption unknown to the author.
Lossless fused kernels	Promising 2025–2026 results [200, 203, 204]; hardware-specific; small adoption so far.
Low-rank + quantization	CALDERA, LQ-LoRA [108, 109]; niche vs. strong plain quantization baselines.
Post-hoc SVD	Well-developed research line [94–97]; low adoption; wall-clock caveats (Example 14.2).
LM-as-compressor / MDL / PAC-Bayes	Intellectually central [211, 227]; little direct engineering use; score reflects *engineering* priority, not scientific value.
Structured / TT	Little LLM-body adoption [119, 128]; strong in embeddings and recommenders only.

19.4 Final ranking, with confidence

Confidence is the author's qualitative judgment of how likely the rank *band* is to survive further evidence, combining the spread of ranks across the five weightings (Table 19.2) and the quality of public evidence. "Direction" is the author's expected trend for the next 12–24 months.

How to use this ranking. A team's own priorities depend on its binding constraint (Chapter 18). The ranking is most useful as a default ordering of what to try and what to learn, not as a prescription. For instance a team that only distributes weights should rank lossless storage coding first; a team that serves hundreds of fine-tunes should rank adapters and deltas first; a team that cannot train should rank post-hoc methods first and accept that their rank in Table 19.2 reflects the field as a whole, not their situation.

Sensitivity. Under all five weightings, the top four sub-topics occupy ranks 1–5, and structured matrices rank last; embedding compression and recursion swing most (rank spread of 10) because they are sensitive to how much weight is given to present adoption versus future momentum. The tiers were drawn so that these sensitive items are placed in tiers 3–4 with

Table 19.4: Final tiers. Tiers group sub-topics with close base scores and overlapping rank ranges; the ordering within a tier is not meaningful.

Tier	Sub-topics	Rank range over weightings	Confidence	Direction
1 (core)	On-policy / logit distillation; small-model over-training; LoRA/adapter serving; prune-then-distill	1–5	High	On-policy distillation ↑; others →
2 (important, risk-managed)	Reasoning / black-box distillation; KV/latent low-rank; draft distillation	4–9	Medium	Reasoning distillation →, governance burden ↑; KV latents ↑
3 (useful niches)	Merging; embedding tying/vocab; lossless for storage; tokenizer-mismatch/multi-teacher; delta compression	7–17	Medium–low	Lossless ↑; delta ↑; others →
4 (watch)	Recursion / looping; lossless fused kernels	6–16 (recursion), 9–14 (fused)	Low	Both ↑; adoption contingent on serving support and replication
5 (low priority for engineering)	Low-rank + quantization; post-hoc SVD; LM-as-compressor theory; structured / TT	11–18	Medium–high that they rank low	Post-hoc SVD → (hybridizing); structured →; theory → (valuable but not engineering-actionable)

explicitly low confidence rather than a point rank.

Limitations. The scores rely on one author's reading of public sources through October 2026; adoption is the hardest criterion to assess because most production practice is private; the 1–5 scale has coarse resolution (hence ties); the five criteria are not independent (momentum and adoption correlate); and scores for a topic aggregate heterogeneous techniques. Readers are encouraged to substitute their own scores in Table 19.2: the formula is a weighted sum of five integers.

Chapter 20

Open Problems and Conclusions

20.1 Open problems

The items below are the author's selection of questions that the surveyed evidence leaves unresolved. They are phrased so that a concrete experiment or theorem would count as progress.

20.1.1 Distillation

1. **Sample complexity of on-policy distillation.** How many student rollouts and teacher-scored tokens are needed to reach a given divergence, as a function of horizon T, vocabulary and student capacity? DAgger-style bounds give $O(\epsilon T)$ versus $O(\epsilon T^2)$ [20] but do not account for the near-deterministic structure of most tokens, nor for reasoning traces thousands of tokens long.

2. **A theory of the capacity gap.** Empirically, stronger teachers can yield weaker students [8, 46], and compatible reasoning styles matter [59]. A predictive model of student quality as a function of teacher distribution sharpness, student capacity and tokenization would improve teacher selection. Distillation scaling laws are a start [53]; extending them to on-policy and reasoning settings is open.

3. **Which tokens carry the signal?** Entropy-weighted, disagreement-weighted or prefix-only training [56] suggest large inefficiencies in token usage. Principled token selection with guarantees is not available.

4. **Safe distillation.** Can one *prove* or reliably test that undesired traits are not transmitted [64]? How do safety properties transfer under different divergences and base-model relationships? What audits would detect transferred misalignment [84]?

5. **Provenance and rights.** Technical and legal mechanisms (watermark radioactivity [68], access controls, detection of extraction campaigns [63]) are immature; what is robust to paraphrase and filtering?

6. **Cross-tokenizer distillation at scale.** Current approaches align or approximate [72, 75]; guarantees on the induced objective are limited.

7. **Recursive distillation without collapse.** How many generations of student-to-student distillation are safe, and which data injections prevent tail loss [65]?

20.1.2 Low-rank, structured and hybrid

1. **End-to-end guarantees for rank allocation.** The layer-wise additive surrogate (Theorem 8.2) ignores interactions; a tractable bound that includes Jacobian propagation would

permit principled global allocation.

2. **Why are activations low-rank but weights not?** [102] documents it; a mechanistic explanation tied to training dynamics (and to the low intrinsic dimension of updates [117]) is missing, and could suggest *training* procedures that yield compressible weights.

3. **Hardware-conscious factorization.** Joint optimization of rank, alignment and fusion so that parameter savings translate to speed (Example 14.2); kernel libraries for fused $A(Bx)$; sharding of factorized layers in tensor parallelism.

4. **Low-rank experts.** Applying shared bases and low-rank deltas *across experts* of an MoE is natural (experts are fine-tuned relatives); published evidence in this survey is thin.

5. **Structured parameterizations at LLM scale.** Does pretraining with Monarch- or TT-structured FFNs reach dense quality at equal wall-clock? Few large-scale comparisons exist.

20.1.3 Sharing and recursion

1. **Scaling laws for looped and recursive models** at equal training FLOPs *and* at equal memory, against dense and MoE baselines up to frontier scale. Existing results are at 135M–2.6B parameters (and 3.5B for recurrent depth) from small numbers of groups [148–150].

2. **Knowledge versus manipulation.** Ouro's claim that looping improves manipulation rather than capacity [149] and the reasoning-versus-memorization trade-off [151] suggest hybrid designs (shared reasoning core plus large, unshared knowledge modules such as MoE). Which combinations dominate?

3. **Serving.** Continuous depth-wise batching, heterogeneous exit depths, KV policies and speculative decoding interplay; production-grade support is lacking.

4. **Router design.** Expert-choice versus token-choice routing exhibit a quality gap in MoR [148]; causality-safe, balanced and cheap routers remain open.

20.1.4 Lossless and entropy-coded compression

1. **Beyond i.i.d. exponent coding.** Context models using layer, channel or neighbouring-weight structure could exceed the ≈34% BF16 ceiling (Proposition 12.2); whether the gain justifies decode cost is unknown.

2. **Training for compressibility.** Regularizers or schedules that make exponents more concentrated, or weights more predictable, could be co-designed with entropy coders (an MDL-style objective, Chapter 13).

3. **Fused decode across hardware and parallelism.** ZipServ's fused kernels are demonstrated on a few GPUs [204]; coverage of data-center GPUs, multiple vendors, tensor parallelism and FP8/FP4 formats is incomplete.

4. **Lossless coding of activations and KV cache** [205] in a decode-bandwidth-bound regime.

5. **Determinism.** Lossless guarantees and LM-based arithmetic coding both need bit-exact reproducibility across hardware; deterministic inference at scale is expensive.

20.1.5 Theory, evaluation and system co-design

1. **Frontier-scale generalization bounds.** Non-vacuous bounds reached nearly a billion parameters with extreme parameterization [227]; extending them to frontier models, and using them to *predict* which compression will preserve capability, would connect theory to practice.

2. **A unified rate–distortion framework with measured decode cost.** Rank, bit width, sparsity, sharing and entropy coding are usually studied separately; a single optimizer over all of them subject to measured hardware cost is missing. Precision-aware scaling laws [177] are a template.

3. **Fidelity metrics beyond accuracy.** Flip rates and distribution distances [189], knowledge-intensive suites [190] and trustworthiness suites [191] need standardization and statistical-power guidance (Example 14.4).

4. **Contamination-robust evaluation** for distilled models whose teachers are opaque [81, 82].

5. **Cost transparency.** Distillation papers rarely report teacher generation costs; community norms for accounting (Example 7.2) would make comparisons fair.

20.2 Conclusions

Four families of compression have been examined through a common rate–distortion–decode lens. The principal conclusions are as follows.

1. **Distillation is the workhorse that changes the model's size class.** On-policy objectives with dense teacher feedback have become the preferred way to do it, and they are supported by theory (imitation learning) and by cost evidence (reported 9–30× FLOP advantage over extrapolated SFT in one write-up; 1,800 versus 17,920 GPU-hours in a quoted Qwen3 comparison) [47, 52]. The dominant risks are not algorithmic but legal, safety-related and evaluative.

2. **Reasoning distillation shows how little data can unlock behaviour in a capable base** (about a thousand examples can elicit long reasoning in some settings), while also showing the limits of imitation: capacity gaps, style incompatibility, hallucination, and contamination [30, 42, 43, 46].

3. **Low-rank factorization of pretrained weights is intrinsically harder than low-rank structure in updates or activations.** Whitened formulations are the right objective (Theorem 8.1); heterogeneous allocation is optimal under an additive surrogate (Theorem 8.2); but parameter-count reductions often fail to produce wall-clock gains (Proposition 14.2). Hybrids (shared bases, sparse dictionaries, low-rank plus quantization) are the likely future of the family.

4. **Structured and tensorized operators are elegant but have not displaced dense GEMMs for LLM bodies**; they are established in embeddings and recommenders.

5. **Parameter sharing trades FLOPs for memory.** Looping and Mixture-of-Recursions show competitive small-scale results and connect to reasoning, but they require adaptive serving to realize speed and have no frontier-scale replication yet.

6. **Family-level compression (adapters, deltas, merging) is the most mature form of compression in multi-tenant serving** [167, 170, 171].

7. **Lossless coding is bounded by entropy:** about 34% for BF16 and about 10–18% for FP8, with exponent entropy near 2.5 bits derived analytically and observed empirically [200, 201, 203]. Its main virtue is exactness; its challenge is decode kernels.

8. **Compression and learning are two views of one phenomenon.** Language models are compressors, and compression yields generalization bounds that, at LLM scale, show larger models to be more compressible [211, 227].

9. **The baseline moves.** Over-training and capacity-density growth mean compressed models compete against a rising bar [174, 176].

Closing guidance

Choose the method whose savings map onto the bottleneck you actually have. If you can train, distill. If you cannot, quantize, then consider whitened low-rank or slicing, and always consider lossless coding for storage and transfer. For many variants, store deltas. Measure bytes, FLOPs and latency in your regime. Re-evaluate safety and fidelity on every compressed artifact. Treat third-party teachers as a legal and security matter. Compare against a small model trained well.

Appendix A

Notation and Glossary

A.1 Notation

Table A.1: Symbols used in the paper.

Symbol	Meaning
$p_{\mathrm{T}},\ p_\theta$	Teacher and student distributions (the student is parameterized by θ)
$\mathcal{V},\ V$	Vocabulary and its size
τ	Temperature; softmax(z/τ)
$\mathrm{KL}(p \,\|\, q)$	Kullback–Leibler divergence $\sum_x p(x) \log \frac{p(x)}{q(x)}$
$\mathrm{JS}(p, q)$	Jensen–Shannon divergence
$H(p),\ H(p, q)$	Entropy and cross-entropy
λ	Mixing coefficient (KD loss weight or on-policy fraction, as stated)
$\boldsymbol{W} \in \mathbb{R}^{m \times n}$	Weight matrix of a linear layer
$\boldsymbol{X} \in \mathbb{R}^{n \times N}$	Calibration activations (columns are inputs)
σ_i	Singular values in nonincreasing order
$r,\ r^\star$	Rank; break-even rank $mn/(m+n)$
ρ	Fraction of parameters kept; or compressed-size fraction for lossless coding (context)
S	Whitening factor, $\boldsymbol{X}\boldsymbol{X}^\top = SS^\top$ (also speedup in Chapter 14)
$\|\cdot\|_{\mathrm{F}},\ \|\cdot\|_2$	Frobenius and spectral norms
$A \otimes B$	Kronecker product
$K,\ R,\ L$	Unique blocks, recursion count, number of layers
E	Floating-point exponent field value $\lfloor \log_2 \lvert w \rvert \rfloor$
H_E	Entropy of E in bits
$b_w,\ b_Q$	Bytes per weight element; bits per quantized entry
$\Pi,\ \beta$	Peak compute (FLOP/s), memory bandwidth (B/s)
B	Batch size (tokens per step)
$N,\ D$	Parameter count; training tokens
Δ	Distortion (Chapter 2); quantizer step (Chapter 12)

A.2 Glossary

ANS Asymmetric numeral systems; an entropy coder combining the speed of Huffman coding with near-arithmetic-coding efficiency [198].

Arithmetic coding Entropy coding that maps a message to a sub-interval of $[0, 1)$ using model probabilities [197].

BF16 16-bit brain floating point: 1 sign, 8 exponent, 7 mantissa bits.

Black-box distillation
: Distillation using only teacher-generated text.

Capacity gap
: Phenomenon in which a student fails to benefit from, or is harmed by, a much stronger teacher.

CoT
: Chain of thought; intermediate reasoning text.

Delta / task vector
: Difference between fine-tuned and base weights.

Dark knowledge
: Information in the relative probabilities of incorrect classes in a teacher's output [3].

Entropy coding
: Lossless coding whose rate approaches the Shannon entropy of the source.

Forward / reverse KL
: $\mathrm{KL}(p_{\mathrm{T}} \,\|\, p_\theta)$ and $\mathrm{KL}(p_\theta \,\|\, p_{\mathrm{T}})$ respectively.

GKD
: Generalized knowledge distillation [23].

Huffman code
: Optimal symbol-by-symbol prefix code [196].

Intrinsic dimension
: Minimal dimension of a random subspace in which training reaches a target performance [118].

LoRA
: Low-rank adaptation of a weight matrix [105].

MDL
: Minimum description length principle [217].

MLA
: Multi-head latent attention, a low-rank KV cache design [104].

MoE
: Mixture of experts.

MoR
: Mixture-of-Recursions [148].

On-policy distillation (OPD)
: Distillation on sequences sampled from the student, with teacher feedback per token.

PAC-Bayes
: Generalization bounds expressed through KL between posterior and prior [228].

Prequential code length
: Sum of online log-losses, equal to the area under the training-loss curve in bits [219].

Roofline
: Performance model with memory-bound and compute-bound regimes [235].

SeqKD
: Sequence-level knowledge distillation [10].

Speculative decoding
: Draft-and-verify decoding that preserves the target distribution [77].

SVD Singular value decomposition.

TT Tensor train (matrix product state) format [129].

Whitening Transformation making calibration activations have identity second-moment matrix.

Appendix B

Selected Derivations

This appendix collects derivations that were summarized or omitted in the main text.

B.1 Policy gradient of the reverse KL (MiniLLM)

Let q_θ be the student distribution over sequences y given x, p the teacher. Write $J(\theta) = \mathrm{KL}(q_\theta \,\|\, p) = \sum_y q_\theta(y)\,[\log q_\theta(y) - \log p(y)]$. By the product rule and $\nabla q = q \nabla \log q$,

$$\begin{aligned}\nabla J &= \sum_y \nabla q_\theta(y)\,[\log q_\theta(y) - \log p(y)] + \sum_y q_\theta(y) \nabla \log q_\theta(y) \\ &= \mathbb{E}_{y\sim q_\theta}\big[(\log q_\theta(y) - \log p(y)) \nabla \log q_\theta(y)\big] + \underbrace{\mathbb{E}_{y\sim q_\theta}[\nabla \log q_\theta(y)]}_{=0}. \end{aligned} \tag{B.1}$$

The last expectation vanishes because $\sum_y \nabla q_\theta(y) = \nabla 1 = 0$. Define the reward $r(y) = \log p(y) - \log q_\theta(y)$, so $\nabla J = -\mathbb{E}[r(y) \nabla \log q_\theta(y)]$: descending J is policy-gradient *ascent* on expected reward r.

For autoregressive $q_\theta(y) = \prod_t q_\theta(y_t|y_{<t})$, $\nabla \log q_\theta(y) = \sum_t \nabla \log q_\theta(y_t|y_{<t})$. Because future tokens do not influence the past conditional's score function in expectation, one can replace the full reward by the reward-to-go $R_t = \sum_{t' \geq t} r_{t'}$ with $r_{t'} = \log p(y_{t'}|y_{<t'}) - \log q_\theta(y_{t'}|y_{<t'})$:

$$\nabla J = -\mathbb{E}_{y\sim q_\theta}\Big[\sum_t \nabla \log q_\theta(y_t|y_{<t})\, R_t\Big]. \tag{B.2}$$

Subtracting any baseline $b(y_{<t})$ from R_t leaves the expectation unchanged and reduces variance. MiniLLM's *single-step decomposition* computes the t-th immediate term $\mathbb{E}_{y_t\sim q}[\cdot]$ in closed form over the vocabulary (the per-token reverse KL), sampling only for the future reward-to-go [22].

B.2 Correctness of speculative sampling

Let p be the target and q the draft distribution at a position. Draft a token $x \sim q$; accept it with probability $\min(1, p(x)/q(x))$; if rejected, sample from the residual $p'(x) = \frac{\max(0, p(x) - q(x))}{\sum_{x'} \max(0, p(x') - q(x'))}$.

Proposition B.1. *The output token is distributed exactly as p, and the acceptance probability is $\alpha = \sum_x \min(p(x), q(x)) = 1 - \mathrm{TV}(p, q)$.*

Proof. The probability of emitting x via acceptance is $q(x)\min(1, p(x)/q(x)) = \min(q(x), p(x))$. The rejection probability is $1 - \alpha$ with $\alpha = \sum_x \min(p, q)$. Since $\sum_x \max(0, p - q) = \sum_x (p - \min(p, q)) = 1 - \alpha$, the residual satisfies $(1 - \alpha)p'(x) = \max(0, p(x) - q(x))$. Total probability of emitting x: $\min(p(x), q(x)) + \max(0, p(x) - q(x)) = p(x)$. □

With i.i.d. acceptance rate α per position and γ drafted tokens, the number of tokens produced per target pass is $1 + \sum_{i=1}^{\gamma} \alpha^i = \frac{1-\alpha^{\gamma+1}}{1-\alpha}$ (the accepted prefix plus one token from the target's residual or bonus sample), the formula used in Example 6.2.

B.3 Hoeffding–Occam bound

For a fixed hypothesis h and n i.i.d. samples with losses in $[0, 1]$, Hoeffding gives $P(L(h) - \hat{L}(h) > \epsilon) \le e^{-2n\epsilon^2}$. Weight the failure probability of h by $\delta_h = \delta\, 2^{-\ell(h)}$ with $\sum_h 2^{-\ell(h)} \le 1$ (Kraft). Setting $e^{-2n\epsilon_h^2} = \delta_h$ gives $\epsilon_h = \sqrt{(\ell(h) \ln 2 + \ln(1/\delta))/(2n)}$; a union bound over h makes all ϵ_h simultaneously valid with probability $\ge 1 - \delta$. The code length $\ell(h)$ is exactly the compressed size of the hypothesis, hence "compression implies generalization".

B.4 Entropy of a two-sided geometric exponent

Suppose $P(E = k) = c\, q^{|k|}$ on $k \in \mathbb{Z}$ with $0 < q < 1$. Normalization gives $c = (1 - q)/(1 + q)$. The mean absolute value is $\mathbb{E}|E| = 2c \sum_{k \ge 1} k q^k = 2cq/(1 - q)^2 = 2q/(1 - q^2)$. Hence in bits

$$H(E) = -\log_2 c - \mathbb{E}|E| \log_2 q = \log_2 \frac{1+q}{1-q} + \frac{2q}{1-q^2} \log_2 \frac{1}{q}. \tag{B.3}$$

At $q = 1/4$ (the Gaussian-like case $q = 2^{-\alpha}$ with $\alpha = 2$ in the notation of [203]) this gives $\log_2(5/3) + \frac{0.5}{0.9375} \cdot 2 = 0.737 + 1.067 = 1.80$ bits, within the range 1.6–2.67 quoted there for $\alpha = 2$. For the exactly Gaussian weight distribution of Example 12.1 the exponent is not two-sided geometric (the large-magnitude side decays faster than any geometric), and we computed $H(E) = 2.545$ bits numerically.

B.5 Entropy-coded scalar quantization gap

For a continuous source with differential entropy $h(X)$ and a uniform quantizer of step Δ at high resolution, the index entropy is $H(Q) \approx h(X) - \log_2 \Delta$ and the squared-error distortion is $D \approx \Delta^2/12$. The Shannon lower bound $R(D) \ge h(X) - \frac{1}{2} \log_2(2\pi e D)$ is tight for Gaussian sources. Therefore

$$H(Q) - R(D) \approx -\log_2 \Delta + \tfrac{1}{2} \log_2(2\pi e \Delta^2/12) = \tfrac{1}{2} \log_2 \frac{\pi e}{6} \approx 0.254 \text{ bits}. \tag{B.4}$$

For $\Delta = 0.4$ and unit Gaussian: $D = 0.0133$, $R(D) = \frac{1}{2} \log_2(1/0.0133) = 3.12$ bits, and the numerically computed index entropy is 3.37, a gap of 0.26 bits as in Example 12.2.

B.6 Variance of the signed hashing estimator

With $\phi(x)_k = \sum_{j:h(j)=k} \xi_j x_j$ and $⊮_{ij} = ⊮[h(i) = h(j)]$, the inner product is

$$Z = \langle \phi(x), \phi(y) \rangle = \sum_i x_i y_i + \sum_{i \ne j} \xi_i \xi_j x_i y_j ⊮_{ij}.$$

The second term has mean zero. Its variance is the double sum

$$\sum_{i\neq j}\sum_{i'\neq j'} \mathbb{E}\big[\xi_i\xi_j\xi_{i'}\xi_{j'}\mathbb{1}_{ij}\mathbb{1}_{i'j'}\big]\, x_i y_j\, x_{i'} y_{j'}.$$

The expectation is nonzero only if $\{i',j'\}=\{i,j\}$ as sets, and then it equals $P(h(i)=h(j))=1/K$. The two matchings $(i',j')=(i,j)$ and (j,i) give

$$\begin{aligned}\operatorname{Var} Z &= \frac{1}{K}\sum_{i\neq j}\big(x_i^2y_j^2 + x_iy_ix_jy_j\big)\\ &= \frac{1}{K}\Big(\,\|x\|^2\,\|y\|^2 + \langle x,y\rangle^2 - 2\sum_i x_i^2y_i^2\Big)\\ &\le \frac{1}{K}\big(\,\|x\|^2\,\|y\|^2 + \langle x,y\rangle^2\big).\end{aligned} \tag{B.5}$$

B.7 Parameter count of Monarch and tensor-train layers

Monarch with $n=b^2$: $\boldsymbol{L}$ and $\boldsymbol{R}$ each contain b blocks of size $b\times b$, i.e. $b\cdot b^2=n^{3/2}$ parameters; the total is $2n^{3/2}$. For $n=4096$ ($b=64$): $2\cdot 64^3=524{,}288$.

Tensor train for an $n\times n$ matrix reshaped as d modes with row and column sizes n_k ($\prod n_k=n$) and ranks $r_0=r_d=1$: core k has $r_{k-1}n_k^2r_k$ entries. With $d=4$, $n_k=8$, uniform inner ranks r: $64(r+r^2+r^2+r)$, which at $r=16$ equals 34,816.

B.8 Break-even in the roofline model of factorized layers

Let $t_m=b_wmn/\beta$ be the time to stream the dense weight and t_ℓ the per-kernel overhead. Dense: $T_d=t_m+t_\ell$. Factorized: $T_{lr}=\rho t_m+2t_\ell$. Then $T_{lr}<T_d$ iff $(1-\rho)t_m>t_\ell$, i.e. $\rho<1-t_\ell/t_m$. For $m=n=4096$, $b_w=2$, $\beta=3\times10^{12}$: $t_m=11.18\,\mu$s; with $t_\ell=5\,\mu$s the threshold is $\rho<0.553$, and at $\rho=0.5$: $T_d=16.18$, $T_{lr}=5.59+10=15.59$, $S=1.038$, matching Example 14.2.

Bibliography

[1] C. Buciluǎ, R. Caruana, A. Niculescu-Mizil. Model compression. *KDD*, 2006.

[2] J. Ba, R. Caruana. Do deep nets really need to be deep? *NeurIPS*, 2014.

[3] G. Hinton, O. Vinyals, J. Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[4] A. Romero et al. FitNets: Hints for thin deep nets. *ICLR*, 2015. arXiv:1412.6550.

[5] S. Zagoruyko, N. Komodakis. Paying more attention to attention: improving CNN performance via attention transfer. *ICLR*, 2017.

[6] T. Furlanello et al. Born again neural networks. *ICML*, 2018.

[7] H. Mobahi, M. Farajtabar, P. Bartlett. Self-distillation amplifies regularization in Hilbert space. *NeurIPS*, 2020.

[8] J. H. Cho, B. Hariharan. On the efficacy of knowledge distillation. *ICCV*, 2019.

[9] S. I. Mirzadeh et al. Improved knowledge distillation via teacher assistant. *AAAI*, 2020.

[10] Y. Kim, A. M. Rush. Sequence-level knowledge distillation. *EMNLP*, 2016.

[11] V. Sanh, L. Debut, J. Chaumond, T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108, 2019.

[12] X. Jiao et al. TinyBERT: Distilling BERT for natural language understanding. *Findings of EMNLP*, 2020.

[13] Z. Sun et al. MobileBERT: a compact task-agnostic BERT for resource-limited devices. *ACL*, 2020.

[14] W. Wang et al. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *NeurIPS*, 2020.

[15] A. K. Menon et al. A statistical perspective on distillation. *ICML*, 2021.

[16] D. Lopez-Paz, L. Bottou, B. Schölkopf, V. Vapnik. Unifying distillation and privileged information. *ICLR*, 2016.

[17] M. Phuong, C. Lampert. Towards understanding knowledge distillation. *ICML*, 2019.

[18] S. Stanton et al. Does knowledge distillation really work? *NeurIPS*, 2021.

[19] L. Zhang et al. Be your own teacher: improve the performance of convolutional neural networks via self distillation. *ICCV*, 2019.

[20] S. Ross, G. Gordon, J. A. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *AISTATS*, 2011.

[21] M. Ranzato et al. Sequence level training with recurrent neural networks. *ICLR*, 2016.

[22] Y. Gu, L. Dong, F. Wei, M. Huang. MiniLLM: Knowledge distillation of large language models. *ICLR*, 2024. arXiv:2306.08543.

[23] R. Agarwal et al. On-policy distillation of language models: learning from self-generated mistakes. *ICLR*, 2024. arXiv:2306.13649.

[24] J. Ko et al. DistiLLM: Towards streamlined distillation for large language models. *ICML*, 2024.

[25] Y. Wen, Z. Li, W. Du, L. Mou. f-divergence minimization for sequence-level knowledge distillation. *ACL*, 2023.

[26] R. Taori et al. Stanford Alpaca: an instruction-following LLaMA model. GitHub repository, 2023.

[27] Y. Wang et al. Self-Instruct: Aligning language models with self-generated instructions. *ACL*, 2023.

[28] S. Mukherjee et al. Orca: Progressive learning from complex explanation traces of GPT-4. arXiv:2306.02707, 2023.

[29] A. Mitra et al. Orca 2: Teaching small language models how to reason. arXiv:2311.11045, 2023.

[30] A. Gudibande et al. The false promise of imitating proprietary LLMs. arXiv:2305.15717, 2023.

[31] C.-Y. Hsieh et al. Distilling step-by-step! Outperforming larger language models with less training data and smaller model sizes. *Findings of ACL*, 2023.

[32] N. Ho, L. Schmid, S.-Y. Yun. Large language models are reasoning teachers. *ACL*, 2023.

[33] L. C. Magister et al. Teaching small language models to reason. *ACL (short)*, 2023.

[34] Y. Fu et al. Specializing smaller language models towards multi-step reasoning. *ICML*, 2023.

[35] L. Yu et al. MetaMath: Bootstrap your own mathematical questions for large language models. *ICLR*, 2024.

[36] S. Gunasekar et al. Textbooks are all you need. arXiv:2306.11644, 2023.

[37] C. Zhou et al. LIMA: Less is more for alignment. *NeurIPS*, 2023.

[38] M. Xia et al. LESS: Selecting influential data for targeted instruction tuning. *ICML*, 2024.

[39] L. Chen et al. AlpaGasus: Training a better Alpaca with fewer data. *ICLR*, 2024.

[40] W. Liu et al. What makes good data for alignment? A comprehensive study of automatic data selection in instruction tuning (DEITA). *ICLR*, 2024.

[41] DeepSeek-AI. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv:2501.12948, 2025.

[42] N. Muennighoff et al. s1: Simple test-time scaling. arXiv:2501.19393, 2025.

[43] Y. Ye et al. LIMO: Less is more for reasoning. arXiv:2502.03387, 2025.

[44] E. Guha et al. OpenThoughts: Data recipes for reasoning models. arXiv:2506.04178, 2025.

[45] M. Abdin et al. Phi-4-reasoning technical report. arXiv:2504.21318, 2025.

[46] Y. Li et al. Small models struggle to learn from strong reasoners. arXiv:2502.12143, 2025.

[47] A. Yang et al. Qwen3 technical report. arXiv:2505.09388, 2025.

[48] Gemma Team. Gemma 2: Improving open language models at a practical size. arXiv:2408.00118, 2024.

[49] Gemma Team. Gemma 3 technical report. arXiv:2503.19786, 2025.

[50] A. Dubey et al. The Llama 3 herd of models. arXiv:2407.21783, 2024.

[51] Meta AI. Llama 3.2: Revolutionizing edge AI and vision with open, customizable models. Company blog post, September 2024.

[52] K. Lu, Thinking Machines Lab. On-policy distillation. Blog post, thinkingmachines.ai, 27 October 2025.

[53] D. Busbridge et al. Distillation scaling laws. *ICML*, 2025. arXiv:2502.08606.

[54] S. Zhao, Z. Xie, M. Liu, J. Huang, G. Pang, F. Chen, A. Grover. Self-distilled reasoner: On-policy self-distillation for large language models. arXiv:2601.18734, 2026.

[55] J. Ko, S. Abdali, Y. J. Kim, T. Chen, P. Cameron. Scaling reasoning efficiently via relaxed on-policy distillation. arXiv:2603.11137, 2026.

[56] D. Zhang et al. Fast and effective on-policy distillation from reasoning prefixes. arXiv:2602.15260, 2026.

[57] A. Zhao et al. Decoupling KL and trajectories: A unified perspective for SFT, DAgger, offline RL, and OPD in LLM distillation. arXiv:2605.16826, 2026.

[58] Z. Shen et al. On the geometry of on-policy distillation. arXiv:2606.07082, 2026.

[59] Y. Li et al. Rethinking on-policy distillation of large language models: phenomenology, mechanism, and recipe. arXiv:2604.13016, 2026.

[60] A. Bercovich et al. Puzzle: Distillation-based NAS for inference-optimized LLMs. arXiv:2411.19146, 2024.

[61] S. Muralidharan et al. Compact language models via pruning and knowledge distillation (Minitron). *NeurIPS*, 2024. arXiv:2407.14679.

[62] S. Sreenivas et al. LLM pruning and distillation in practice: the Minitron approach. arXiv:2408.11796, 2024.

[63] Anthropic. Company blog post alleging industrial-scale distillation campaigns against Claude by DeepSeek, Moonshot AI and MiniMax, 23 February 2026; figures as reported by The Register, 24 February 2026.

[64] A. Cloud et al. Subliminal learning: Language models transmit behavioral traits via hidden signals in data. arXiv:2507.14805, 2025.

[65] I. Shumailov et al. AI models collapse when trained on recursively generated data. *Nature* 631, 755–759, 2024.

[66] E. Dohmatob, Y. Feng, J. Kempe. Model collapse demystified: the case of regression. *NeurIPS*, 2024.

[67] J. Kirchenbauer et al. A watermark for large language models. *ICML*, 2023.

[68] T. Sander et al. Watermarking makes language models radioactive. *NeurIPS*, 2024.

[69] N. Carlini et al. Stealing part of a production language model. *ICML*, 2024.

[70] F. Tramèr et al. Stealing machine learning models via prediction APIs. *USENIX Security*, 2016.

[71] K. Krishna et al. Thieves on Sesame Street! Model extraction of BERT-based APIs. *ICLR*, 2020.

[72] N. Boizard et al. Towards cross-tokenizer distillation: the universal logit distillation loss for LLMs. *TMLR*, 2025. arXiv:2402.12030.

[73] F. Wan et al. Knowledge fusion of large language models (FuseLLM). *ICLR*, 2024.

[74] S. Zhang et al. Dual-space knowledge distillation for large language models. *EMNLP*, 2024.

[75] B. Minixhofer, I. Vulić, E. M. Ponti. Cross-tokenizer distillation via approximate likelihood matching. arXiv:2503.20083, 2025.

[76] Y. Zhou et al. DistillSpec: Improving speculative decoding via knowledge distillation. *ICLR*, 2024.

[77] Y. Leviathan, M. Kalman, Y. Matias. Fast inference from transformers via speculative decoding. *ICML*, 2023.

[78] C. Chen et al. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318, 2023.

[79] C. Snell, D. Klein, R. Zhong. Learning by distilling context. arXiv:2209.15189, 2022.

[80] Z. Yang et al. Self-distillation bridges distribution gap in language model fine-tuning. *ACL*, 2024.

[81] H. Zhang et al. A careful examination of large language model performance on grade school arithmetic (GSM1k). *NeurIPS Datasets & Benchmarks*, 2024.

[82] Y. Oren et al. Proving test set contamination in black-box language models. *ICLR*, 2024.

[83] X. Qi et al. Fine-tuning aligned language models compromises safety, even when users do not intend to. *ICLR*, 2024.

[84] J. Betley et al. Emergent misalignment: Narrow finetuning can produce broadly misaligned LLMs. *ICML*, 2025.

[85] Z. Gekhman et al. Does fine-tuning LLMs on new knowledge encourage hallucinations? *EMNLP*, 2024.

[86] C. Eckart, G. Young. The approximation of one matrix by another of lower rank. *Psychometrika* 1, 211–218, 1936.

[87] L. Mirsky. Symmetric gauge functions and unitarily invariant norms. *Quart. J. Math.* 11, 50–59, 1960.

[88] N. Halko, P.-G. Martinsson, J. A. Tropp. Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review* 53(2), 2011.

[89] E. J. Candès, X. Li, Y. Ma, J. Wright. Robust principal component analysis? *J. ACM* 58(3), 2011.

[90] T. N. Sainath et al. Low-rank matrix factorization for deep neural network training with high-dimensional output targets. *ICASSP*, 2013.

[91] E. Denton et al. Exploiting linear structure within convolutional networks for efficient evaluation. *NeurIPS*, 2014.

[92] M. Jaderberg, A. Vedaldi, A. Zisserman. Speeding up convolutional neural networks with low rank expansions. *BMVC*, 2014.

[93] Y.-C. Hsu et al. Language model compression with weighted low-rank factorization (FWSVD). *ICLR*, 2022.

[94] Z. Yuan et al. ASVD: Activation-aware singular value decomposition for compressing large language models. arXiv:2312.05821, 2023.

[95] X. Wang et al. SVD-LLM: Truncation-aware singular value decomposition for large language model compression. *ICLR*, 2025. arXiv:2403.07378.

[96] X. Wang, S. Alam, Z. Wan, H. Shen, M. Zhang. SVD-LLM V2: Optimizing singular value truncation for large language model compression. *NAACL*, 2025. arXiv:2503.12340.

[97] Q. Wang et al. Dobi-SVD: Differentiable SVD for LLM compression and some new perspectives. *ICLR*, 2025. arXiv:2502.02723.

[98] J. Wang et al. Basis sharing: Cross-layer parameter sharing for large language model compression. *ICLR*, 2025.

[99] D. Makhov et al. CoSpaDi: Compressing LLMs via calibration-guided sparse dictionary learning. arXiv:2509.22075, 2025.

[100] S. Ashkboos et al. SliceGPT: Compress large language models by deleting rows and columns. *ICLR*, 2024.

[101] P. Sharma, J. T. Ash, D. Misra. The truth is in there: Improving reasoning in language models with layer-selective rank reduction (LASER). *ICLR*, 2024.

[102] H. Yu, J. Wu. Compressing transformers: features are low-rank, but weights are not! *AAAI*, 2023.

[103] C.-C. Chang et al. Palu: Compressing KV-cache with low-rank projection. arXiv:2407.21118, 2024.

[104] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. arXiv:2405.04434, 2024.

[105] E. J. Hu et al. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022. arXiv:2106.09685.

[106] T. Dettmers et al. QLoRA: Efficient finetuning of quantized LLMs. *NeurIPS*, 2023.

[107] Y. Li et al. LoftQ: LoRA-fine-tuning-aware quantization for large language models. *ICLR*, 2024.

[108] H. Guo et al. LQ-LoRA: Low-rank plus quantized matrix decomposition for efficient language model finetuning. *ICLR*, 2024.

[109] R. Saha, N. Sagan, V. Srivastava, A. Goldsmith, M. Pilanci. Compressing large language models using low rank and low precision decomposition (CALDERA). *NeurIPS*, 2024. arXiv:2405.18886.

[110] Y. Li et al. LoSparse: Structured compression of large language models based on low-rank and sparse approximation. *ICML*, 2023.

[111] S. Zhang, V. Papyan. OATS: Outlier-aware pruning through sparse and low rank decomposition. *ICLR*, 2025.

[112] Q. Zhang et al. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. *ICLR*, 2023.

[113] J. Zhao et al. GaLore: Memory-efficient LLM training by gradient low-rank projection. *ICML*, 2024.

[114] D. Biderman et al. LoRA learns less and forgets less. *TMLR*, 2024.

[115] J. Schulman, Thinking Machines Lab. LoRA without regret. Blog post, thinkingmachines.ai, 2025.

[116] N. Houlsby et al. Parameter-efficient transfer learning for NLP. *ICML*, 2019.

[117] A. Aghajanyan, S. Gupta, L. Zettlemoyer. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. *ACL*, 2021.

[118] C. Li, H. Farkhoor, R. Liu, J. Yosinski. Measuring the intrinsic dimension of objective landscapes. *ICLR*, 2018.

[119] T. Dao et al. Monarch: Expressive structured matrices for efficient and accurate training. *ICML*, 2022. arXiv:2204.00595.

[120] T. Dao, A. Gu, M. Eichhorn, A. Rudra, C. Ré. Learning fast algorithms for linear transforms using butterfly factorizations. *ICML*, 2019.

[121] T. Dao et al. Kaleidoscope: An efficient, learnable representation for all structured linear maps. *ICLR*, 2021.

[122] B. Chen et al. Pixelated butterfly: Simple and efficient sparse training for neural network models. *ICLR*, 2022.

[123] Q. Le, T. Sarlós, A. Smola. Fastfood: approximating kernel expansions in loglinear time. *ICML*, 2013.

[124] Y. Cheng et al. An exploration of parameter redundancy in deep networks with circulant projections. *ICCV*, 2015.

[125] V. Sindhwani, T. Sainath, S. Kumar. Structured transforms for small-footprint deep learning. *NeurIPS*, 2015.

[126] U. Thakker et al. Compressing RNNs for IoT devices by 15–38x using Kronecker products. arXiv:1906.02876, 2019.

[127] A. Edalati et al. KronA: Parameter efficient tuning with Kronecker adapter. arXiv:2212.10650, 2022.

[128] A. Novikov et al. Tensorizing neural networks. *NeurIPS*, 2015. arXiv:1509.06569.

[129] I. V. Oseledets. Tensor-train decomposition. *SIAM J. Sci. Comput.* 33(5), 2011.

[130] T. G. Kolda, B. W. Bader. Tensor decompositions and applications. *SIAM Review* 51(3), 2009.

[131] V. Lebedev et al. Speeding-up convolutional neural networks using fine-tuned CP-decomposition. *ICLR*, 2015.

[132] Y.-D. Kim et al. Compression of deep convolutional neural networks for fast and low power mobile applications. *ICLR*, 2016.

[133] Z.-F. Gao et al. Compressing deep neural networks by matrix product operators. *Physical Review Research* 2, 023300, 2020.

[134] C. Yin et al. TT-Rec: Tensor train compression for deep learning recommendation models. *MLSys*, 2021.

[135] V. Khrulkov et al. Tensorized embedding layers for efficient model compression. arXiv:1901.10787, 2019.

[136] W. Chen et al. Compressing neural networks with the hashing trick. *ICML*, 2015. arXiv:1504.04788.

[137] K. Weinberger et al. Feature hashing for large scale multitask learning. *ICML*, 2009.

[138] D. Svenstrup, J. M. Hansen, O. Winther. Hash embeddings for efficient word representations. *NeurIPS*, 2017.

[139] H.-J. M. Shi et al. Compositional embeddings using complementary partitions for memory-efficient recommendation systems. *KDD*, 2020.

[140] Z. Lan et al. ALBERT: A lite BERT for self-supervised learning of language representations. *ICLR*, 2020. arXiv:1909.11942.

[141] C. Tao et al. Scaling laws with vocabulary: larger models deserve larger vocabularies. *NeurIPS*, 2024.

[142] O. Press, L. Wolf. Using the output embedding to improve language models. *EACL*, 2017.

[143] H. Inan, K. Khosravi, R. Socher. Tying word vectors and word classifiers: A loss framework for language modeling. *ICLR*, 2017.

[144] M. Dehghani et al. Universal transformers. *ICLR*, 2019. arXiv:1807.03819.

[145] S. Bai, J. Z. Kolter, V. Koltun. Deep equilibrium models. *NeurIPS*, 2019.

[146] S. Takase, S. Kiyono. Lessons on parameter sharing across layers in transformers. *SustaiNLP Workshop*, 2023 (arXiv 2021).

[147] S. Bae, A. Fisch, H. Harutyunyan, Z. Ji, S. Kim, T. Schuster. Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA. *ICLR*, 2025. arXiv:2410.20672.

[148] S. Bae et al. Mixture-of-Recursions: Learning dynamic recursive depths for adaptive token-level computation. arXiv:2507.10524, 2025.

[149] R.-J. Zhu et al. Scaling latent reasoning via looped language models (Ouro). arXiv:2510.25741, 2025.

[150] J. Geiping et al. Scaling up test-time compute with latent reasoning: a recurrent depth approach. arXiv:2502.05171, 2025.

[151] N. Saunshi et al. Reasoning with latent thoughts: On the power of looped transformers. *ICLR*, 2025.

[152] A. Giannou et al. Looped transformers as programmable computers. *ICML*, 2023.

[153] L. Yang et al. Looped transformers are better at learning learning algorithms. *ICLR*, 2024.

[154] Z. Liu et al. MobileLLM: Optimizing sub-billion parameter language models for on-device use cases. *ICML*, 2024.

[155] D. Raposo et al. Mixture-of-Depths: Dynamically allocating compute in transformer-based language models. arXiv:2404.02258, 2024.

[156] W. Fedus, B. Zoph, N. Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR* 23, 2022.

[157] A. Jiang et al. Mixtral of experts. arXiv:2401.04088, 2024.

[158] D. Dai et al. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. *ACL*, 2024.

[159] A. Clark et al. Unified scaling laws for routed language models. *ICML*, 2022.

[160] M. Wortsman et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. *ICML*, 2022. arXiv:2203.05482.

[161] G. Ilharco et al. Editing models with task arithmetic. *ICLR*, 2023. arXiv:2212.04089.

[162] P. Yadav et al. TIES-Merging: Resolving interference when merging models. *NeurIPS*, 2023.

[163] L. Yu et al. Language models are Super Mario: Absorbing abilities from homologous models as a free lunch (DARE). *ICML*, 2024.

[164] M. Matena, C. Raffel. Merging models with Fisher-weighted averaging. *NeurIPS*, 2022.

[165] E. Yang et al. Model merging in LLMs, MLLMs, and beyond: methods, theories, applications and opportunities. arXiv:2408.07666, 2024.

[166] C. Goddard et al. Arcee's MergeKit: A toolkit for merging large language models. *EMNLP Industry*, 2024.

[167] J. Liu et al. BitDelta: Your fine-tune may only be worth one bit. *NeurIPS*, 2024. arXiv:2402.10193.

[168] B. Ping et al. Delta-CoMe: Training-free delta-compression with mixed-precision for large language models. *NeurIPS*, 2024.

[169] X. Yao, A. Klimovic. DeltaZip: Efficient serving of multiple full-model-tuned LLMs. arXiv:2312.05215, 2023.

[170] L. Chen et al. Punica: Multi-tenant LoRA serving. *MLSys*, 2024.

[171] Y. Sheng et al. S-LoRA: Serving thousands of concurrent LoRA adapters. *MLSys*, 2024. arXiv:2311.03285.

[172] J. Kaplan et al. Scaling laws for neural language models. arXiv:2001.08361, 2020.

[173] J. Hoffmann et al. Training compute-optimal large language models (Chinchilla). arXiv:2203.15556, 2022.

[174] N. Sardana, J. Portes, S. Doubov, J. Frankle. Beyond Chinchilla-optimal: Accounting for inference in language model scaling laws. *ICML*, 2024.

[175] S. Y. Gadre et al. Language models scale reliably with over-training and on downstream tasks. arXiv:2403.08540, 2024.

[176] C. Xiao et al. Densing law of LLMs. arXiv:2412.04315, 2024 (*Nature Machine Intelligence*, 2025).

[177] T. Kumar et al. Scaling laws for precision. *ICLR*, 2025.

[178] P. Zhang et al. TinyLlama: An open-source small language model. arXiv:2401.02385, 2024.

[179] L. B. Allal et al. SmolLM2: When Smol goes big — data-centric training of a small language model. arXiv:2502.02737, 2025.

[180] M. Abdin et al. Phi-3 technical report: A highly capable language model locally on your phone. arXiv:2404.14219, 2024.

[181] H. Cai et al. Once-for-all: Train one network and specialize it for efficient deployment. *ICLR*, 2020.

[182] H. Wang et al. HAT: Hardware-aware transformers for efficient natural language processing. *ACL*, 2020.

[183] M. Tan et al. MnasNet: Platform-aware neural architecture search for mobile. *CVPR*, 2019.

[184] D. So et al. Primer: Searching for efficient transformers for language modeling. *NeurIPS*, 2021.

[185] S. Han, H. Mao, W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *ICLR*, 2016. arXiv:1510.00149.

[186] E. Frantar et al. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *ICLR*, 2023.

[187] E. Frantar, D. Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. *ICML*, 2023.

[188] J. Lin et al. AWQ: Activation-aware weight quantization for LLM compression and acceleration. *MLSys*, 2024.

[189] A. Dutta et al. Accuracy is not all you need. *NeurIPS*, 2024.

[190] A. Jaiswal et al. Compressing LLMs: The truth is rarely pure and never simple. *ICLR*, 2024.

[191] J. Hong et al. Decoding compressed trust: Scrutinizing the trustworthiness of efficient LLMs under compression. *ICML*, 2024.

[192] L. Gao et al. A framework for few-shot language model evaluation (lm-evaluation-harness). Zenodo, 2021.

[193] P. Liang et al. Holistic evaluation of language models. *TMLR*, 2023.

[194] L. Zheng et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS Datasets & Benchmarks*, 2023.

[195] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal* 27, 1948.

[196] D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proc. IRE* 40(9), 1952.

[197] I. H. Witten, R. M. Neal, J. G. Cleary. Arithmetic coding for data compression. *Commun. ACM* 30(6), 1987.

[198] J. Duda. Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding. arXiv:1311.2540, 2013.

[199] T. M. Cover, J. A. Thomas. *Elements of Information Theory*, 2nd ed. Wiley, 2006.

[200] T. Zhang et al. 70% size, 100% accuracy: Lossless LLM compression for efficient GPU inference via dynamic-length float (DFloat11). arXiv:2504.11651, 2025.

[201] M. Hershcovitch et al. ZipNN: Lossless compression for AI models. arXiv:2411.05239, 2024.

[202] Y. Hao, Y. Cao, L. Mou. NeuZip: Memory-efficient training and inference with dynamic compression of neural networks. arXiv:2410.20650, 2024.

[203] Z. Yang et al. To compress or not? Pushing the frontier of lossless GenAI model weights compression with exponent concentration. arXiv:2510.02676, 2025.

[204] R. Fan et al. ZipServ: Fast and memory-efficient LLM inference with hardware-aware lossless compression. arXiv:2603.17435, 2026.

[205] A. Heilper, D. Singer. Lossless compression of neural network components: weights, checkpoints, and K/V caches in low-precision formats. arXiv:2508.19263, 2025.

[206] P. Lindstrom, M. Isenburg. Fast and efficient compression of floating-point data. *IEEE TVCG* 12(5), 2006.

[207] T. Pelkonen et al. Gorilla: A fast, scalable, in-memory time series database. *VLDB*, 2015.

[208] A. Eisenman et al. Check-N-Run: A checkpointing system for training deep learning recommendation models. *NSDI*, 2022.

[209] W. Li et al. ExCP: Extreme LLM checkpoint compression via weight-momentum joint shrinking. *ICML*, 2024.

[210] A. Gersho, R. M. Gray. *Vector Quantization and Signal Compression*. Kluwer, 1992.

[211] G. Delétang et al. Language modeling is compression. *ICLR*, 2024. arXiv:2309.10668.

[212] C. S. K. Valmeekam et al. LLMZip: Lossless text compression using large language models. arXiv:2306.04050, 2023.

[213] F. Mittu et al. FineZip: Pushing the limits of large language models for practical lossless text compression. arXiv:2409.17141, 2024.

[214] Y. Huang et al. Compression represents intelligence linearly. *COLM*, 2024.

[215] Z. Jiang et al. "Low-resource" text classification: a parameter-free classification method with compressors. *Findings of ACL*, 2023.

[216] J. Rissanen. Modeling by shortest data description. *Automatica* 14, 1978.

[217] P. D. Grünwald. *The Minimum Description Length Principle*. MIT Press, 2007.

[218] G. E. Hinton, D. van Camp. Keeping neural networks simple by minimizing the description length of the weights. *COLT*, 1993.

[219] L. Blier, Y. Ollivier. The description length of deep learning models. *NeurIPS*, 2018.

[220] R. J. Solomonoff. A formal theory of inductive inference. *Information and Control* 7, 1964.

[221] M. Li, P. Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*, 3rd ed. Springer, 2008.

[222] M. Hutter. *Universal Artificial Intelligence*. Springer, 2005.

[223] G. K. Dziugaite, D. M. Roy. Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data. *UAI*, 2017.

[224] S. Arora et al. Stronger generalization bounds for deep nets via a compression approach. *ICML*, 2018.

[225] W. Zhou et al. Non-vacuous generalization bounds at the ImageNet scale: a PAC-Bayesian compression approach. *ICLR*, 2019.

[226] S. Lotfi et al. PAC-Bayes compression bounds so tight that they can explain generalization. *NeurIPS*, 2022.

[227] S. Lotfi et al. Non-vacuous generalization bounds for large language models. *ICML*, 2024.

[228] D. A. McAllester. Some PAC-Bayesian theorems. *Machine Learning* 37, 1999.

[229] N. Tishby, N. Zaslavsky. Deep learning and the information bottleneck principle. *ITW*, 2015.

[230] R. Shwartz-Ziv, N. Tishby. Opening the black box of deep neural networks via information. arXiv:1703.00810, 2017.

[231] A. M. Saxe et al. On the information bottleneck theory of deep learning. *ICLR*, 2018.

[232] E. Elmoznino, T. Marty, et al. In-context learning and Occam's razor. arXiv:2410.14086, 2024.

[233] T. Dao et al. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.

[234] E. Frantar et al. MARLIN: Mixed-precision auto-regressive parallel inference on large language models. arXiv:2408.11743, 2024.

[235] S. Williams, A. Waterman, D. Patterson. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52(4), 2009.

[236] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. *SOSP*, 2023.

[237] R. Pope et al. Efficiently scaling transformer inference. *MLSys*, 2023.